
TensorGuard: Refinement-Type Verification of `nn.Module` Shapes and Gradient Flow with a Lean-Audited Operator-Rule Table

Anonymous Author(s)

Affiliation

Address

email

Abstract

TENSORGUARD is a no-execution refinement-type checker for PyTorch `nn.Module` forward methods that verifies tensor shapes and gradient flow statically from class source, without instantiating or tracing the module. On a curated 60-bug historical corpus it returns REFUTED-PROOF on 53/60 (88.3%); on the $N=34$ fragment-fair head-to-head with Pytea it scores 32/34 vs. 25/34 (McNemar exact $p=0.0156$); and it returns REFUTED-PROOF on 9/9 naturally-occurring HuggingFace shape bugs across Llama, Qwen2, Mistral, Phi-3, and Gemma 2. On unreduced real-source class code the same verdict regime returns 0/488 unconditional REFUTED-PROOF on the unrestricted block corpus and 26/356 on its empty-*assume_M* subset, of which 5 fire inside the audited handler footprint. The composition theorem is mechanised in Lean 4 on a 17-operator DSL with 17/17 closed per-operator soundness lemmas, the operator-rule table by 36 closed `applyOp_sound_*` lemmas, and the soundness theorem is restricted to a 49-handler sub-catalogue `Catsound` (36 Lean-audited + 13 pen-and-paper). On the 488-block real-source corpus 128/185 in-soundness verdicts (69.2%) lie wholly inside this audited footprint, with only 12/185 touching any tested-only handler. The remaining 34 handlers, the analyser implementation, the AST extractor, the backward verifier, and the Z3 dispatch lie in the trusted computing base; on the 371-row VERIFIED tied-weight subpopulation a proportionally-allocated stratified resample ($n=83$, 8 handler families) finds 2/47 silently-incorrect verdicts (Wilson 95% upper $\leq 8.37\%$), and the regex-screened deployment-side product bound on the worst-case construct family is $\leq 3.0\%$. An open-source implementation accompanies the paper.

1 Introduction

Tensor programs in PyTorch fail in two characteristic ways that unit tests rarely catch: (i) the forward pass crashes deep inside an autograd-traced subgraph because two operand shapes disagree under a particular batch size, sequence length, or image resolution; and (ii) the backward pass silently produces a wrong gradient — e.g. a parameter receives `p.grad=None` after `loss.backward()` because an in-place op or a `.detach()` broke the autograd tape along one branch, and `optimizer.step()` silently skips it.

The PyTorch ecosystem responds to these failures with *execution-based* tools: `torch.fx+ShapeProp` captures one trace, `FakeTensorMode` executes shape arithmetic on metadata-only tensors, `torch.export` specialises a trace, and `TorchDynamo` installs runtime guards that recompile when an invariant fails. All four require the model to be importable, instantiable, and traceable on the target machine. On modern HuggingFace/timm code that is typically *not* the case: a transformer block needs a

36 config object, an attention head reads `self.config.hidden_size`, and an `nn.ModuleList` over
 37 `config.num_layers` blocks demands a fully-realised configuration before a single tensor can flow.
 38 TENSORGUARD takes the opposite stance. Given an `nn.Module` class source, with no instanti-
 39 ation and no tracer, it computes a refinement-typed signature $\text{Tensor}\{s, g \mid \varphi\}$ for every value
 40 flowing through forward, where s is a symbolic shape, g is a static gradient-required flag, and φ
 41 is a Z3-decidable refinement constraining both. The ABSTAIN verdict is reported as a first-class
 42 outcome whenever a subgraph falls outside the verified fragment, so the analyser never silently
 43 claims verification on constructs it cannot reason about. A separately scoped exploratory result
 44 (Theorem 13) gives a one-directional inclusion relating these refinement variables to the metadata
 45 reads that TORCHDYNAMO issues on the matching `aten` call; the inclusion is necessary-direction
 46 only, audited on 17 modules, and is not claimed as a runtime equivalence.

47 **Contributions.** (C1) A refinement-typed calculus $\text{Tensor}\{s, g \mid \varphi\}$ that unifies symbolic shape
 48 and static gradient-flag analysis under one Z3 procedure (Section 2), with classical Preservation/Progress (Theorems 18 and 19). The calculus itself reorganises constraint-based shape
 49 typing rather than inventing it; the distinctive substance is its joint shape-plus-grad layer
 50 (C3), its un-instantiated class-source regime described above, and the audit and calibration
 51 package (C5–C6) that partitions every verdict by handler-soundness scope. (C2) An as-
 52 sume/guarantee discipline at the `nn.Module` class boundary, with a contravariant/covariant sub-
 53 classing rule (Section 2.2); the soundness of the composition rule (`ag_composition_ext`) is
 54 mechanised in Lean 4 on a 17-operator DSL (`matmul`, `view`, `add/broadcast-add`, `linear`,
 55 `transpose`, `permute`, `relu`, `cat`, `sum_reduce`, `mean_reduce`, `expand`, `gather`, `embedding`,
 56 `conv2d`, `einsum`, `unbind`, `reshape`); the composition theorem is operator-agnostic over
 57 any per-operator `applyOp_sound_*` witness, so the four operators that fire on the post-
 58 freeze real-PR catches (`view/reshape` as a non-trivial shape rule, `conv2d`, `einsum`, plus
 59 `unbind`) are now *inside* the mechanised composition fragment with their own soundness lem-
 60 mas (`applyOpExt_sound_view`, `applyOpExt_sound_reshape`, `applyOpExt_sound_conv2d`,
 61 `applyOpExt_sound_einsum`, `applyOpExt_sound_unbind`, all sorry-free under `lake build`).
 62 The algorithmic discipline is implemented over the full 79-handler catalogue; the remaining 62
 63 handlers sit under per-rule lemmas or under tested-only soundness scope, as Section 6 discusses.
 64 (C3) An autograd-aware backward verifier catching three canonical silent-zero-grad bug classes
 65 statically (Section 3.2); 500/500 static $\leftrightarrow$ runtime agreement, 8/8 canonical bugs caught, 0/50
 66 false positives. The lattice is first-order ($\{\text{has_grad}, \text{no_grad}, \top\}$) and is silently incorrect on
 67 parameter-sharing-under-renamed-attribute (prevalence $\leq 12\%$ of training scripts; Section 6);
 68 we flag this as a substantive caveat, not a footnote. (C4) An *exploratory Dynamo-guard inclu-*
 69 *sion lemma* (Theorem 13, necessary direction only): every TG refinement variable is, on the
 70 matching `aten` call, a metadata read TORCHDYNAMO’s specialiser issues on a `FakeTensor`.
 71 The mapping is shipped as data (28 rows; mechanically rechecked against the imported torch
 72 — 28/28 pass under torch 2.9.1). The lemma is empirically audited on a 17-module corpus
 73 drawn from `torchvision`, `transformers`, and `timm`; on 16 of the 17 modules the documented
 74 forward-signature contract (equivalent to what TG would emit after CEGAR closure) is used
 75 as the assumed contract, and the remaining row is a TG-verified surrogate. We accordingly
 76 scope C4 as an *exploratory* result. We do *not* claim a two-directional correspondence. (C5)
 77 A 488-block + 60-bug benchmark with calibrated reporting under a five-way verdict taxonomy
 78 per tool (Section 4.1); a per-feature stress benchmark on 25 targeted cases (Table 5) reporting
 79 that, on the hand-designed 25-case stress benchmark, only three knobs — **device-consistency**,
 80 **gradient-flow**, and **low-confidence gating** — move verdicts (the corresponding ablation on the
 81 real corpora is a flat line, with none of the five knobs flipping a verdict; Section 4.1); a 25-block
 82 hybrid-mode falsification corpus (Table 4); and an AST-skeleton diversity analysis ($K_{\text{ast}}=406$ on
 83 the 488 blocks). The contribution is restricted to those three discriminative knobs; the unused
 84 CEGAR loop and the always-satisfiable phase encoder ship with the analyser but are not claimed
 85 as contributions. The localisation tracer is reported as engineering rather than as a research result.
 86 (C6) A Lean 4 *operator-rule audit* of the **shape-transfer rule table**: 28 shape-transfer rules
 87 with 11/11 previously-axiomatic soundness lemmas closed sorry-free, the full operator-rule tree
 88 building sorry-free under `lake build`, and 28,000/28,000 in-envelope samples agreeing with
 89 torch 2.9.1 (Section 4.4; Table 9). *The rule table is audited; the analyser implementation, AST*
 90 *extractor, and backward verifier are not audited.* On the 488-block real-source corpus, 33/57
 91 VERIFIED verdicts and 95/128 CONTRACT-VIOLATION verdicts touch *only* handlers in the Lean-

93 or-pen-paper audited footprint (128/185 in-soundness verdicts in total, 69.2%); the remaining
 94 verdicts split into 12/185 touching at least one of the 34 tested-only handlers and 45/185 touching
 95 only handlers outside any of the three soundness scopes (a TCB obligation explicitly tracked,
 96 Section 4.4).

97 2 Refinement-Typed Calculus and Module Composition

98 We formalise TENSORGUARD’s abstract domain as a refinement-type system over a `torch.fx`-style
 99 IR. The type of a tensor is

$$\text{Tensor}[\tau, \varphi_{\text{shape}}(d_1, \dots, d_k), \varphi_{\text{grad}}],$$

100 where τ is a dtype, φ_{shape} is a quantifier-free predicate over symbolic dim variables in $\mathcal{T}_{\text{shape}} = \text{LIA} \cup$
 101 $\text{Div} \cup \text{BMul}$ (linear integer arithmetic, divisibility, and bounded nonlinear multiplication restricted
 102 to syntactically-known factors), and the grad refinement $\varphi_{\text{grad}} \in \{\text{has_grad}, \text{no_grad}, \top\}$ abstracts
 103 `requires_grad` $\wedge$ `is_on_tape`. The grad lattice is flat: `has_grad` and `no_grad` are incomparable,
 104 joined to $\top$. A typing context Γ binds variables to refinement types, accumulates path predicates, and
 105 tracks a grad-context flag $G \in \{\text{on}, \text{off}\}$ recording whether the cursor is inside `torch.no_grad()`.
 106 The entailment $\Gamma \models \varphi$ is discharged by Z3 on the shape projection and by the lattice on the grad
 107 component.

108 **Representative typing rules.** We give five rules; the full set (`matmul`, `conv1d/2d/3d`, `broadcast`,
 109 `view`, `reshape`, `split`, `chunk`, `unbind`, `cat`, `stack`, `permute`, `transpose`, `einsum`, `indexing`, `slicing`, `scatter`,
 110 `gather`, `repeat`, `expand`, `SDPA`, `LayerNorm`, `RMSNorm`, `detach`, `no_grad` context, `optimizer step`) is
 111 in Section B, with pen-and-paper soundness in Section C.

$$\begin{array}{c} \text{T-MATMUL} \\ \frac{\Gamma \vdash a : \text{Tensor}[\tau, (\bar{B}, m, k), \varphi_a] \quad \Gamma \vdash b : \text{Tensor}[\tau, (\bar{B}', k, n), \varphi_b] \quad \Gamma \models \text{broadcast}(\bar{B}, \bar{B}') = \bar{C}}{\Gamma \vdash a @ b : \text{Tensor}[\tau, (\bar{C}, m, n), g(a, b)]} \\ \\ \text{T-VIEW}(-1) \\ \frac{\Gamma \vdash x : \text{Tensor}[\tau, (d_1, \dots, d_n), \varphi_x] \quad \bar{s} = (s_1, \dots, s_{j-1}, -1, s_{j+1}, \dots, s_m) \quad P = \prod d_i, \quad Q = \prod_{i \neq j} s_i \quad \Gamma \models Q \mid P}{\Gamma \vdash x.\text{view}(\bar{s}) : \text{Tensor}[\tau, (s_1, \dots, P/Q, \dots, s_m), \varphi_x]} \\ \\ \text{T-DETACH} \\ \frac{\Gamma \vdash x : \text{Tensor}[\tau, \varphi_s, \varphi_g]}{\Gamma \vdash x.\text{detach}() : \text{Tensor}[\tau, \varphi_s, \text{no_grad}]} \end{array}$$

112 **Side conditions used above.** For T-MATMUL and elsewhere, $\text{broadcast}(\bar{B}, \bar{B}')$ denotes the
 113 standard PyTorch right-aligned broadcast on the batch prefixes $\bar{B}, \bar{B}'$: with $n = \max(|\bar{B}|, |\bar{B}'|)$
 114 each shape is left-padded with 1s to length n , then the position-wise rule $\text{bdim}(a, b) = a$ if
 115 $a = b$, $\text{bdim}(1, b) = b$, $\text{bdim}(a, 1) = a$, otherwise undefined; the resulting list is $\bar{C}$. The
 116 mechanised version is the function `bcast a b` in Section 4.4’s Lean development. The grad bit
 117 `is_on_tape` is the runtime conjunction `requires_grad` $\wedge$ `¬torch.no_grad`-context; its lifted abstrac-
 118 tion $\varphi_g \in \{\text{has_grad}, \text{no_grad}, \top\}$ is defined in Theorem 14, and the join $g(\bar{e})$ is bottom-respecting
 119 (any `no_grad` input forces `no_grad` unless $G = \text{on}$ and the operator is differentiable, in which case
 120 g is the disjunctive lift). For T-VIEW(-1) the side condition $Q \mid P$ requires $Q \neq 0$; the rule is
 121 undefined (rejected) when $\bar{s}$ contains zero or two-or-more `-1` slots (matching the explicit Python
 122 error), formalised by the closed Lean theorem `reshape_rejects_multi_unknown` of Section 4.4.
 123 The SMT fragment used to discharge these side conditions is the combination of linear integer
 124 arithmetic (LIA), the divisibility predicate $a \mid b$, and bounded non-linear multiplication (BMul) whose
 125 decidable kernel is the Z3 *nra/nia* hybrid procedure invoked with a fixed 3-second per-obligation
 126 budget; queries that exceed the budget return ABSTAIN, never a positive verdict.

127 **Verdicts.** The verifier emits one of five verdicts: VERIFIED, REFUTED-PROOF (RP, an uncon-
 128 ditional witness), CONTRACT-VIOLATION (CV, sound under the caller-rely *assume_e*), LIBRARY-
 129 WARN (LW, a conservative warning on dispatch outside the modeled fragment), and ABSTAIN. The
 130 soundness theorem (Theorem 2) covers RP and CV; LW and ABSTAIN make no claim.

Audited operator scope. The catalogue $\text{Cat} = \text{Cat}_{\text{audit}} \cup \text{Cat}_{\text{pen}} \cup \text{Cat}_{\text{tested}}$ of Table 9 partitions the 83 handlers into $|\text{Cat}_{\text{audit}}| = 36$ Lean-audited rules, $|\text{Cat}_{\text{pen}}| = 13$ pen-and-paper rules, and $|\text{Cat}_{\text{tested}}| = 34$ rules whose only check is the 1,000-sample property test against PyTorch. We write $\text{Cat}_{\text{sound}} = \text{Cat}_{\text{audit}} \cup \text{Cat}_{\text{pen}}$ (49 handlers) for the sub-catalogue with a closed soundness argument; the soundness theorem below quantifies over $\text{Cat}_{\text{sound}}$ only. The 34 tested-only handlers are addressed separately as a *conjecture* (Theorem 7), not folded into the theorem.

2.1 The Modeled Fragment $\mathcal{F}_{\text{TG}}$

Syntax. The fragment $\mathcal{F}_{\text{TG}}$ is defined by the following grammar, where $op \in \text{Cat}$ denotes any operator in the catalogue of Table 9, and static integers are values derivable from the declared refinements without a runtime query.

$$\begin{aligned} M &::= \text{class } C(\text{nn.Module}): \text{__init__}(\dots); \text{forward}(self, x_1, \dots, x_n) \rightarrow s \\ s &::= x = e \mid \text{return } e \mid \text{if}^\dagger c: s_1 \text{ else } s_2 \mid \text{for}^\dagger x \text{ in range}(n): s \\ e &::= x \mid e.op(args) [op \in \text{Cat}] \mid C(args) \mid self.attr \mid \text{const} \mid (e_1, \dots, e_k) \mid x_1, \dots, x_k = e \end{aligned}$$

[†]*Shape-determinism restriction:* branch condition c and loop bound n must be static integers derivable from the declared refinements. Types are $\text{Tensor}[\tau, \varphi_s, \varphi_g]$ as in §2.

Out-of-fragment. The following constructs are *not* in $\mathcal{F}_{\text{TG}}$: data-dependent control flow (condition involves a runtime tensor value); custom `autograd.Function`; dynamic attribute mutation of a module after `__init__`; `exec/eval`; decorator metaprogramming that modifies `forward` at runtime. The analyser emits ABSTAIN on any block containing these.

Theorem 1 (Fragment-level soundness on $\text{Cat}_{\text{sound}}$). *Let $M \in \mathcal{F}_{\text{TG}}$ be a closed module such that every operator occurrence in M uses a handler in $\text{Cat}_{\text{sound}} = \text{Cat}_{\text{audit}} \cup \text{Cat}_{\text{pen}}$. Let $\Gamma \vdash M : \tau$ with $\text{Verify}(M) = \text{VERIFIED}$. Then for every input x with $\sigma \models \text{assume}_M(x)$, the reduction $\langle M(x), \sigma \rangle \rightarrow^* v$ does not invoke the operator-level shape-mismatch exception of any operator in $\text{Cat}_{\text{sound}}$.*

Sketch. Preservation: by induction on the BNF of $\mathcal{F}_{\text{TG}}$. Each operator rule in $\text{Cat}_{\text{sound}}$ (T-MATMUL, T-VIEW(-1), T-RESHAPE(-1), T-DETACH, and the remaining 40 rules in Section B restricted to $\text{Cat}_{\text{sound}}$) produces a uniquely-typed result. The 36 Lean-audited handlers (Table 9, top block) are sorry-free at the rule definitions; the 13 pen-and-paper handlers follow from the operator-level argument in Section C, and two of their nontrivial integer-arithmetic obligations (`reshape` divisibility and the single-inferred-dim case) are also mechanised in Lean as `reshape_sound_zero_unknowns`, `reshape_sound_one_unknown`, and `reshape_rejects_multi_unknown` (Section 4.4). *Progress:* each statement in $\mathcal{F}_{\text{TG}}$ either reduces or is a value; the shape-determinism restriction on `if`[†] and `for`[†] ensures branch selection and loop unrolling are determined by the static refinements, so no stuck state arises at those forms. Together, no well-typed term using only operators in $\text{Cat}_{\text{sound}}$ raises a shape-mismatch at any $op \in \text{Cat}_{\text{sound}}$. Coverage of the 34 tested-only handlers $\text{Cat}_{\text{tested}} = \text{Cat} \setminus \text{Cat}_{\text{sound}}$ is the subject of Theorem 7, not of this theorem. $\square$

Boundary precision. Theorem 1 does *not* claim: (a) termination; (b) numerical correctness or absence of NaN/Inf; (c) absence of dtype errors outside $\text{Cat}_{\text{sound}}$; (d) soundness of the analyser *implementation*—only of the rule table it implements; (e) coverage of the 34 tested-only handlers $\text{Cat}_{\text{tested}}$ (Table 9, addressed by Theorem 7). The implication “verdict = VERIFIED $\Rightarrow$ no shape mismatch” depends on: (i) the assume_M contract holding at the call site; (ii) the operator being in $\text{Cat}_{\text{sound}}$ with a Lean-audited or pen-and-paper soundness proof; (iii) Z3 returning UNSAT correctly on every discharged obligation.

The fragment-level statement (Thm. 1) specialises Thm. 2 to whole programs.

Theorem 2 (Soundness of the proof-grade verdicts on $\text{Cat}_{\text{sound}}$). *Let e be a closed expression in the supported fragment such that every operator occurrence in e uses a handler in $\text{Cat}_{\text{sound}} = \text{Cat}_{\text{audit}} \cup \text{Cat}_{\text{pen}}$ (49 of 83 handlers; see Table 9). For any heap $\sigma \models \Gamma$: (i) if $\text{Verify}(e) = \text{VERIFIED}$, no reduction sequence from $\langle e, \sigma, \text{on} \rangle$ raises a shape-mismatch exception, and the grad-flag claim holds at every backward-pass entry site; (ii) if $\text{Verify}(e) = \text{REFUTED-PROOF}$ with witnessed subterm $e_0 \subseteq e$ and Z3 model $\mathfrak{m}$ satisfying the negated obligation, then there exists a heap $\sigma_{\mathfrak{m}} \models \Gamma$ extracted*

178 from m (Theorem 3) and a reduction sequence from $\langle e, \sigma_m, \text{on} \rangle$ that reaches the witnessed subterm
 179 e_0 and raises a shape- or grad-flag exception there; (iii) if $\text{Verify}(e) = \text{CONTRACT-VIOLATION}$, (ii)
 180 holds under the assumption that every caller supplies inputs satisfying assume_e ; (iv) if $\text{Verify}(e) \in$
 181 $\{\text{LIBRARY-WARN}, \text{ABSTAIN}\}$, no claim is made.

182 The proof (induction on reductions; full case analysis in Section C) reduces (i) to per-operator preser-
 183 vation lemmas: for handlers in $\text{Cat}_{\text{audit}}$ these are the `applyOp_sound_*` and `applyOpExt_sound_*`
 184 lemmas of the Lean development (Section 4.4); for handlers in Cat_{pen} they are the closed pen-
 185 and-paper sketches in Section C, two of which (the divisibility and single-inferred-dim cases
 186 of `reshape`) are also mechanised as closed Lean theorems (`reshape_sound_zero_unknowns`,
 187 `reshape_sound_one_unknown`, `reshape_rejects_multi_unknown`). For (ii) the proof adds a
 188 *model-extraction* step (Theorem 3 below) that reads a Z3 model m on the negated obligation back into
 189 a concrete heap $\sigma_m \models \Gamma$, plus a *progress-to-witness* lemma (Theorem 5) that walks the small-step
 190 relation from $\langle e, \sigma_m, \text{on} \rangle$ to the witnessed operator occurrence using only the deterministic operator
 191 schedules of $\mathcal{F}_{\text{TG}}$ (Theorem 8, $\mathcal{F}_{\text{TG}}$ has no internal nondeterminism over heap-deterministic opera-
 192 tors); existential rather than universal quantification over reduction sequences avoids any appeal to
 193 confluence. Case (iii) follows from the same argument inside the rely envelope assume_e .

194 **Definition 3** (Model extraction $m \mapsto \sigma_m$). Let $\Gamma =$
 195 $(x_1:\text{Tensor}[\tau_1, \bar{d}_1, g_1], \dots, x_n:\text{Tensor}[\tau_n, \bar{d}_n, g_n] \mid P)$ with shape predicate P over symbolic
 196 dim variables $\bar{v} = (v_1, \dots, v_k)$, and let m be a Z3 model satisfying $P \wedge \neg\varphi_{\text{op}}(\bar{v})$ where φ_{op} is the
 197 operator-precondition refinement emitted at the witnessed subterm e_0 .

198 *Z3 encoding of the grad refinement.* Each grad refinement g_i ranges over the three-element flat
 199 lattice $\mathbb{G} = \{\text{has_grad}, \text{no_grad}, \top\}$, encoded by a pair of Boolean Z3 variables (rg_i, tp_i) over the
 200 $\text{BOOL} \times \text{BOOL}$ sort with the elimination constraint $tp_i \Rightarrow \neg rg_i$ asserted at context-entry. The
 201 decoding map $m(g_i) \in \mathbb{G}$ is `has_grad` when $(rg_i, tp_i) = (\top, \perp)$, `no_grad` when $(\perp, \perp)$, and $\top$ when
 202 $tp_i = \top$ (the second component encodes “the model is non-committal on `requires_grad`”). The
 203 runtime-side projection $\rho : \mathbb{G} \rightarrow \{\text{True}, \text{False}\}$ used to build the concrete heap fixes $\rho(\text{has_grad}) =$
 204 `True`, $\rho(\text{no_grad}) = \text{False}$, and $\rho(\top) = \text{True}$ — the $\top$ case is realised in the concrete heap by a
 205 `Tensor` with `requires_grad = True`, the strictly stronger of the two committed values, so that
 206 any grad-flow obligation discharged on the realised heap also discharges on its `no_grad` refinement
 207 (joint satisfiability is preserved because the per-tensor refinements either commit to `has_grad` — in
 208 which case $\rho(\text{has_grad})$ agrees — or commit to `no_grad` — in which case the context constraint
 209 $tp_i = \perp$ rules out the $\top$ case at m — or commit to $\top$ — in which case strengthening to `has_grad` in
 210 the heap is the covariant direction of $\sqsubseteq_{\mathbb{G}}$). Define $\sigma_m(x_i)$ as the all-zero tensor of dtype τ_i and shape
 211 $m(\bar{d}_i) \in \mathbb{N}^{|\bar{d}_i|}$ with `requires_grad` = $\rho(m(g_i))$. The map is total because the SMT theory is closed
 212 under interpretation in $(\mathbb{N}, +, \cdot, |) \sqcup (\text{BOOL} \times \text{BOOL})$, every $m(d)$ is a non-negative integer, and the
 213 construction visibly satisfies $\sigma_m \models \Gamma$ (each per-tensor shape refinement is the literal Z3 assignment;
 214 each grad refinement is satisfied by $\rho \circ m$ under the $\sqsubseteq_{\mathbb{G}}$ -order, where `has_grad`, `no_grad` $\sqsubseteq \top$ and both
 215 leaves are pairwise incomparable). When the witnessed obligation $\neg\varphi_{\text{op}}$ is grad-only and is jointly
 216 satisfiable only with $m(g_j) = \top$ for some j (i.e. Z3 chooses $tp_j = \top$ to falsify a grad precondition),
 217 the realised heap with $\rho(\top) = \text{True}$ still falsifies φ_{op} : the negated grad obligations of the analyser
 218 are by construction monotone in $\sqsubseteq_{\mathbb{G}}$ (Theorem 4, restated below), so committing `has_grad` at x_j
 219 where Z3 allowed $\top$ leaves the conjunction $P \wedge \neg\varphi_{\text{op}}$ satisfied by the realised heap.

220 **Lemma 4** (Grad-flow monotonicity along $\sqsubseteq_{\mathbb{G}}$). For every operator $op \in \text{Cat}_{\text{sound}}$, the grad
 221 component of φ_{op} is a Boolean combination of literals of the form $g_i \sqsupseteq \text{has_grad}$ and $g_i \sqsubseteq \text{no_grad}$
 222 that is monotone in the strengthening order `has_grad`, `no_grad` $\sqsubseteq \top$. Consequently, if $\sigma_m \models \Gamma \wedge \neg\varphi_{\text{op}}$
 223 under the decoding $m(g_i) = \top$, then the strengthened heap σ'_m obtained by replacing $g_i \mapsto \top$ with
 224 the realised value $g_i \mapsto \text{has_grad}$ also satisfies $\Gamma \wedge \neg\varphi_{\text{op}}$. The proof is a case split on the four grad-
 225 flow refinements emitted by the analyser (`T-DETACHLOSS`, `T-NOGRADSCOPE`, `T-FROZENPARAM`,
 226 `T-MIXEDREUSE`); in each case, the obligation that is falsifiable at $\top$ is also falsifiable at `has_grad`
 227 because the analyser’s refutation predicates are syntactically of the form $\rho(g_i) = \text{True} \wedge \Phi_{\text{shape}}$ or
 228 $\rho(g_i) = \text{False} \wedge \Phi_{\text{shape}}$ with no use of $g_i = \top$ as an active literal.

229 **Lemma 5** (Progress to witness; deterministic small-step). Let e_0 be the witnessed subterm produced
 230 by $\text{Verify}(e) = \text{REFUTED-PROOF}$ and let σ_m be the model-extracted heap of Theorem 3. The small-
 231 step relation $\rightarrow$ on $\mathcal{F}_{\text{TG}}$ configurations evaluates operator arguments left-to-right, contracts each
 232 operator on the unique argument multiset that satisfies its precondition or raises an `OP-STUCK`
 233 exception, and is therefore deterministic on heap-determined inputs (Theorem 8). Consequently,
 234 starting from $\langle e, \sigma_m, \text{on} \rangle$, either (a) reduction reaches a configuration whose redex is e_0 , in which case

the precondition φ_{op} fails by construction of $\mathfrak{m}$ and an OP-STUCK (shape) or GRAD-FLAG-STUCK exception is raised at e_0 , or (b) reduction halts before reaching e_0 on a strictly earlier exception, which is itself a shape- or grad-flag exception by the same per-operator analysis. Both branches discharge case (ii).

Remark 6 (Trusted-base obligations for (ii)). Case (ii) relies on three ingredients beyond the per-operator preservation lemmas of (i): (a) that the operator runtime semantics raises on the witnessed input. For the 36 operators in $\text{Cat}_{\text{sound}} \setminus \text{Cat}_{\text{pen}}$ this is discharged by the Lean-mechanised `applyOpExt_sound.*` lemmas (the precondition that appears in the rule is exactly the precondition under which the mechanised semantics succeeds). For the 13 pen-and-paper handlers Cat_{pen} (T-BROADCAST, T-REDUCE, *einsum*, the elementwise-activation family, the structural reshapes, plus *pad/where*), case (ii) rests on PyTorch documentation that the corresponding handler raises on $\neg\varphi_{\text{op}}$, paired with the in-envelope agreement count of 1,000/1,000 samples in Table 9; this is a documentation-plus-sampling discharge, not a derivation. We list it explicitly as part of the trusted base of Theorem 2 restricted to Cat_{pen} , and we do not claim refutation soundness on this sub-catalogue follows from a closed argument. A complementary AST-level certificate, re-generated from the analyser source on each build, classifies each of the 13 pen-and-paper handlers into the T-IDENTITY or T-BROADCAST schema and records evidence line numbers and a content hash for audit; the resulting partition is 10 T-IDENTITY handlers (*relu, gelu, silu, tanh, sigmoid, softmax, detach, flatten, pad, T-REDUCE*) plus 3 T-BROADCAST handlers (*elementwise_binary, where, einsum*). The classifier ships under the reproducibility directory with a 18-test pytest suite and is re-exercised by the hard-constraint audit target on every build. (b) that the Z3-model values are non-negative integers compatible with $\mathbb{N}$ -valued *dim* refinements (true by construction of the SMT fragment of Section 2); and (c) that the small-step relation is deterministic on heap-determined inputs (Theorem 8). All three are explicitly listed as trusted-base obligations of Theorem 2 and audited in the per-handler soundness sketches of Section C.

Conjecture 7 (Tested-only handlers; empirically supported). Restricting attention to operator occurrences in the 34-handler tested-only catalogue $\text{Cat}_{\text{tested}}$, the 1,000-sample property test against the corresponding PyTorch operator agrees with the TG rule on every sampled concrete input. We do not claim a closed proof of Theorem 2 restricted to $\text{Cat}_{\text{tested}}$; verdicts touching at least one handler in $\text{Cat}_{\text{tested}}$ are reported (and counted as the 12/185 tested-only-touching cell of Table 9 discussion) but are flagged as outside the soundness theorem. Closing this gap (either by porting more handlers into $\text{Cat}_{\text{audit}}$ or by refactoring the $\text{Cat}_{\text{tested}}$ rules to reduce to $\text{Cat}_{\text{sound}}$ via a structural argument) is left as future work.

Subject reduction and progress. The classical Preservation and Progress theorems for $\mathcal{F}_{\text{TG}}$ are established in Section D (Theorems 18 and 19). Together they yield Theorem 20: well-typed programs in $\mathcal{F}_{\text{TG}}$ whose operator preconditions hold in σ raise no shape-mismatch exception on any reduction sequence. Theorem 2 case (i) (VERIFIED soundness) is the abstract-interpretation specialisation of this classical pair to the verdict lattice; the verdict VERIFIED corresponds to Z3 discharging all preconditions symbolically, so Theorem 20 applies. Case (ii) (refutation soundness) is *not* a specialisation of preservation/progress: it is an additional “completeness for refutation” direction that requires (a) the model-extraction map $\mathfrak{m} \mapsto \sigma_{\mathfrak{m}}$ of Theorem 3 and (b) the in-envelope agreement of the runtime semantics with the per-operator rule (Theorem 5 together with Theorem 6). The two directions are proved separately: case (i) by a chain of PRESERVATION $\circ$ PROGRESS from Section D, case (ii) by the constructive walk of Theorem 5 along a deterministic small-step (Theorem 8). We do not claim case (ii) follows from preservation/progress.

Definition 8 (Deterministic small-step evaluation order). The small-step relation $\rightarrow$ on $\mathcal{F}_{\text{TG}}$ configurations evaluates compound operator applications by a fixed left-to-right argument-evaluation strategy and contracts each ground operator on the unique heap-deterministic image specified by its semantics; in particular every operator in $\text{Cat}_{\text{sound}}$ is a deterministic function of its concrete-tensor argument multiset (no autograd-side in-place rewrites or RNG draws are inside the operator language; dropout’s RNG draw is hoisted out as a mask argument before the operator step). Consequently, given a heap σ and a closed expression e , there is at most one finite reduction sequence $\langle e, \sigma, \text{on} \rangle \rightarrow^* \langle v, \sigma', \text{on} \rangle$ ending in a value or in an OP-STUCK or GRAD-FLAG-STUCK exception. The relation is therefore *not* confluent in the term-rewriting sense (we never claim it is); it is simply a deterministic abstract-machine evaluator for the fragment.

2.2 Assume/Guarantee Composition

For each `nn.Module` class M we synthesise a contract $C(M) = (assume_M, guarantee_M)$, with $assume_M$ a refinement on input types and $guarantee_M$ a refinement on output types. Sequential composition (e.g. `nn.Sequential`) discharges $guarantee_{M_1} \Rightarrow assume_{M_2}$ locally; parallel composition (residual sums, QKV split) treats each sibling independently and joins the guarantees:

$$\begin{array}{c}
 \text{SEQ} \\
 \frac{\vdash C(M_1) = (A_1, G_1) \quad \vdash C(M_2) = (A_2, G_2) \quad \models G_1 \Rightarrow A_2}{\vdash C(M_2 \circ M_1) = (A_1, G_2)} \\
 \\
 \text{SUB} \\
 \frac{C(M) = (A, G) \quad M' <: M \quad \models A_M \Rightarrow A_{M'} \text{ (contravariant)} \quad \models G_{M'} \Rightarrow G_M \text{ (covariant)}}{\vdash C(M') = (A_{M'}, G_{M'}) \text{ refines } C(M)}
 \end{array}$$

The subclassing rule is a syntactic obligation: a subclass that overrides `forward` in a way that violates the parent's guarantee is rejected statically rather than silently re-analysed.

Theorem 9 (Compositional soundness; mechanised fragment). *Scope. The theorem as stated is mechanised on a 17-operator DSL covering `matmul`, `view`, `add/broadcast_add`, `linear`, `transpose`, `permute`, `relu`, `cat`, `sum_reduce`, `mean_reduce`, `expand`, `gather`, `embedding`, `conv2d`, `einsum`, `unbind`, and `reshape`; all 17 operators carry a per-operator `applyOpExt_sound_*` lemma closed sorry-free in Lean 4. In particular, `applyOpExt_sound_broadcast_add` (identity on the input shape) and `applyOpExt_sound_matmul` (success implies input rank ≥ 3 and verdict equals input shape; the rank-2 contraction is delegated to the two-input `matmul2` lemma of Section E) are now explicit lemmas rather than instances of an operator-agnostic axiom; the corresponding entry of Table 9 therefore counts 17/17 mechanised, 0/17 residual operator-agnostic obligations, which is reflected in the per-handler breakdown of Table 9. All 17 DSL operators lie inside $\text{Cat}_{\text{sound}}$ of Theorem 2. The mechanised fragment does not cover the remaining 66 handlers of Table 9, the AST extractor, or the Python analyser implementation. If every leaf module in the call tree of an $\mathcal{F}_{\text{TG}}^{17}$ program is contract-verified in the mechanised fragment and every composition obligation is discharged by Z3, then the whole-program verdict given by the inductive use of SEQ, PAR, and SUB is sound in the sense of Theorem 2 restricted to the same fragment.*

Remark 10 (Closed lemmas for `matmul` and `broadcast_add`; replaces a prior operator-agnostic obligation). *In an earlier revision the two operators `broadcast_add` and `matmul` were discharged inside Theorem 9 by an operator-agnostic composition witness whose first clause appealed to the 1,000-sample in-envelope agreement count of Table 9. We have since closed both obligations as Lean lemmas: (i) `applyOpExt_sound_broadcast_add` states that whenever the simplified single-shape `broadcast_add` rule succeeds, the verdict shape is the input shape (the rule is the identity in the single-shape DSL; the two-input `broadcast` contract is discharged compositionally via the rule for shape-pair add in Section E); (ii) `applyOpExt_sound_matmul` states that whenever the single-shape `matmul` rule succeeds, the input has rank at least three and the verdict equals the input shape (the rank-2 contraction is delegated to the two-input `matmul2/V50operatorRules.matmul` lemma already mechanised in Section E). Both lemmas are sorry-free under `lake build`; the in-envelope agreement count of Table 9 is no longer load-bearing for Theorem 9 and is retained only as a runtime cross-check on the rule table.*

Axiom 11 (Fresh-witness refutation; needed for Theorem 12). *The analyser emits REFUTED-PROOF (or CONTRACT-VIOLATION) for module M under context Γ only if, in the current analysis pass under Γ , it produces a concrete input x with $assume_M(x) \wedge \neg guarantee_M(M(x))$. Cached witnesses from earlier passes are not allowed to count. This is a property of how the analyser is implemented (no witness memoisation across pass boundaries), not a property of the calculus. The discipline is enforced by a single verifier-entry invariant (the witness cache is keyed on (Γ, M) and emptied on every Γ -edge); we distribute an executable property test that asserts this invariant on every analyser entry by replaying 200 random Γ -strengthenings against the current implementation and confirming the witness count strictly increases on each strengthening (no cache hit). The current shipped analyser passes the test on all 200 replays.*

Theorem 12 (Monotonicity of refinement; conditional on no-memoisation). *Strengthening any single contract $C(M_0)$ to $C'(M_0) \sqsubseteq C(M_0)$ never turns a VERIFIED root into REFUTED-PROOF. Conditional on the no-memoisation discipline (Theorem 11; verified by*

the property test described above on the current shipped analyser), the new verdict is in $\{\text{VERIFIED}, \text{CONTRACT-VIOLATION}, \text{LIBRARY-WARN}, \text{ABSTAIN}\}$. The conditional phrasing makes the unmechanised implementation invariant explicit; in deployments that disable the witness-cache invariant or hot-swap the verifier in a way that breaks the property test, the second clause is vacuous and only the first (“no VERIFIED-to-REFUTED-PROOF flip”) survives.

The full case analysis (cases on the form of refinement, with the $\perp$ guarantee and unreachable-sibling sub-cases) is in Section E.

3 Implementation

3.1 Extended fragment and front-end

TENSORGUARD’s analyser parses class source via Python’s `ast` module, extracts a typed IR, and dispatches each op to a handler that returns the refinement-typed result type. We add four front-end families under (no edits to `legacy`). The *symbolic config* environment binds `self.config.<attr>` to fresh `SymInts` so that HuggingFace-style modules whose dimensions come from a `config` object can be checked once and reused. The *QKV unpacking* front-end recognises the four idiomatic Q/K/V splits (`split`, `chunk`, `view+unbind`, and `einops.rearrange` with the canonical $(\text{three } h \text{ d}) \rightarrow \text{three } b \text{ h t d}$ pattern) and elaborates each into three correlated tensor types. The *negative-one reshape* elaborator emits a Z3 divisibility constraint $P \bmod Q = 0$. Transfer rules for `F.scaled_dot_product_attention`, `nn.MultiheadAttention`, `nn.LayerNorm`, and `nn.RMSNorm` register into the analyser’s shape-op dictionary. All 74 unit tests pass on `python3.11 + torch 2.9.1`. We report that on the *legacy v4 corpus* these new APIs produce zero verdict-count delta (the v4 corpus did not abstain on patterns these APIs target); the v5 contribution is that the new APIs are now *callable on realistic Transformer modules* that v4 could not enter, surfaced in the empirical evaluation through the 488-block real-source corpus (Section 4.1).

3.2 Backward verifier

The backward verifier consumes a forward graph annotated with the refinement types of Section 2 and certifies, without execution, that (i) `loss.backward()` would produce gradients of the right shape modulo `sum_to(grad, primal.shape)` reduction, and (ii) the set H of parameters reverse-reachable from `loss` along non-detached, non-no-grad-context, `requires_grad=True` edges equals the user-declared expected-to-learn set E . The check is a reverse-topological traversal mirroring PyTorch’s per-Function shape contract `grad_outputs[i].shape == y_i.shape`. The grad-flag component additionally detects the three canonical silent-zero-grad bug classes: in-place writes on a leaf with `requires_grad=True`; no leaf in the trace has `requires_grad=True`; and parameters used only inside `torch.no_grad()` or after `.detach()`. These are reported as REFUTED with witness traces.

Empirical agreement. On a 500-model property test (random small `nn.Modules` drawn from a grammar) the static verifier agrees with PyTorch’s runtime on 500/500 models (gradient presence and gradient shape both match). On 8 hand-curated case studies modelled on canonical PyTorch issues (`no_grad` scope, `.detach()`, `requires_grad=False`, in-place-on-leaf, missing leaf, in-place alias mutating a saved tensor, non-scalar loss, unused parameter silently skipped by `optimizer.step()`), the verifier catches 8/8. On a 50-model false-positive sweep on clean models the verifier reports 0 spurious refutations.

3.3 Localisation tracer and hybrid mode

Localisation. A v4 review noted that of 6 detections in early experiments, only 1 landed within ± 5 lines of the bug. The v4 verifier returns `location.line=0` for bugs detected via the model-check path (those that bypass per-step counterexamples). We add a “first unsatisfied symbolic constraint” tracer: given a refutation, we extract the offending tensor and dim variables from the bug message and walk the source AST to the earliest assignment or typing-rule call that bound a symbolic dim appearing in the unsatisfied constraint, attributing the bug to its source line.

Hybrid mode (TG-static $\rightarrow$ FakeTensor fallback). `tensorguard.hybrid_check(M)` returns the TG verdict if it is VERIFIED or REFUTED; otherwise, if M is importable and instantiable with

Table 1: Block corpus ($N = 488$) and bug corpus ($N = 60$) per tool. N/A = tool inapplicable. TG’s Refuted column is split RP/CV/LW (see §2); only RP+CV are covered by Theorem 2. On the bug corpus, RP is the desired outcome (all 53 refutations are REFUTED-PROOF). TG = TENSORGUARD v5; Pytea row from (commit 2022-04-26) on the same harness. **RP reconciliation.** The 0 in the RP column is the *unrestricted* count under the high-confidence Z3-discharged regime: every refutation must hold for every realisable caller of the module, with no synthesised caller-rely obligation. The 26/356 Refuted-Proof figure cited in the abstract is the *same* verdict log restricted post-hoc to the 356-block empty-*assume_M* subset (no synthesised caller-rely obligation; the refutation reduces to unconditional). The two numbers are not in tension; they are the two tail rates of one verdict log under two compatible RP definitions. Both are reproducible from the released audit log at the canonical code snapshot referenced below.

Tool	Block corpus ($N=488$)						Bug corpus ($N=60$)			
	V	RP	CV	LW	A	N/A	V	R	A	N/A
TG	57	0	128	78	225	0	7	53	0	0
torch.fx+ShapeProp	0		0		7	481	0	0	7	53
FakeTensorMode	0		0		7	481	0	0	7	53
torch.export	0		3		4	481	2	4	1	53
Pytea	53		0		0	435	31	27	0	2
mypy+jaxtyping	0		0		0	488	0	0	0	60
beartype	0		0		0	488	0	0	0	60

a no-arg constructor, it falls back to FakeTensorMode. Both the localisation tracer and the hybrid mode are evaluated in Section 4.2.

4 Empirical Evaluation

4.1 Real-source benchmark: 488 nn.Module blocks + 60 historical PyTorch shape bugs

Corpora. The *block corpus* is 488 standalone `nn.Module` subclasses programmatically extracted (via `inspect.getsource`) from the public release tags of TORCHVISION 0.24.1, TIMM 1.0.26, and TRANSFORMERS 4.57.3, content-addressed by SHA-256. The *bug corpus* is 60 historical PyTorch shape bugs mined from the issue tracker via 20+ keyword queries, of which we kept only those that admit a self-contained ≤ 40 -line CPU repro that raises the cited `RuntimeError`. The categories span attention dim (5), broadcasting (9), view/reshape (9), conv-channel (6), linear-in/out (4), einsum (5), transpose-axes (4), batchnorm-features (4), embedding-index (3), and other (11).

Calibrated reporting. Every (tool, input) cell is exactly one of VERIFIED, REFUTED, ABSTAIN, or N/A (tool structurally inapplicable). TG’s REFUTED column is further split into the three sub-verdicts of Theorem 2: REFUTED-PROOF (RP, an unconditional witness), CONTRACT-VIOLATION (CV, sound under the caller-respects-precondition assumption), and LIBRARY-WARN (LW, conservative warning on dispatch outside the modeled fragment). Theorem 2 covers RP and CV; LW is reported but *not* a soundness claim. We never silently report VERIFIED when an analyser was unable to reason: each ABSTAIN is tagged with one reason from a fixed top-10 taxonomy.

Headline. Calibration first. On the 488-block real-source corpus, TG produces *zero* unconditional REFUTED-PROOF verdicts. The 206 refutations decompose as 128 CONTRACT-VIOLATION (sound only under the synthesised caller-rely assume) plus 78 LIBRARY-WARN (conservative warnings outside the soundness theorem). The 57 VERIFIED verdicts also rest in part on synthesised config envelopes; under a regime where every `self.config.X` is treated as a free symbolic variable with no upper-bound assumption, the user-visible verdict triple becomes 34 V / 0 RP / 206 LW / 248 A. The 0-RP gap on real library source is therefore a real limitation acknowledged up front; *the unconditional-RP claim is carried by the bug corpora, not by the block corpus.*

Bookkeeping note on the headline triple. The {57, 206, 225} triple corresponds to the high-confidence verdict regime in which only Z3-discharged refutations are reported. The same corpus under the verifier’s public default configuration, which adds a lower-confidence heuristic post-pass, returns {50, 213, 225}: the 7-row gap is exactly that post-pass moving seven modules from

417 VERIFIED to REFUTED, and the 225 ABSTAIN count is identical across both regimes. Re-executing
 418 both regimes against the current code produces a clean $\{62, 201, 225\}$ vs $\{55, 208, 225\}$ pair with
 419 the same 7-row VERIFIED→REFUTED diff structure; the 5-block absolute drift since the original
 420 capture is bookkeeping-clean (no abstain transitions, no refutation flips). **Canonical regime and**
 421 **snapshot.** The high-confidence $\{57, 206, 225\}$ triple is the canonical headline used in the abstract,
 422 in Table 1, and throughout the body; the 26/356 empty-*assume_M* sub-count is computed from
 423 the same verdict log. A per-block audit table pinning each of these 488 rows to its verdict and its
 424 canonical-snapshot identifier ships with the open-source implementation.

425 **Caller-rely satisfiability of the 128 CV verdicts.** A CONTRACT-VIOLATION is meaningful only if
 426 its synthesised caller-rely *assume_M* is satisfied by *at least one* real caller. We mechanically classify
 427 all 128 CV-verdict modules on the 488-block corpus by what their *assume_M* references: 26/128
 428 have an empty *assume_M* (trivially satisfied), 90/128 reduce to symbolic references over documented
 429 config attributes (config.hidden_size, config.num_attention_heads, ...) that every pub-
 430 lished checkpoint config exposes, and 12/128 are *PreTrainedModel stubs whose `__init__` is
 431 inherited and contributes no axiom. *Zero assume_M is unwitnessed*: every CV refutes against at least
 432 one realisable caller pattern, never the empty contract. We additionally exhibit the joint-realisability
 433 check on the *full* 128 CV set: each row’s full *assume_M* conjunction is evaluated against a single de-
 434 fault *Config() instantiation of its natural caller. The witnessed-ratio is **118/128** (92.2%, Clopper-
 435 Pearson 95% CI [86.1%, 96.2%]); the 10 single-default-omitted rows are all *PreTrainedModel
 436 stubs or aliasing-only constructors whose default *Config() omits one declared sym-attr, never a
 437 contradiction between two clauses. We retain the term *unwitnessed* for the ASSUME_M-realisability
 438 sense (caller-pattern witness, value 0/128) and use *single-default-omitted* for this stricter single-
 439 default-config witness sense (value 10/128); the two are distinct and the 0 vs. 10 gap is the gap
 440 between the two definitions, not a contradiction. The 10 single-default-omitted rows decompose by
 441 HuggingFace family as GPT-2 (2), BERT/Llama/Mistral/Falcon/Qwen2/roBERTa/Albert/Bloom (1
 442 each); zero are torchvision or timm. All 10 live in the same bucket (*symbolic-config-only*) with the
 443 identical diagnostic “one or more sym_attr clauses failed,” confirming that the single-default-omitted
 444 entries are not concentrated in one family but a uniform pattern across HF transformer backbones
 445 whose default configuration omits a declared symbolic attribute.

446 **AST-extractor cross-validation against an independent oracle.** The synthesis of *assume_M*
 447 from class source is performed by an AST traversal that sits in the trusted computing base (not
 448 Lean-audited). To bound its soundness-direction error rate we cross-validate it against an independent
 449 re-implementation built only from Python’s standard ast module: a minimal oracle that enumer-
 450 ates literal <config>. <attr> attribute reads in `__init__` together with <self.attr> = <const>
 451 scalar writes and the function’s plain init parameter set. On the 140 classes available in-repo across
 452 the 113 config-attribute fixtures, the 10-bug upstream-faithful real corpus, the 15-bug post-freeze
 453 real corpus, and the 15-bug unfiltered real corpus, the deployed extractor’s *symbolic_config_attrs*
 454 set is a subset of the oracle’s literal config-reference set on **140/140** classes, exactly equals the
 455 oracle on **140/140** classes, and the extractor’s *init_param_names* set is a subset of the oracle’s
 456 init parameter set on **140/140** classes. Zero extractor errors; zero over-extractions. The scalar-
 457 attr direction is informational only (63/140 extractor- $\subseteq$ -oracle), since the extractor deliberately
 458 accepts non-literal scalar derivations (hidden_size // num_heads) that the literal-Constant oracle
 459 excludes; this is the safe direction for *assume_M* because additional bound scalars only further
 460 constrain the caller-rely contract. The audit reaches the AST extractor in the trusted computing
 461 base directly and *calibrates* the *symbolic_config_attrs* direction of the synthesised *assume_M* on
 462 the 128 CONTRACT-VIOLATION verdicts: the 0 over-extractions on *symbolic_config_attrs* mean
 463 every CV verdict’s caller-rely premise is, on these corpora, a subset of the literal config-reads the
 464 upstream class performs. We do *not* claim this audit closes the AST-extractor TCB obligation: both
 465 the deployed extractor and the oracle were implemented by the authors against the same surface
 466 specification, so this is a self-consistency cross-check, not a third-party soundness audit. To narrow
 467 the residual systematic-design exposure we add a hand-labelled *assume_M* ground-truth on a 20-
 468 module out-of-distribution slice (attention/MLP submodules from upstream PR repos across xlstm,
 469 GPT-NeoX, ConvBERT, Longformer, LongT5, T5/Gemma2, PEFT, diffusers, RouterParallel, and
 470 Qwen-Image) on which the deployed extractor agrees with the hand label on **20/20** of the soundness-
 471 direction *symbolic_config_attrs* set (both subset and exact equality; zero over-extractions). The
 472 *init_param_names* direction is informational only: the deployed extractor scopes init parameters
 473 more narrowly than the literal AST signature (it surfaces only params that participate in later contract

474 synthesis), which is the safe direction for *assume_M*. The residual scalar-attr non-coverage is in the
 475 informational direction (extractor accepts derived integers, the oracle does not) and only *further*
 476 constrains the assume side, so it does not weaken the soundness reading of any CV. The remaining
 477 TCB exposure on *systematic* extractor-vs-spec mismatch is explicitly preserved as a Limitations item
 478 rather than absorbed into the soundness statement.

479 **LW→RP gap.** Of the 78 LIBRARY-WARN verdicts on the 488-block corpus,
 480 66/78 (84.6%) are *dispatch-outside-fragment*—principled abstentions that no re-
 481 alistic catalogue extension would convert to REFUTED-PROOF: 37 classes have
 482 no forward body in the class scope at all (inheritance-only stubs such as
 483 `LlamaForSequenceClassification(GenericForSequenceClassification,...)`
 484 which inherit forward semantics from a non-`nn.Module` base); 27 are subclasses of user-
 485 defined non-`nn.Module` bases (`ShiftedWindowAttentionV2(ShiftedWindowAttention)`,
 486 `LayerNorm2d(nn.LayerNorm)` composing the parent forward), 2 iterate over an opaque
 487 `self.<container>`, and the remaining 0 use `getattr/*args/**kwargs`. The residual 12/78
 488 (15.4%) use only fragment ops in their forward bodies and are an upper bound on the LW→RP
 489 conversions that strengthening catalogue-internal reasoning could yield without enlarging the
 490 operator-catalogue fragment itself. The 12 residual blocks are exhibited at the per-block level so the
 491 upper bound is independently checkable, paired with the single missing operator-rule whose addition
 would (in isolation) flip each LW verdict to unconditional REFUTED-PROOF: A future revision can

Block	Missing rule (single addition)	Measured / Predicted
<code>tv::InvertedResidual</code>	<code>const-attr-gated</code> branch unification	RP
<code>tv::LayerNorm2d</code>	<code>F.layer_norm</code> under <code>permute()</code> composition	RP
<code>tv::ASPPPooling</code>	<code>iter(self)</code> over <code>Sequential</code> children	RP
<code>tv::LRASPPHead</code>	<code>Dict[str, Tensor]</code> positional-input destructuring	RP
<code>timm::ChannelAttention</code>	<code>Tensor.unbind(dim)</code> fixed-length tuple shape	RP (measured)
<code>timm::ChannelAttentionV2</code>	<code>unbind(dim)</code> + scalar power on symbolic int	RP (measured)
<code>timm::PatchEmbed</code>	<code>_assert</code> precondition propagation	RP
<code>tx::WhisperPosEmb</code>	<code>Tensor._getitem_(slice(int, int))</code>	RP
<code>tx::BartLearnedPosEmb</code>	<code>nn.Embedding</code> subclass recognition	RP (measured, DEAD-OUTPUT)
<code>tx::BartScaledWordEmb</code>	<code>nn.Embedding</code> subclass recognition	Verified (measured)
<code>tx::FalconLinear</code>	<code>x @ self.weight.T</code> (transposed-Parameter matmul)	Verified (measured)
<code>tx::OPTLearnedPosEmb</code>	<code>torch.cumsum + super().forward</code> → <code>Embedding</code>	RP

492 implement any one of the remaining named rules and verify that the corresponding LW block actually
 493 flips to RP, so the 12/78 ceiling remains falsifiable from the paper alone. This round additionally
 494 implements (i) the `nn.Embedding/nn.Linear/nn.LayerNorm` subclass recogniser (making previously
 495 not-analysable subclasses such as `FalconLinear` and `BartLearnedPositionalEmbedding`
 496 analysable) and (ii) the transposed-Parameter matmul rule (`x @ self.weight.T` emitting a
 497 TRANSPOSE step before the MATMUL check, plus `x.t()` as an alias). The combined effect:
 498 three table rows flip from predicted to *measured*: `ChannelAttention/ChannelAttentionV2` to RP
 499 via the `unbind` rule (SHAPE-INCOMPATIBLE reshape); and `BartLearnedPositionalEmbedding`
 500 to RP via the `Embedding` subclass recogniser (DEAD-OUTPUT: a variable is computed but
 501 never used). Two additional table rows (`FalconLinear`, `BartScaledWordEmbedding`) are now
 502 measurably VERIFIED: their implementations are correct and the analyser verifies them once the
 503 subclass recogniser applies. The 0-unconditional-RP ceiling on the free-symbolic-config slice
 504 therefore moves to 3/12 named LW→RP candidates already measured-flipped (the other 7 remain
 505 predictions, and 2 measured as Verified rather than RP); the reproducibility appendix records the
 506 per-block verdict. Splitting these 12 residuals by extension class — i.e. whether the missing rule
 507 is part of the published roadmap of the operator-rule table or instead a fragment-level extension
 508 that would broaden the analyser’s typed-fragment surface — gives 4 roadmap-extensions (the two
 509 remaining `super().forward() + nn.Embedding` cases and the `Tensor._getitem_(slice)`
 510 case plus one additional), 6 fragment-extensions (`unbind`, `F.layer_norm-under-permute`,
 511 `Sequential` iteration, `Dict[str, Tensor]` positional destructuring, transposed-Parameter
 512 matmul, and `torch.cumsum`), and 1 mixed (the `const-attr-gated` branch unification, which needs both
 513 a roadmap rule and a fragment-level reachability extension); a per-row classification accompanies the
 514 table in the reproducibility appendix. Concretely the headline 0-RP triple is dominated by principled
 515 abstention rather than unrealised reasoning; reproducibility artefact provided.

517 **Two denominators on the rule-extension rerun.** With the input-shape contract honoured, the
 518 `Tensor.unbind(dim)` handler, the transposed-Parameter matmul rule (`Tensor.T / Tensor.t()`),

and the `nn.Embedding/nn.Linear/nn.LayerNorm` subclass recogniser all enabled, the rerun on the same 488 blocks returns 55 VERIFIED / 15 REFUTED-PROOF / 418 ABSTAIN. Of the 418 Abstains, 193 are *not-analysable*: the per-block extractor strips the surrounding class context for those rows and the analyser declines on absence of an `nn.Module` subclass. The two well-defined headline rates are therefore $15/488 = 3.07\%$ (Wilson 95% CI [1.87%, 5.01%]) over the *full* corpus and $15/295 = 5.08\%$ (Wilson 95% CI [3.11%, 8.22%]) over the *analysable* subset of $488 - 193 = 295$ blocks. The abstract reports the user-visible default-regime headline as the unconditional refutation count on the no-synthesised-assume subset of the corpus: 26 REFUTED-PROOF verdicts on the 356-block subset where the CONTRACT-VIOLATION classifier returns an empty caller-rely obligation, of which 5 fire through a handler chain whose detected operators lie entirely inside $\text{Cat}_{\text{sound}}$ (the audited footprint) — a small but non-zero in-theorem real-source catch population. Each of these five catches is itemised in Table 7 and pinned to the specific Lean lemmas that discharge its per-step obligations. The input-shape-contract rerun rates above characterise the corpus’s behaviour once a contract is supplied and serve as the falsifiability anchor for the 12-row LW→RP candidate table.

Smallest-cost LW→RP candidates. Among the 12 candidates the two smallest-cost extensions are (i) the `Tensor.unbind(dim)` fixed-length tuple shape rule (rows `ChannelAttention/ChannelAttentionV2`, both now measured RP via SHAPE-INCOMPATIBLE reshape) and (ii) the `nn.Embedding/nn.Linear/nn.LayerNorm` subclass recogniser (rows `BartLearnedPosEmb/BartScaledWordEmb/FalconLinear`). For the unbind rule its soundness obligation is exactly the static-axis-length lemma already in `??`. For the subclass recogniser the implementation adds the three inherited base classes to the module-class collector and injects symbolic shapes for inherited parameters (`self.weight`, `self.bias`), enabling the transposed-Parameter matmul rule (`x @ self.weight.T` via a dedicated TRANSPOSE step) to fire on `FalconLinear`: the analysis finds the code correct (Verified), not buggy. `BartLearnedPositionalEmbedding` fires RP via the DEAD-OUTPUT checker (a variable is computed but never used or returned), which the analyser can only reach once the class is admitted as an `nn.Embedding` subclass.

Per-block survival of the 57 Verifieds. Of the 57 VERIFIED verdicts under the synthesised config envelope, the user-visible (free-symbolic-config) recomputation collapses 23 to ABSTAIN and preserves 34 as Verified; the survivors are the larger and more structurally complex set (median forward length 51 LoC vs. 17 LoC). The per-block correspondence is in the reproducibility appendix. **On the bug corpus** TG issues REFUTED-PROOF on 53/60 (88.3%, Wilson 95% CI [77.8%, 94.2%]); this is the headline figure throughout the paper. *Sub-catalogue decomposition.* Of the 56 historical- corpus RPs (the headline-regime 53 plus three additional high-confidence catches with the runtime-supplied input-shape dictionary), the bug-firing handler chain lies entirely inside $\text{Cat}_{\text{sound}} = \text{Cat}_{\text{audit}} \cup \text{Cat}_{\text{pen}}$ on 46/56 (82.1%) of catches and touches at least one $\text{Cat}_{\text{tested}}$ handler on the remaining 10/56 (17.9%); concretely the out-of-soundness firings split as `batch_norm` (4), `cross_entropy` (3), `max_pool2d` (1), `repeat_interleave` (1), `linalg` (1) (reproducibility: per-bug primary-handler/scope mapping, total 56 rows). On the 10-bug upstream-faithful re-extracts, all 7/7 RPs lie entirely inside $\text{Cat}_{\text{sound}}$. Combined, 53/63 (84.1%) of bug-corpus RPs sit inside the soundness theorem’s footprint; the remaining 10/63 are flagged as “empirical RP only, outside Theorem 2”. The same 60-bug corpus, when each repro is supplemented with the concrete input-shape dictionary that the bug report bundles for runtime reproduction, produces 56/60 *Refuted-Proof* at the same 0.99 confidence threshold. The three additional refutations (bugs 003, 004, 005) require a concrete batch- or channel-constant from the caller to refute and are therefore beyond what a free-symbolic analysis of the model body alone can establish; they are not low-confidence heuristic fires. We carry 53/60 throughout because it corresponds to the harder zero-input-shape regime in which the verifier is given no contextual information from the caller; the 56/60 figure is the one reported as “refuted” in the per-feature ablation, which mirrors the input-shape regime. A single end-to-end command emits both numbers (the count, the high-confidence sub-count, and the per-item verdict pair) so the two figures are reconciled by direct re-execution rather than by narrative. The 10-bug *real public-repo bug corpus* (Table 2) is the upstream-faithful re-extract that preserves each `nn.Module` class with its real `__init__` (binding constructor scalars used by the buggy view target). On these, TG returns 7/10 REFUTED-PROOF at confidence ≥ 0.99 (Clopper-Pearson 95% CI [34.8%, 93.3%], `rb_003–rb_007`, `rb_009`, `rb_010`) plus 1/10 at confidence 0.80 (`rb_008`); the residual 2/10 (`rb_001`, `rb_002`) are silent verifieds whose buggy and correct view targets agree on total element count for the supplied input shape, so no shape-arithmetic check could refute them.

A separate *shape-pattern coverage suite* of hand-distilled one-liners on which TG returns 10/10 REFUTED-PROOF at 0.99 is relegated to Section J as a catalogue-coverage sanity check rather than a live-class evaluation, and is excluded from headline counts. The constructor-bound integer-attribute envelope synthesiser that lifts the upstream-faithful corpus to 7/10 unconditional RP — a three-stage pipeline (init-time local-scalar fold; single-dim shape-alias recogniser; shape-tuple propagator with starred view(`*new_shape`) expansion) — is described in Section 3.

Cross-family coverage: Llama 2/3, Qwen2, Mistral, Gemma, Phi-3. To assess generalisability beyond the torchvision/timm/BART training corpus, we run TG on six representative Llama 2/3 architecture modules (LlamaMLP, LlamaRMSNorm, LlamaRotaryEmbedding, LlamaAttention, LlamaDecoderLayer, and a deliberately mismatched LlamaMLP_buggy variant with `intermediate_size` 11008 used in a context expecting 8192). TG returns VERIFIED on 4/6 modules (LlamaMLP, LlamaRMSNorm, LlamaRotaryEmbedding, LlamaDecoderLayer) and REFUTED-PROOF on 2/6: LlamaAttention fires a division guard (the static analyser cannot rule out a zero `num_heads` without an explicit precondition, so this is a conservative refutation rather than a caught real bug), and LlamaMLP_buggy fires SHAPE-INCOMPATIBLE on the intentional 11008/8192 mismatch. We extend the cross-family evaluation by four further decoder families that are *not* in the 488-block real-source corpus and are *not* covered by the Llama set: **Qwen2** (5 modules: Qwen2RMSNorm, Qwen2MLP, Qwen2GQAAttention, Qwen2DecoderLayer, plus a deliberately broken Qwen2MLP_buggy with 18944/12288 split), **Mistral 7B** (5 modules with sliding-window GQA and SwiGLU MLP, plus a 14336/11008 mismatch variant), **Gemma** (5 modules with (1+weight) RMSNorm scaling and GeGLU MLP, plus a 24576/16384 GeGLU width mismatch variant), and **Phi-3** (5 modules with a fused `gate_up.proj` and a fused QKV projection, plus a $1\times$ vs. $2\times$ `chunk(2)` mismatch variant). Aggregated across all five families (26 modules total: 6 Llama + 5 Qwen2 + 5 Mistral + 5 Gemma + 5 Phi-3), TG returns VERIFIED on 19/26, REFUTED-PROOF on 6/26 (the five intentionally-broken variants are all correctly refuted with shape-incompatible explanations spanning three distinct upstream-realistic bug classes — `intermediate_size` mismatch, GeGLU width mismatch, and fused-projection chunk-count mismatch — plus the LlamaAttention division guard discussed above), and ABSTAIN on 1/26 (GemmaRMSNorm, where the $(1.0 + \text{Parameter})$ scalar broadcast is not yet propagated by the current handler set, so the verifier abstains rather than overclaim). One additional clean module (Phi3SdpaAttentionFusedQKV) returns a known false-positive Refuted-Proof on the symbolic-slice handler over the fused-QKV projection, which is logged as a known limitation of the slice handler on fused projections rather than papered over. None of the 26 modules appeared in the 488-block training corpus; the per-module verdicts and runtimes are released in the reproducibility appendix as a cross-family coverage datapoint, with 5 distinct decoder families and 5 distinct shape-arithmetic bug classes refuted.

Naturally-occurring cross-family bugs. The cross-family refutations described above include deliberately-broken variants used as positive controls. To isolate what TG catches on *naturally-occurring* upstream defects — not authored by us — we further evaluate on 7 minimal modules transcribed verbatim (or minimally adapted to a self-contained `nn.Module`) from real HuggingFace transformers bug-fix PRs and issues across four decoder families: Llama (PR #24815 rotary-emb head_dim mismatch; PR #29445 MLP intermediate-size mismatch), Qwen2 (PR #28857 attention head_dim reshape; issue #29733 target-dim reshape), Mistral (PR #27931 GQA repeat_kv projection mismatch; issue #27330 attention-output head arrangement), and Phi-3 (PR #29055 fused-QKV slice). Each module reproduces the buggy shape disagreement that landed in (or was reported against) the upstream library, with the public PR/issue number cited per row. TG returns REFUTED-PROOF on 7/7 of these naturally-occurring cases, with shape-incompatibility traces that pin the defect to the same operator the upstream patch modified. A round-5 cross-family extension to a fifth decoder family, Gemma 2, adds two further upstream-faithful repros (head_dim-divisibility violation and a per-head reshape mismatch in the GQA path); TG returns REFUTED-PROOF on 2/2, lifting the cross-family naturally-occurring tally to 9/9 across five decoder families. This is the natural-distribution bug-finding result on cross-family decoder source that the prior cross-family evaluation, which leaned on injected variants and the conservative LlamaAttention division guard, did not directly demonstrate. *Execution-based baselines on the same 9 minimal repros.* We additionally rerun `torch.compile` (`fullgraph=True`, `dynamic=False`) and `FakeTensorMode` on the 9 self-contained `nn.Modules` and they likewise raise on 9/9 during tracing. This is the expected outcome whenever the buggy fragment has been reduced to a constructor-default-instantiable `nn.Module` with concrete tensor inputs: the execution-based baselines have a tracer to run. The TG advantage on this family is

Table 2: Real public-repo shape bugs (upstream-faithful). Catalogue freeze: commit 040f6f3, 2026-04-07; rule-catalogue and per-PR first-read dates in the reproducibility appendix. Each row reproduces the upstream `nn.Module` class with its real `__init__` and multi-step forward; TG catches 7/10 at ≥ 0.99 (**bold**) plus 1/10 at 0.80 (*italic*); 2/10 are semantically-aliased silent verifieds (–; see Section 6). A separate *shape-pattern coverage suite* of hand-distilled `forward(x): return x.view(...)` one-liners is reported in Section J as a catalogue-coverage sanity check rather than a live-class evaluation, and is excluded from the headline counts.

ID	Repo / Model	Fix link	Bug class	Upstream-faithful
rb.001	HF transformers / xLSTM (matQ)		view total-size	–
rb.002	HF transformers / xLSTM (matK)		view total-size	–
rb.003	HF transformers / GPT-NeoX		QKV view dim	0.99
rb.004	HF transformers / ConvBERT		head_ratio view	0.99
rb.005	HF transformers / Longformer		axis swap	0.99
rb.006	HF transformers / LongT5		TP-shard divis.	0.99
rb.007	EleutherAI / gpt-neox		GQA truncation	0.99
rb.008	HF diffusers / UNet1D		embed dim	<i>0.80</i>
rb.009	HF peft / PrefixTuning		view total-size	0.99
rb.010	HF peft / DoRA Conv2d		groups view	0.99

therefore not a head-to-head margin on the reduced self-contained repro — it is that TG also fires on the *original class source* as it appears in the upstream transformers package, where the same operator chain sits inside an attention block whose constructor requires a populated `config` object and which therefore is not instantiable on the target machine without first reconstructing the whole library configuration. The execution-based baselines’ $\geq 435/488$ mechanical N/A on the 488-block real-source corpus is the corresponding measurement on naturally-occurring *class source* (rather than reduced repros).

Mechanical N/A on $\geq 435/488$ blocks for the PyTorch baselines and Pytea is a structural fact about constructor-argument requirements — it is *not* a TG win over those baselines. The fair head-to-head is the *modern subset* of the 60-bug corpus restricted to operators in Pytea’s 2022 catalogue ($N=34$): TG 32/34 (94.1%, Wilson 95% CI [80.9%, 98.4%]) vs. Pytea 25/34 (73.5%, [57.0%, 85.5%]). A 7-point gap on $N=34$ is the comparable number, not the 488-block coverage triple. The matched-pair test on this subset (Pytea-refutes is a strict subset of TG-refutes, so $b=7$ TG-only catches and $c=0$ Pytea-only) gives McNemar’s exact two-sided $p=0.0156$ and a paired-bootstrap 95% CI on the refute-rate difference of [+8.8 pp, +35.3 pp] around a point estimate of +20.6 pp; the lower bound is well above zero so the gap is not a small- N artifact. Pytea inapplicability rows are treated conservatively as not-refute in the headline; the alternative silent-skip-reclassification convention gives $b=10$, $p=0.00195$, recorded alongside the headline in Section L so the convention choice is auditable. The full 34-row matched-pair contingency table (each row shipped with bug-id, primary operator, TG verdict under fragment-fairness enforcement, and Pytea verdict) is included in Section L so the $b=10/c=0$ structure is directly auditable per bug. The 32/34 figure is reproduced *at verification time* (not by post-hoc filtering of a verdict log) by a small membership predicate that AST-screens each repro for non-catalogue ops (forced-abstain on hit) and additionally forensics-scans TG’s emitted `Bug.message` stream for non-catalogue operator names (forced-abstain post); the resulting fragment-fair count is recorded in the reproducibility appendix.

Pytea baseline: catalogue confound and modern subset. 23/29 of Pytea’s misses on the 60-bug corpus are pure operator-catalogue gaps (no Pytea handler for SDPA, RMSNorm, einsum, Conv1d/3d, GroupNorm, BatchNorm1d, etc., verified by an AST scan of the upstream Pytea operator dispatch); the remaining 6 are symbolic-fragment weaknesses (e.g. no divisibility for `view(-1)`). On the *modern subset* of the 60-bug corpus restricted to operators in Pytea’s 2022 catalogue ($N=34$), TG 32/34 (94.1%) vs. Pytea 25/34 (73.5%)¹ — still a significant gap, isolated from the catalogue confound.

¹The 25/34 figure is the *conservative* convention: each Pytea “inapplicable” or silent-skip row is counted as a non-refutation. Under the alternative convention that reclassifies the two silent-skip rows (#13 and #23 of Table 10) as Pytea catches, the count becomes 22/34; the 3-catch discrepancy is therefore a labelling convention, not a verdict disagreement, and the matched-pair structure ($b=7$ TG-only, $c=0$ Pytea-only) is invariant under either convention.

Pytea modern-subset filter — explicit protocol. The modern-subset filter is a static AST predicate over the 60-bug repros: a bug b is included iff every operator called from `forward` appears in Pytea’s operator dispatch as of commit `cb02a8a` (2022-04-26). The full inclusion list and the membership rule are released as data; the membership table is the per-bug contingency in Table 10. The *same* fragment fairness is enforced on TG: TG is restricted to its 2022-catalogue handler intersection at verification time (no TG-only operator handler is allowed to fire), so the 32/34 figure is on the same operator surface as Pytea’s 25/34. The full per-bug verdict pairing (Section L) shows that Pytea-refutes is a strict subset of TG-refutes on this corpus ($a=25$ both-refute, $b=7$ TG-only, $c=0$ Pytea-only, $d=2$ neither), so the matched-pair structure is reported directly from the per-bug log without any silent-skip reclassification. For completeness on the question of whether Pytea has acquired new handlers between 2022 and the present submission, the upstream Pytea repository (`ropas/pytea`) has zero commits after `cb02a8a` (2022-04-26): the 2022 catalogue is the 2024–26 catalogue. The 32/34 vs. 25/34 headline is therefore already the modern-mirror intersection; we record this as a structural rather than empirical answer (verifiable from the upstream commit log).

Contemporary execution-based baselines on the same 34 bugs. Two off-the-shelf contemporary tools were evaluated on the identical fragment-fair subset. First, hand-annotated `jaxtyping` 0.3.9 shape contracts enforced by `beartype` 0.22.9 on the natural function boundary of each repro: the contracts add no catches over the underlying torch runtime, 0/34, because the faulty operation lies inside the body and the boundary is crossed only with already-correct argument shapes. These runtime contract tools are complementary to, not a substitute for, static class-source analysis. Second, `torch.compile` (PyTorch 2.9.1) under the FakeTensor backend with `fullgraph=True`: in both `dynamic=False` and `dynamic=True` modes the trace surfaces the shape error at compile time on 34/34 repros, exceeding TG’s 32/34 on this subset. The setting asymmetry is the point of the comparison: `torch.compile` requires both an instantiated module and concrete example inputs to begin tracing; on the 488-block real-source corpus, 481/488 blocks have neither, which is the inapplicability observation already reported. TG is the only tool of the four that operates from class source alone with no instantiated module and no example inputs; within that operating regime the 32/34 vs. 25/34 gap stands, and outside it `torch.compile` is the strongest available baseline. Per-bug verdicts and the comparison protocol are released with the implementation.

Worked symbolic-calculus example. To make the rule-driven Z3 path concrete we walk a single post-freeze real-PR bug end to end — a GPT-NeoX QKV view-reshape bug whose forward fragment projects x through a linear layer to $3 \cdot H$ features per token and then views into per-head $(B, T, N, 3 \cdot \lfloor H/N \rfloor)$. The T-View typing rule fires and emits the per-head divisibility obligation $N \mid H$, reified as the integer-arithmetic predicate $B \cdot T \cdot 3 \cdot H = B \cdot T \cdot N \cdot 3 \cdot \lfloor H/N \rfloor$ with the floor-division semantics $N \cdot hd \leq H < N \cdot (hd+1)$. Under the buggy guard $H=1024, N=12$ Z3 returns SAT for the negated obligation, producing the Refuted-Proof witness $\langle H=1024, N=12, hd=85, B=1, T=5, in_numel=15360, tgt_numel=15300 \rangle$; under the corrected guard $H=1024, N=16$ ($hd=64$) the same query is UNSAT and the verifier passes. An AST-pattern matcher cannot in principle decide such a divisibility predicate in module-config symbols; the SMT-LIB query and witness are released in the reproducibility appendix.

Post-freeze generalisation. To rule out the concern that the 60-bug corpus and the 10-bug upstream-faithful re-extracts could have been used to retro-fit handlers, we mined a *post-freeze* generalisation set of fix-PRs filed *strictly after* the catalogue freeze date **2026-04-07** (commit `040f6f3`; freeze hash `cc75834b` . . . , recorded in the reproducibility appendix). $N=6$ post-freeze upstream-faithful repros were ported verbatim from upstream HuggingFace transformers, diffusers, and peft fix-PRs into the post-freeze evaluation harness. TG was run with *no rule edits* after freeze (verified by the freeze hash). The post-freeze headline is 3/6 **Refuted-Proof at confidence** ≥ 0.99 (`rb_pf_001`, `rb_pf_003`, `rb_pf_004`) plus 0/6 at 0.80 and 3/6 silent verifieds (`rb_pf_002`, `rb_pf_005`, `rb_pf_006`). The three remaining silent misses fall in the same constructor-bound integer-attribute envelope class flagged in Section 6 (an `int(dim*ff_mult)` or `q_len = sliding_window+1` arithmetic chain whose unsoundness is on a numerical comparison the TG fragment does not currently fold), with no new silent-miss class introduced post-freeze. The `rb_pf_001` result — REFUTED-PROOF@0.99 on the intermediate `= int(dim*ff_mult)` *Linear-chain in/out feature-dim* pattern — comes from numeric `int/float/round` casts now folded when the inner expression is statically known.

Unfiltered pre-registered post-freeze sample. The earlier post-freeze corpus ($N=6$) was filtered for "self-contained CPU repro" and so still favoured the TG fragment. We extend it to a strictly broader $N=15$ *unfiltered* sample drawn by a pre-registered GitHub-search query (, pre-registered **2026-04-08**, one day after the freeze) across,, and, taking the first 15 matching merged PRs in chronological order with *no further filter* for fragment fit; out-of-fragment cases (distributed-shape, dtype, autograd parameter sharing, data-dependent control flow) are explicitly retained as explicit AB-STAIN/silent-verified entries. Table 3 reports the per-tool verdict triple. TG *catches the upstream-fixed bug* on 5/15 entries (33.3%, Wilson 95% CI [15.2%, 58.3%]): rb_pf_001, rb_pf_003, rb_pf_004, rb_uf_008, and rb_uf_012. The headline catch rate is the comparable to FakeTensorMode's 2/15 (13.3%, [3.7%, 37.9%]) and Pytea's 3/15 (20%, [7.1%, 45.2%], silent-skip-corrected). The pairwise Fisher-exact two-sided p -values are 0.39 (TG vs. FakeTensorMode) and 0.68 (TG vs. Pytea), and after Benjamini–Hochberg correction across the three pairwise tests every adjusted p sits at 1.00. The point-estimate gap is descriptive only, not statistically separable at $\alpha=0.05$ on $N=15$. We report intervals explicitly here rather than treating 5/15 as a point estimate. The headline is therefore 5/15 *point above* the two execution-based baselines, not a separation; we do not rely on a Bayesian supplement to upgrade the claim. We separately report a *Refuted-Proof fire rate* of 6/15 (40%): TG also fires on rb_uf_010, but along an axis (device mismatch on the same module) different from the dtype root cause of the upstream PR; we record this off-axis fire as a false positive against ground truth and *do not* count it as a catch in the headline. rb_pf_001 and rb_uf_012 are catches enabled by the `int(...)/float(...)` cast folding that closes the `intermediate = int(dim*ff_mult)` pattern. The remaining 3/5 catches (rb_pf_003, rb_pf_004, rb_uf_008) are caught by the v4 path (operator-rule application without the init-time local-scalar fold or the single-dim shape-alias recogniser): on retrofit TG-v4 binaries (catalogue freeze without the three constructor-bound integer-attribute additions), the headline collapses to 3/15 rather than 5/15. The 2/5 marginal attribution is therefore the contribution of the constructor-bound integer-attribute envelope to the unfiltered post-freeze headline. FakeTensorMode catches 2/15 (13.3%, rb_uf_008, rb_uf_009); Pytea catches 3/15 (20%, silent-skip-corrected). The post-freeze unfiltered catch rate (33.3%, 95% Wilson [15.2%, 58.3%]) is the calibrated denominator; TG's point catch rate is descriptively above the two execution-based baselines on the same unfiltered surface, although on $N=15$ the head-to-head Fisher-exact comparisons are not statistically separable at $\alpha=0.05$ (two-sided $p=0.39$ vs. FakeTensorMode, $p=0.68$ vs. Pytea, all BH-adjusted to 1.00); we report this as a calibrated-confidence result rather than a significance claim. A Benjamini–Hochberg correction across the three pairwise Fisher-exact tests (TG vs. FakeTensorMode, TG vs. Pytea, FakeTensorMode vs. Pytea: raw two-sided $p=0.39, 0.68, 1.00$) leaves every BH-adjusted p at 1.00, so no pairwise contrast survives at $\alpha=0.05$ after correction either. A power calculation on the observed point estimates — conditioning on 5/15 for TG, 2/15 for FakeTensorMode, and 3/15 for Pytea, and adding a fresh second wave N_{new} sampled at the same per-tool rates — locates the smallest second wave at which the head-to-head Fisher exact reaches $p < 0.05$ on either pair: $N_{\text{new}}=26$ for TG vs. FakeTensorMode (one-sided), $N_{\text{new}}=77$ for TG vs. Pytea (one-sided), and $N_{\text{new}}=56$ for either pair to reach a Bayes factor $\text{BF}_{10} \geq 10$ under a uniform prior. These are pre-conditions for a second wave, not a claim of significance on the present $N=15$. *Soundness footprint of the 6 RP fires.* On a per-fire classification of the operator handlers each catch traverses against the soundness scope of Table 9, **all 5/5 catches** touch at least one Lean-audited shape-transfer rule (linear for rb_pf_001 and rb_pf_004; view/reshape for rb_uf_008; view/permute/conv2d for rb_uf_012; einsum/expand for rb_pf_003). The bug-firing handler in each case is within the in-soundness $36+13=49$ -handler footprint of Theorem 2; the tested-only handlers that co-occur in the same forward body (softmax and unsqueeze are themselves now in the in-soundness footprint after the flatten/squeeze/unsqueeze/softmax view-reducible promotions, so the co-occurring tested-only handlers in this set reduce further) are not on the bug path. The false-positive fire rb_uf_010 (device-mismatch handler) also touches at least one tested-only handler (the device-mismatch check is not in the Lean-mechanised scope), so all 6/6 RP fires are *mixed* (they touch both in-soundness and tested-only handlers); zero fires traverse *only* in-soundness handlers. Across the union of all four corpora (60-bug, 10-real-bug, and $N=15$ post-freeze): 66 TP, 1 FP (rb_uf_010); union precision = $66/67 = 0.985$ (Wilson 95% CI [0.920, 0.997]). The full per-PR table, the cached verdict JSON, the manifest, and the verifier are released in the reproducibility appendix.

Bug-corpus exclusion denominator. The historical 60-bug corpus selected from GitHub issues filtered the 1,087 initial keyword-search hits described in the reproducibility appendix via four exclusion rules: (i) not-a-shape-bug (~ 612 , 56%: hardware/CUDA/build/installer is-

Table 3: Unfiltered pre-registered post-freeze real-PR sample ($N=15$, query and inclusion rule frozen on 2026-04-08). TG catches the upstream-fixed bug on 5/15 entries (33.3%); the point estimate sits descriptively above FakeTensorMode (2/15, 13.3%) and Pytea (3/15, 20%, silent-skip-corrected), but no pairwise contrast is statistically separable at $\alpha=0.05$ on $N=15$ (BH-adjusted Fisher-exact $p=1.00$ on every pair). A sixth REFUTED-PROOF fire (rb_uf_010) lands off-axis (device-mismatch where the upstream PR fixes a dtype bug) and is recorded as a false positive against ground truth, not as a catch; the comparable RP-fire rate is 6/15 (40%). 4/15 are out-of-fragment principled abstentions (distributed/data-dependent/dtype/autograd-sharing).

	TG	FakeTensorMode	Pytea
Catches upstream bug	5/15	2/15	3/15
RP fire (any axis)	6/15	2/15	3/15
Off-axis fire (FP)	1/15	0/15	0/15
Silent verified	9/15	0/15	2/15 (excl.)
Abstain / N/A	0/15	13/15	10/15

sues mis-tagged with shape-related keywords); (ii) closed user-error (no upstream fix; ~ 178 , 16%); (iii) *distributed-shape* bugs whose shape arithmetic crosses a process boundary (`torch.distributed.{all_reduce, all_gather, broadcast}`; ~ 124 , 11%, all marked ABSTAIN in our taxonomy and excluded by protocol); (iv) *config-attribute* bugs that reduce to constructor sentinel-resolution rather than to shape arithmetic (~ 113 , 10%, the same silent-miss class flagged above in Section 6). 60 remain. In particular, the (iii)+(iv) exclusion of $\sim 237/1,087$ hits ($\sim 22\%$) is a known scope cap, not a sample-selection manipulation: the same exclusion is applied to the post-freeze corpus above (e.g. rb_pf_005 required no exclusion under (iii) or (iv)).

RP rate on the config-attribute exclusion slice. The 113 bugs excluded under rule (iv) are minimal repros of 14 canonical config-attribute archetypes (head-count divisors, gated-MLP dimensions, GQA parameters, etc.) spanning 72 buggy archetypes and 41 clean perturbations. On this slice TG returns 0/113 unconditional REFUTED-PROOF; 16/113 are SILENT-VERIFIED (the static-integer subset populates the assume envelope but the buggy comparison is on a constructor scalar the Z3 fragment does not fold) and 97/113 ABSTAIN/VERIFIED. This is the operationally correct behaviour of the symbolic-config front-end on bugs whose root cause is a wrong integer constant in the constructor rather than a symbolic shape mismatch in the forward pass. The 53/60 headline is therefore conditional on excluding this regime; the 0/113 number is the corresponding scope bound.

Rule-development holdout (60-bug corpus). The 60-bug historical corpus was assembled by keyword search without first inspecting any forward body; the operator catalogue was enumerated from `torch.nn.modules` and `torch._refs`. Two leave-one-out audits are reported in the reproducibility appendix: a category-keyword LOO (no-op by design, disables orchestration modules) and a handler-class LOO (removes the operator handlers most directly on each category’s bug path). Both leave the aggregate RP rate at 53/60. The signal carried by the handler-class LOO is the silent-to-error transition rather than RP degradation: under every per-category handler removal the seven silent mis-verifications surface as analyser errors instead, with the shift concentrated in the held-out category (`view_reshape.total.size: 2 silent→err; broadcasting: 2 silent→err; attention.dim: 1 silent→err`) and the global silent→err count exactly equal to the seven silent catalogue gaps quoted elsewhere. The RP-resilience to single-category removal is itself a design finding: the catalogue handlers form a redundant network, so each catch is load-bearing for coverage but not for RP-attribution. To isolate the operator-rule contribution from the AST-pattern contribution, two further reproductions report the same number on each path in isolation: with the *AST-pattern path disabled* (AST-pattern-disabled-only) the operator dispatch alone refutes 53/60, and with the *operator dispatch globally disabled* the AST-pattern path alone also refutes 53/60. The catalogue is therefore over-determined: the two paths each independently saturate the bug surface, and per-bug attribution overlaps almost everywhere. A quad-disabled configuration (operator dispatch + AST pattern + constraint residue all switched off) returns a degenerate 53/60 from a single ‘no nn.Module subclass found’ parser-failure marker on the class-stripped repro files; that count is a parser artefact, not a fourth refute path, and the post-quad-LOO refute count with the parser-failure marker filtered out is 0/60. Per-handler keyword attribution (load-bearing share) gives `view_reshape.total.size`

Table 4: Hybrid falsification corpus ($N=25$ importable, no-arg `nn.Module` blocks). Cells are block counts.

	FT REFUTED	FT VERIFIED	Total
TG REFUTED	0	20	20
TG VERIFIED/ABSTAIN	5	0	5
Total	5	20	25

and broadcasting 7 each, `conv_channel_mismatch` 6, `einsum_dim` 5, and four further handlers 4 each.

Block-corpus diversity. AST-skeleton clustering of the 488 blocks (`literals`→`_INT_`, `attrs`→`_ATTR_`, vars positionally renamed, then SHA-256 of the normalised `forward` body) yields $K_{\text{ast}}=406$ distinct skeletons; handler-call multiset clustering (Jaccard ≥ 0.85) yields $K_{\text{handler}}=345$. The largest cluster has 20 members (DeepLabV3 variants); 369/406 skeletons are singletons. The 488-block corpus is therefore not a few near-duplicates dressed in size — the effective number of distinct shape-reasoning challenges is ≥ 345 .

Importable vs. unimportable surface. On the 488-block corpus, where most blocks need a `config` object, the abstention surface and the non-importable surface coincide; this is corpus-baked. We falsify a stronger reading by building a 25-block stress set of *importable, no-arg-constructor* `nn.Modules` selected to exercise (i) symbolic-shape refutation under a contract no concrete trace satisfies, (ii) `requires_grad / detach` flow, and (iii) concrete shape arithmetic recoverable only by execution. The contingency table is Table 4: TG-only catches 20 (12 symbolic shape, 8 grad-flag); FakeTensorMode-only catches 5 (concrete `.size()`-arithmetic outside the TG fragment); the two surfaces are *complementary* on this hand-designed importable stress set. We do not claim general complementarity on a natural distribution: on the 488-block real-source corpus hybrid mode returns the same {57, 206, 225} verdict triple as TG alone (zero gain), and the result reported here should be read as an *existence* demonstration that complementary cases are constructible, not as a distributional improvement.

4.2 Per-feature ablation and localisation / hybrid-mode results

Per-feature stress benchmark. On the 488-block + 60-bug aggregate corpus the per-feature ablation is a flat line: every analyser knob (CEGAR, device-consistency, phase-check, gradient-flow, low-confidence-violations) leaves both the 53/60 bug-refute count and the 46.1% block-abstain rate unchanged. To test whether each feature *can* discriminate, we built a 25-case stress benchmark (5 cases per feature), where each case is hand-designed so that exactly one feature fires the verdict change.

Live secondary-check demonstration on real source. Beyond the stress benchmark, three small but realistic `nn.Module` sources — a residual block whose bias is moved to a different device than the activation, a head whose shape diverges between the train and eval branches of an `if self.training` switch, and a block wrapped in activation checkpointing that lifts it out of the first-order grad-flow fragment — are committed to the open-source implementation as a live end-to-end demonstration that the three secondary-check flags (`check_devices`, `check_phases`, `check_gradients`) flip the overall verdict on un-instantiated class source. On all 3/3 examples, toggling the corresponding flag from on to off (with the other two flags held at off) flips the analyser’s verdict from REFUTED-PROOF to VERIFIED, and the resulting verdict matrix — 3 examples $\times$ 8 flag combinations = 24 verdicts — is committed as a JSON artifact under the reproducibility directory. This converts the five-theory product domain (Shape $\times$ Device $\times$ Phase $\times$ Stride $\times$ Permutation) from a stress-benchmark proxy to a demonstrated contribution on real-source class code.

Table 5 below reports the *three discriminative knobs* (device-consistency, gradient-flow, and low-confidence violations) that move verdicts on the stress benchmark; the two non-discriminative knobs (CEGAR and phase) ship with the analyser but are recorded as zero-delta in the same staircase format and are not part of the ablation contribution. CEGAR’s internal predicates are stored as metadata but never surfaced as Bug objects (architectural gap between ShapeCEGARLoop and the

Table 5: Per-feature stress benchmark ($N=25$, 5 cases per feature). The headline result is the contribution of the **three discriminative knobs** (L2 device-consistency, L4 gradient-flow, L5 low-confidence): together they refute 18/25 stress cases. The two zero-delta rows (L1 CEGAR, L3 phase-check) are listed for completeness as *shipped, did not discriminate* no-ops in the current implementation; an additional source-level deletion audit confirms zero verdict-touching call sites outside their own modules (one consumer in the contrastive-explanation generator only suppresses explanation prose for already-decided verdicts). **Stress-only: this benchmark was constructed so that each feature would discriminate; do not read it as a real-corpus ablation. The corresponding real-corpus ablation on the 488-block + 60-bug aggregate is a flat line.**

Level	Feature added	Refuted/25	Δ
L0	base fragment, high-conf only, no CEGAR	0	—
L1	+ CEGAR (3 iters) [<i>shipped, no-op</i>]	0	0
L2	+ device-consistency check	5	+5
L3	+ train/eval-phase check [<i>shipped, no-op</i>]	5	0
L4	+ gradient-flow check	10	+5
L5	+ low-confidence violations	18	+8

verdict pipeline); the phase-check encodes $\text{Or}(\text{TRAIN}, \text{EVAL})$, which is always satisfiable. Both are shipped, did not discriminate on the aggregate corpus, and are recorded as such; the live real-source demonstration above shows that the phase check *does* discriminate when the source contains a multi-phase conditional with branch-divergent shapes.

Real-corpus ablation . We also re-ran the five-feature ladder of Table 5 on the upstream-faithful 10-bug real corpus from Table 2. The result is a flat line at (7/10 RP@0.99, 8/10 RP@0.80 cumulative, 2/10 silent verified) for every disabled feature: none of CEGAR, device-flag, phase, grad-flow, or low-conf gating discriminates on the real bugs. This corroborates the “stress-only” caveat already in the caption of Table 5 with a direct empirical measurement on the real-public-repo subset.

Localisation. On a 30-item marker-only audit — repros carrying an explicit # BUG comment as the ground-truth line, with no information shared with the AST-walk strategy — 17 of the 30 were refuted by TG with a computable distance to the marker. Of those 17, the “first unsatisfied symbolic constraint” tracer (Section 3.3) lands within ± 5 lines on 14/17 (82%) and within ± 1 line on 11/17 (65%). This is a calibration result: the tracer is accurate to the nearest few source lines on the bugs it refutes, but the 17 computable cases are the ones TG successfully catches, so the numerator is self-selecting. The 13 non-computable cases (no distance because TG did not refute or returned a `location.line=0` witness) are the relevant coverage gap. *End-to-end fraction.* The runtime-tracer-comparable headline is therefore $14/30 = 46.7\%$ within ± 5 lines (Wilson 95% CI [30.2%, 63.9%]), with the $14/17 = 82\%$ figure retained as a calibration on the catches. On a per-item analyser pass under the user-visible default configuration, all 13/13 resolve to *silent-verified* (no Bug, no Abstain), and 1/13 falls in the constructor-bound integer-attribute envelope class flagged in Section 6 (`rb_pf_006`); the remaining 12/13 are semantically-aliased-view bugs (the upstream and fixed view targets agree on total element count for the input shape that exercises the bug) plus three runtime-traceback grad-flow / cross-entropy weight cases whose root cause is not in the shape fragment. The full per-item breakdown is provided in the reproducibility appendix.

Hybrid mode on the 488-block corpus. `tensorguard.hybrid_check` returned the same {57, 206, 225} verdict triple as TG alone: of the 225 TG abstains, 155 failed at constructor-call time (`ctor_failed`) and 70 at `exec/import` (`exec_failed`). On this corpus the abstention surface and the FakeTensor-inapplicability surface coincide *because the corpus selects for non-importable code*; the 25-block falsification stress set above (Table 4) shows that complementary cases *exist* when the input is importable, but the zero-gain hybrid result on the 488-block corpus is the natural-distribution reading and the 25-block result is to be taken as an existence demonstration only.

4.3 Dynamo-guard correspondence

Framing. Theorem 13 below is a *necessary-direction inclusion* statement, not an equivalence: on the catalogue intersection it claims that every shape/dtype/rank guard Dynamo installs is a refinement

variable of some Section 2 rule. We deliberately do not claim that every Dynamo recompile is shape-driven; the empirical 48/544 (8.8%) in-contract recompile rate quantifies the gap to a two-directional reading. The proof reduces rule-by-rule to PyTorch 2.9.1’s specialiser bits as exposed by `torch._dynamo.symbolic_convert`; we pin the freeze SHA in Section F and mark the statement as audited against that release rather than as a moving-target equivalence.

Proposition 13 (Dynamo-guard correspondence; empirical, audited at PyTorch 2.9.1). *Let M be an `nn.Module` in the supported fragment with TENSORGUARD contract $C(M) = (A, G)$. Then, on the intersection of the TG fragment and Dynamo’s traceable subset, TG-abstention is necessary for guard-stability: if TG reports VERIFIED on M then every shape/dtype/rank refinement bit Dynamo reads on the trace at any input x with $A(x)$ true is a refinement variable in φ for some rule in the Section 2 catalogue. The converse is not promised in either direction: Dynamo additionally specialises on metadata outside the TG fragment (e.g. Python ints, list lengths, integer parameters captured at trace time), and therefore may install guards (and recompile) on inputs that satisfy $C(M)$. Outside the catalogue intersection, TG abstains and Dynamo breaks the graph. Status. The correspondence is established by per-rule inspection against the SHA-pinned PyTorch 2.9.1 specialiser (Section F); it is therefore an empirically audited statement at a fixed release rather than a structural theorem, and any future PyTorch release that changes the specialiser bits requires a re-audit. We label it PROPOSITION (rather than THEOREM) to make this provenance explicit.*

Sketch. Each typing rule’s premise is a function of fake-tensor metadata bits Dynamo’s specialiser reads on the matching aten call (T-MATMUL reads k and the broadcast batch shape, T-VIEW(-1) reads element count and explicit factors, etc.); details in Section F. The reverse direction fails by inspection: Dynamo’s guard set strictly extends the shape contract. $\square$

Empirical audit. We instantiate 17 modules from torchvision 0.24.1, HuggingFace 4.57.3, timm 1.0.26, plus a hand-built TG-verified TinyMLP as positive control. For each module we fix TG’s contract, sample 32 in-contract inputs and 2–3 out-of-contract perturbations, and run `torch.compile(dynamic=True)`. Aggregate in-contract recompile rate is 48/544 (8.8%) — the calibrated reminder of one-directionality, since Dynamo also specialises on metadata strictly outside the TG fragment (Python ints, list lengths) — and the out-of-spec violation rate is 46/47 (97.9%). Per-module breakdown is in Section F. On 16 of the 17 modules the audit takes the documented forward signature as the contract because the full instantiated module exceeds end-to-end constraint solving; we therefore treat the 8.8%/97.9% row as a calibration of Dynamo against hand-written contracts rather than as an in-theorem instantiation of Theorem 13.

Extended end-to-end audit on 14 modules. To expand the end-to-end evidence base for Theorem 13 we run a broader audit: 14 importable `nn.Module` subclasses (9 CNN-type: BasicBlock, Bottleneck, InvertedResidual (MobileNetV2), Fire, ConvNeXtBlock, DenseLayer, ShuffleNet InvertedResidual, MobileNetV3 InvertedResidual, RegNet Bottleneck; and 4 transformer-type: timm ViT Block, Swin SwinTransformerBlock, MLP-Mixer MixerBlock, torchvision EncoderBlock) plus one ResNet50 layer. Each block is compiled under `torch.compile(dynamic=True)` with 24 varied inputs. The aggregate result is 19 recompile events, all classified as $\{\text{SHAPE}:19, \text{DTYPE}:0, \text{RANK}:0, \text{INT}:0\}$. All 19 are on input-shape refinement variables in the TG catalogue; zero guards are outside the catalogue, so the falsification predicate evaluates to false on this corpus. *CNN-only restriction.* Restricting the aggregate to the 10 fully end-to-end CNN-type subjects (excluding the 4 transformer-surrogate blocks) gives 13 recompile events, all classified as $\{\text{SHAPE}:13, \text{DTYPE}:0, \text{RANK}:0, \text{INT}:0\}$ and all on input-shape refinement variables in the TG catalogue; the falsifier is therefore false on the CNN-only restriction as well. We treat the CNN-only restriction as the headline for the necessary direction of Theorem 13; the full 14-module aggregate is reported alongside as context. *Scope.* The four transformer blocks are audited via the documented forward-signature surrogate; their full instantiation (window-partition, positional-encoding dispatch) exceeds end-to-end constraint solving. This is an inherent limitation of the current implementation on large transformer components. The nine CNN-type blocks are fully end-to-end. The per-module breakdown ($\{\text{SHAPE} : k\}$ per row) is provided in the reproducibility appendix. An additional 3 sublayer modules drawn from the T5 and BERT families of the HuggingFace Transformers library were verified end-to-end and all 3/3 returned SAFE under TG; under `torch.compile(dynamic=True)` each subject completed 24/24 in-contract forward passes with 1 recompile (initial warm-up only), exercising the same necessary-direction inclusion as the CNN corpus above.

Falsification predicate for Theorem 13. A measurement that *would have* falsified the necessary direction is: $\exists$ in-contract recompile r with $r.guard_kind \in \{\text{SHAPE}, \text{DTYPE}, \text{RANK}\}$ and $r.guard_var \notin catalogue(M)$. The extended 14-module audit finds zero such events; the falsifier evaluates to false on this corpus. An adversarial custom op that reads a shape bit not declared by any rule could in principle fire the falsifier on future modules; we exhibit a curated $N=5$ corpus of such fixtures (one each of Shape, Dtype, Rank, interior-shape, and numel-product reads) in the reproducibility appendix that wrap `torch.library.custom_ops` reading variables outside the surrounding module’s declared `catalogue(M)`. On this $N=5$ corpus the falsification predicate is non-vacuously evaluable on 5/5 events of kind $\neq$ INT, and the necessary direction of Theorem 13 still holds on 5/5 rows: every analyser refinement variable for each module is in `catalogue(M)`; each custom op’s non-catalogue read is a strict-subset witness, not a counterexample.

Audit on a strictly larger module population. We extend the necessary-direction empirical surface to a strictly larger candidate population: importable `nn.Module` blocks drawn from `torchvision 0.24.1`, `HuggingFace 4.57.3`, and `timm 1.0.26`, run in a subprocess-isolated worker under `torch.compile(dynamic=True)` with a 240s wall-clock kill, 3 warm-up inputs, and 7 in-contract follow-ups per module. Of 107 candidate modules, 55 completed Dynamo warm-up; 35 exceeded the 240s wall-clock kill and 17 failed warm-up. The same falsification predicate as above is evaluated on each successfully-instantiated module: a SHAPE/DTYPE/RANK guard on a variable outside the catalogue `catalogue(M)` would falsify the necessary direction. Across the 55 successful modules we observed 72 in-contract recompiles, all classified as integer / `SymInt` specialisations (kind INT); the falsification predicate is therefore not exercised on this population (denominator 0 SHAPE/DTYPE/RANK guards), which is itself a measurement that the larger `torch.compile` flow on long-tail modules is dominated by integer specialisation rather than shape guards. A further extension of the same harness to a strictly larger candidate pool — 146 candidate modules drawn from `torchvision 0.24.1`, `HuggingFace 4.57.3`, `timm 1.0.26`, an `AutoModel.from_pretrained` snapshot family, and a positive-control sentinel set, of which 67 completed Dynamo warm-up under the same 240s kill (56 subprocess timeouts and 7 warm-up failures excluded) — confirms the denominator stays at 0 SHAPE/DTYPE/RANK in-contract guards and 0 Theorem 5 falsifier events; the in-contract recompile total on this larger pool is 0, because the additional `torchvision` and `timm` vision backbones do not enter the dynamic-input specialisation regime that produced the 72 INT recompiles on the smaller `HuggingFace`-heavy pool. The two audits therefore agree on the only quantity Theorem 13 requires (zero falsifier events, zero out-of-catalogue SHAPE/DTYPE/RANK guards) and disagree only on the in-contract INT-recompile density of the underlying module distribution; both are reported in the reproducibility appendix. We therefore treat the CNN-only 13 SHAPE-recompile restriction above as the *primary* test of the falsification predicate; the 55- and 67-module populations are reported as denominator audits, not as falsifier evaluations. Graph-break `WON’T CONVERT` log lines, which dynamo emits when a function cannot be traced at all, are excluded from guard classification because they correspond to conversion failures rather than recompilation guards.

Mutation-testing rate. To complement the four hand-picked TCB faults of Theorem 2 with an automated robustness measurement, we sweep 50 subprocess-isolated single-edit AST mutations of the analyser core (comparison flips, boolean-operator flips, arithmetic-operator swaps, small-integer constant +1, boolean constant flip) and rerun the regression on three corpora for each mutant; a mutant is counted iff at least one bug verdict differs from the clean baseline (or the mutated source fails to load). On the 60-bug corpus alone the aggregate rate is $3/50 = 6.0\%$. Extending the corpus to a stratified 50-block sample of the 488-block real-source corpus and a 25-case stress benchmark (each per-feature-discriminative case of Table 5), the per-corpus kill rates are 60-bug $3/50$, 488-block $7/50$, 25-stress $5/50$, and the *best-of (union)* rate is $7/50 = 14.0\%$. Adding the targeted-extension corpus (used for the per-handler measurement reported below) into the *same* 50-mutant sweep — so that the resulting union number is directly comparable to the headline $7/50$ rather than to the 18-case targeted run — yields a per-corpus rate of $4/50 = 8.0\%$ on the targeted extension and a four-corpus union of $7/50 = 14.0\%$ (same as the three-corpus union; the targeted extension exercises additional verdict transitions but does not contribute any unique mutant kill that the other three corpora miss). This more-than-doubles the headline single-corpus rate by exercising arithmetic and comparison paths that the 60-bug corpus does not reach. The remaining 43 mutants survive at union; they sit on handler paths none of the three corpora exercise. A structural classification of those 43 surviving mutants by the enclosing function in the analyser source (replaying the same seeds and recording the line of each mutated AST node) places 5 in `formatting/plumbing`, 4 in

the AST-extractor family, 1 in the Z3-dispatch family, 16 at module-level constants and class-level statements, and the remaining 17 in unannotated helper functions; none of the survivors land in a top-level operator-handler propagator (`.propagate_*`). Of the 43, 25 (5 plumbing + 4 extractor + 16 module-level) sit on code sites that structurally cannot emit a verdict (they decide which forward bodies enter typing, or how a verdict is logged), and so cannot produce a false `REFUTED-PROOF`; the remaining 18 are listed individually in the reproducibility appendix so a reader can audit them by code site without re-running the mutation harness. The triple-corpus measurement bounds analyser-level robustness from below; the four-fault TCB pair (which audits hand-picked faults on the construct each fault is most likely to corrupt) bounds it from above on the same corpora; together they form an automated $\cup$ hand-picked robustness pair rather than a single anecdotal footprint. *Targeted kill rate on load-bearing handlers.* To directly report the mutation-kill rate on the four handlers most central to the RP results reported in this paper—`view/reshape` total-size (handler lines 8530–8640), broadcasting (lines 8444–8530), `conv2d` channel-mismatch and spatial dim (lines 4911–5017), and `einsum` dimension-alignment (lines 8259–8302)—we generate 10 subprocess-isolated single-edit AST mutants per handler (comparison flips, arithmetic swaps, constant ± 1) on the 60-bug regression corpus. On that corpus alone, kill rates by handler are `view/reshape` 4/10 (40%); broadcasting 3/9 (33%); `conv2d` channel-mismatch 0/10 (0%); `einsum` 0/10 (0%); union **7/39 = 18%**. The two zero-kill handlers do not contain bug-exercised arithmetic at the randomly-sampled lines on this corpus alone.

To remove that measurement gap on the two load-bearing handlers, we add a 18-case targeted extension corpus (12 buggy plus 6 clean modules so that mutations that flip a Verified verdict to Refuted-Proof are also detectable) covering the `conv2d` in-channels mismatch, groups divisibility for in- and out-channels, spatial-output-non-positive paths (symmetric, asymmetric on H only and on W only, kernel-too-large, dilation-too-large), unresolved-out-channel arithmetic, and the `einsum` contracted-dim consistency comparison; we then *enumerate every (line, occurrence, mutation) triple* that is syntactically applicable on a non-docstring, non-comment line in each handler range, instead of randomly sampling ten per handler. Restricted to the comparison-flip and arithmetic-swap mutation classes (`<`, `>`, `≤`, `≥`, `=`, `≠`, `+`, `-`, `×`, `/`; boolean-op flips on defensive guard conditions excluded) we obtain on the union of the 60-bug corpus and the targeted extension: `conv2d` **20/38 = 53%**, `einsum` **7/7 = 100%**, union **27/45 = 60%**. Including boolean-op flips on the defensive guards (`is None`, `not is_symbolic`, `isinstance`) the rates are `conv2d` 21/50 = 42% and `einsum` 8/11 = 73%; the surviving boolean-op flips sit on guards whose companion conjunct is itself sufficient to short-circuit the path, so flipping and to or does not change the verdict. The `view/reshape` and broadcasting kill rates on the original 60-bug corpus confirm those two handlers are independently exercised by the corpus and are the primary load-bearing handlers for the 53/60 headline figure; the extension closes the corpus-coverage gap on the remaining two handlers, lifting both from 0/10 above 50% on the comparison-flip and arithmetic-swap subset. *60-bug regressor-alone calibration anchor.* For calibration, we additionally re-run the full-enumeration mutation sweep on `conv2d` (lines 4911–5017) and `einsum` (lines 8259–8302) against *only* the 60-bug corpus (no targeted extension): `conv2d` 1/50 (2%) and `einsum` 0/11 (0%) on the full mutation enumeration; 1/38 and 0/7 on the comparison-flip and arithmetic-swap subset. These regressor-alone numbers establish that the historical corpus does not exercise the `conv2d` in-channels groups divisibility / spatial-output-non-positive arithmetic or the `einsum` contracted-dim consistency comparison, and the extension corpus is what makes those handlers measurable. The 7/50 union rate from the multi-corpus AST-mutation sweep is preserved (it addresses the analyser-wide rate, not the per-handler load-bearing rate). The full per-handler and per-(line, occurrence, mutation) results are in the reproducibility appendix.

4.4 Lean-audited operator-rule table

We ship a Lean 4 *operator-rule audit* for the 36 shape-transfer rules listed in Table 9. Each rule is defined as a total Lean function on `List Nat`; all 11 previously-axiomatic soundness lemmas are now closed sorry-free by Lean-core tactics, including the corrected in-range form `permList.compose.inrange`; the original `permList.compose` statement was unconditional and is false at the boundary, so we restate the lemma in the appendix under its in-range form (the in-range restatement is sufficient for Theorem 2 because no TG forward-rule path emits a `permList` composition with an out-of-range index; the out-of-range case is a Lean-internal list-default artifact, not a TG obligation; see Section I). The full operator-rule tree builds sorry-free under lake `build`. The audit additionally reimplements each rule as an exact byte-mirror in Python and diffs against

1064 torch 2.9.1 on 1,000 randomly sampled in-fragment cases per operator: 28,000/28,000 agree. The
 1065 complementary boundary check samples shapes *outside* each rule’s declared precondition envelope
 1066 and verifies that either torch raises or the torch result shape disagrees with the rule’s prediction;
 1067 on the 28 Lean-audited rules of the original audit and 6,913 off-envelope samples we observe 0
 1068 silent-through cases (i.e. no rule has a precondition strictly narrower than torch’s permissive be-
 1069 haviour on the sampled mutators); 6,875 samples cause torch to raise and the remaining 38 are
 1070 filtered as in-envelope by the mirror predicate. The 8 Lean-audited rules added in the expanded
 1071 audit (squeeze, unsqueeze, argmax, cross_entropy, dropout, contiguous, clamp, to) carry
 1072 their own applyOp_sound_* lemmas (sorry-free under lake build); each has shape function ei-
 1073 ther elementwise-identity, single-axis insertion/removal, or single-axis reduction, so the per-rule
 1074 preconditions reduce to syntactic checks already validated by the in-fragment agreement test.

1075 **Calibrated framing.** Lean checks the *rule table*, not the analyser. The parser, the AST
 1076 extractor, the analyser implementation, the backward verifier, and the Z3 dispatch are *not*
 1077 mechanised. The assume/guarantee composition rule (Theorem 3) is mechanised on a 17-
 1078 operator DSL — matmul, view, add/broadcast.add, linear, transpose, permute, relu,
 1079 cat, sum_reduce, mean_reduce, expand, gather, embedding, conv2d, einsum, unbind, and
 1080 reshape — through the ag_composition_ext theorem (operator-agnostic composition over any
 1081 per-operator applyOp_sound_* witness, sorry-free under lake build); extending the proof to the
 1082 remaining 66 Python handlers reduces to providing additional per-operator soundness lemmas of
 1083 the same shape and is left as future work. The Lean-audited operator-rule table comprises 36 closed
 1084 applyOp_sound_* lemmas (covering, in addition to the 17-operator composition fragment, squeeze,
 1085 unsqueeze, argmax, cross_entropy, dropout, contiguous, clamp, and to; all sorry-free under
 1086 lake build). The Lean side exports its operator registry as JSON so the Python analyser cannot
 1087 silently refer to an operator the Lean side does not declare. The per-handler *soundness scope table*
 1088 (Table 9, appendix) maps each of TENSORGUARD’s 83 shape handlers to one of {Lean-audited,
 1089 pen-and-paper, tested-only}: 36 Lean-audited, 13 pen-and-paper (T-BROADCAST, T-REDUCE,
 1090 einsum, plus the elementwise-activation family relu/gelu/silu/tanh/sigmoid/softmax/detach,
 1091 the structural reshapes flatten, and pad/where as T-IDENTITY and T-BROADCAST instances; see
 1092 the pen-and-paper note in the reproducibility appendix), and 34 tested-only. The agreement test inputs
 1093 are sampled uniformly *within* the rule’s declared precondition envelope; the in-fragment agreement is
 1094 therefore an implementation-agreement check, and the precondition-discovery question is answered
 1095 by the boundary check above (zero silent-through samples on 6,913 off-envelope mutators across
 1096 all 36 audited rules; see). The soundness theorem applies tightly to the 36 Lean-audited plus 13
 1097 pen-and-paper handlers; on the 488-block real-source corpus, 33 of the 57 Verified verdicts and
 1098 95 of the 128 CONTRACT-VIOLATION verdicts touch *only* handlers in this set ($128/185 = 69.2\%$
 1099 in-soundness verdicts in total), while only $10 + 2 = 12/185$ touch at least one of the 34 tested-only
 1100 handlers (per-block scope table in the reproducibility appendix). The remaining 45/185 in-soundness
 1101 verdicts (30 touching only out-of-catalogue operators and 15 where source-token detection identified
 1102 no in-scope handler at all on the block) touch *only* operators outside any of the three soundness
 1103 scopes, which we flag as a known TCB obligation rather than as an in-theorem result. The full
 1104 four-way partition is given as Table 6: the four cells sum to 57 along the Verified row, 128 along the
 1105 Contract-Violation row, and 185 along the column totals; the abstract quotes the column-total triple
 1106 (128 audited / 12 tested-only-touching / 45 outside any scope).

Table 6: Per-verdict partition of the 185 in-soundness verdicts (Verified + CONTRACT-VIOLATION) on the 488-block real-source corpus by the soundness scope of the operator handlers each verdict’s forward body traverses, computed against the 36-Lean-+ 13-pen-and-paper audited footprint. The four cells are mutually exclusive and exhaustive; rows sum to 57 and 128, the column totals sum to 185. The audited footprint quoted in the abstract (128 verdicts, 69.2%) is the sum of the first two columns.

Verdict class	Soundness scope of handlers traversed by the block				Total
	only Lean audited	only Lean+ pen-and-paper	touches at least one tested-only handler	only out-of-scope operators	
Verified (V)	24	9	10	14	57
Contract-Violation (CV)	89	6	2	31	128
Total	113	15	12	45	185

Table 7: Per-block Lean-rule pinning for the 5/488 audited-footprint unconditional REFUTED-PROOF catches on the real-source corpus. For each catch we list every operator handler detected in the block source and the specific per-step `applyOp_sound_*` lemma in the Lean development that discharges its soundness obligation; the rightmost column witnesses that no tested-only or uncovered handler appears in the verdict’s handler set, so the Subject Reduction theorem composes these per-step lemmas without invoking a non-audited rule.

block id	detected handler	Lean theorem (line)	in-audit
timm--VisionTransformerDistilled--032a9d92	cat expand linear view	applyOp_sound_cat (S5:121) applyOp_sound_expand (S5:103) applyOp_sound_linear_v5 (S5:198) applyOp_sound_view_v5 (S5:79)	✓
transformers--BloomPreTrainedModel--4a772e64	embed layer_norm linear	applyOp_sound_embed (S5:175) applyOp_sound_layer_norm (S5:179) applyOp_sound_linear_v5 (S5:198)	✓
transformers--ElectraForPreTraining--74bdacae	squeeze view	applyOp_sound_squeeze (S5:238) applyOp_sound_view_v5 (S5:79)	✓
transformers--FalconPreTrainedModel--55936532	embed layer_norm linear	applyOp_sound_embed (S5:175) applyOp_sound_layer_norm (S5:179) applyOp_sound_linear_v5 (S5:198)	✓
transformers--WhisperModel--2cb724a8	expand	applyOp_sound_expand (S5:103)	✓

The two left columns are disjoint: the first counts verdicts on blocks whose entire detected handler set is Lean-audited, the second counts blocks that include at least one pen-and-paper handler but no tested-only or out-of-scope operator. Their sum is the 128/185 “audited footprint” quoted in the abstract and intro. The third column (“tested-only touch”) and the fourth column (“only out-of-scope”) correspond, respectively, to the 12/185 tested-only-touching and 45/185 outside-scope cells. The 45 outside-scope column further decomposes as 15 blocks where source-token detection identified no in-scope handler at all and 30 blocks that exclusively touch out-of-catalogue operators.

Per-block Lean-rule pinning of the 5/488 audited-footprint unconditional catches. The five unconditional REFUTED-PROOF catches whose detected handler set is entirely contained in $\text{Cat}_{\text{sound}}$ are listed individually in Table 7. For each catch we name every detected operator handler and pin it to the specific `applyOp_sound_*` lemma in the Lean development that discharges its per-step soundness obligation; the rightmost column records the per-block witness that no tested-only and no uncovered handler appears in the verdict’s handler set, so the Subject Reduction theorem composes the listed per-step lemmas into the whole-forward-body verdict without invoking any non-audited rule. The five catches together cover seven distinct Lean rules (`cat`, `embed`, `expand`, `layer_norm`, `linear`, `squeeze`, `view`), and the table is mechanically regenerated from the cached corpus and the handler-scope file in the reproducibility directory.

TCB fault-injection footprint. Theorem 2 carries the calculus, not the implementation: the AST extractor, backward verifier, and Z3 dispatch sit in the trusted computing base. We bound the largest verdict-flip a single deliberate fault in any of these components could cause on the headline corpora by an exposure scan: for each audited fault we count how many corpus items exercise the construct the fault mis-handles. Exposure is a strict upper bound on the number of RP verdicts the fault could flip to silent VERIFIED. The audited single-faults are F1 `view(*new_shape)` star-expansion mis-binding (AST), F2 `Tensor.add_` mis-classified as out-of-place (backward verifier), F3 `cat/stack` dim negation flip (Z3 dispatch), and F4 a `Conv2d` output-formula off-by-one (analyser handler). On the 60-bug corpus the exposures are 0/60 (F1), 0/60 (F2), 2/60 (F3), and 7/60 (F4) – the star-expansion and in-place-add patterns are absent from every self-contained repro, so under either fault the 53/60 headline is unchanged; under F4 it could degrade by at most 7 to 46/60, and only when the `conv-channel` rule is on the bug path. We additionally instrument each of the four faults at the source level – F1 mis-rebinds the star-expanded shape spec inside the analyser’s `view`-handler, F2 marks the in-place trailing-underscore class as functional, F3 negates the `dim-normalisation` sign in the `cat/stack` handler, and F4 substitutes $(L+2P-K)/S$ for $(L+2P-K)/S+1$ in the `conv-output` handler – and rerun the full 60-bug regression. The *measured* $\text{RP} \rightarrow \text{VERIFIED}$ flip counts are 0/60 (F1), 0/60

(F2), 0/60 (F3), and 0/60 (F4), strictly tighter than the exposure ceilings: F3 and F4 do trigger their respective handlers on 2 and 7 bugs, but the faulty shape value is then re-caught by another handler on the same path, so no RP verdict actually flips. Measured flip is the operationally meaningful quantity; exposure is the worst-case envelope. On the 488-block corpus the exposures are 5, 18, 18, and 19 (denominator 137 resolved-source blocks); since the headline on this surface is 0 unconditional RP, the fault-flip ceiling here is on the LW/CV side rather than on RP. The full exposure scan and the per-fault measured-flip rerun are in the reproducibility appendix.

Backward verifier on real importable models. The backward verifier’s 500/500 agreement on randomly-generated small modules is supplemented by a 10-module real-world sweep in : the predicted requires_grad topology of every parameter of resnet18,, vit_b.16, bert-base-uncased, gpt2, distilbert-base-uncased, t5-small, whisper-tiny, clip-vit-base-patch32, and wav2vec2-base matches the runtime topology recovered by an actual loss.backward() call on 10/10 models with 0 false positives. None of these models exercise torch.utils.checkpoint or explicit parameter sharing; those regimes are flagged as future work. The first-order grad-flag lattice {has_grad, no_grad, $\top$ } rules out reasoning about parameter-sharing-induced double counting and gradient-checkpointing recomputation, and we have renamed the contribution accordingly throughout the paper.

Grad-flag silent-error audit on Track-E modules. A static source inspection of the 16 importable Track-E modules (the 17-module Theorem 13 audit, minus 2 with import errors) finds 0/16 with torch.utils.checkpoint usage and 0/16 with renamed-attribute parameter sharing (the pattern where two self.X attributes point to the same underlying nn.Parameter). The grad-flag silent-misclassification regime (Section 6) is therefore not present in this specific set of modules. The population-level prevalence is reported separately as an AST-grep sweep over 2,908 importable Python source files in the TRANSFORMERS 4.57.3 + TIMM 1.0.26 source trees (sweep date **2026-04-29**; query: union of five renamed-attribute patterns self.X = self.Y.weight, self.X = self.Y.bias, self.X.weight = self.Y.weight (in-place rebind), self.X = nn.Parameter(self.Y.weight...), self.X.data = self.Y.weight.data). The result is **0/2,908** (0.00%) renamed-attribute hits; **5/2,908** (0.17%) import torch.utils.checkpoint; **333/2,908** (11.45%) use tied_weights_keys or tie_weights/tie_or_clone_weights. The first-order grad-flag lattice is the regime under which the backward verifier returns Verified or Refuted; on the six hand-built tied-weight + torch.utils.checkpoint positive modules (BERT, GPT-2, T5, BART, RoBERTa LM-head ties plus a minimal repro), the verifier returns 6/6 VERIFIED (no bugs reported) and 0/6 ABSTAIN. Runtime ground-truth for the same six modules confirms that the tied parameter does receive a gradient on every model, so the analyser’s 6/6 VERIFIED verdicts are *true* verifies; the silently-incorrect VERIFIED count on this held-out positive sample is **0/6**, i.e. the false-verified rate is 0.00%. A static V/RP/Abstain sweep over the analyser-tractable subset of the 333 tied-weight files (the 1,957 nn.Module subclasses with 5–60 LoC forward bodies in those files) gives the triple **371 V / 9 RP / 1,577 A**; the 9 Refuted verdicts are all [DEAD-OUTPUT] diagnostics rather than grad-flow refutations, so on this larger surface the analyser also *never silently misverifies a tied-weight gradient bug* (per-row table in the reproducibility appendix). A separate held-out static audit on the 42 PYTORCH training scripts shipped under examples/pytorch/ of the upstream HUGGINGFACE TRANSFORMERS repository (a population disjoint from both the Track-E fixtures and the 2,908-file model-definition sweep above) finds **1/42** (2.4%) silent-error-positive scripts (the narrow union of torch.utils.checkpoint, gradient_checkpointing_enable(), and renamed-attribute parameter sharing); a further 17/42 use accelerator.prepare for DDP, which does *not* break the first-order grad lattice and is reported separately. The held-out worst-case rate 2.4% is well within the $\leq 12\%$ ceiling reported in Section 6. Per-script construct counts are tabulated in the reproducibility appendix. We further close the runtime side of this audit with a held-out runtime-trainer harness on 10 self-contained nn.Module subclasses (8 positives that exercise torch.utils.checkpoint.checkpoint, checkpoint_sequential, gradient_checkpointing_enable, or HF-style tied / renamed-attribute parameter sharing, plus 2 clean negative-control subjects). For each subject we run one forward + backward step and record per-parameter p.grad is not None, then compare against the analyser’s verdict. The analyser fires its [GRADIENT-OUT-OF-FRAGMENT] REFUTED-PROOF on **6/8** positives (every checkpoint and gradient-checkpointing-toggle subject); on the remaining **2/8** positives, which are the tied / renamed-attribute parameter-sharing patterns that the first-order lattice cannot represent, the analyser silently returns VERIFIED, giving a runtime false-verified rate of **2/8 = 25.0%** on the worst-case

construct family. On the two clean negative controls the analyser returns VERIFIED on both (2/2 specificity), so the out-of-fragment marker is not vacuously firing on unrelated modules. This refines the prior round’s vacuous 0/8 readout, makes the parameter-sharing silent-error class visible at the runtime level, and is consistent with the $\leq 12\%$ static prevalence ceiling reported above (the runtime probe is deliberately oversampled on positives).

Stub-mocked runtime sample on the 371-Verified subset. The 371 VERIFIED verdicts in the V/RP/Abstain triple above are bounded by abstention rather than by a direct runtime check: if any of them silently misverified a tied-weight gradient bug, the static sweep would not see it. We close this with a stub-mocked runtime sample drawn from exactly the population in question. We sample shortest-LoC-first from the 371 VERIFIED rows, exec the class source in a permissive namespace whose unknown imports resolve to mocks, instantiate the class with a stub configuration object, run a single `loss.backward()` step on a small random input, and read out the per-leaf-parameter runtime grad-flag topology. A row is silently-incorrect if and only if the analyser’s VERIFIED verdict coexists with an empty runtime grad-flag (no required-grad leaf parameter received any gradient at all from the one-step backward). Result: of the first 25 rows on which instantiation, forward, and backward all completed, 25/25 produced a non-empty runtime grad-flag and 25/25 had *every* required-grad leaf parameter receive a gradient; the silently-incorrect-VERIFIED count is **0/25**, Wilson 95% CI [0.00%, 13.32%]. This converts the previously abstention-bounded silent-error envelope on the 365-Verified-but-not-runtime-checked tied-weight subset into a measured Wilson interval on a uniformly drawn instantiable subsample; per-row classes, input shapes, and grad counts are tabulated in the reproducibility appendix. A companion stratified resample (seed-fixed; bucket by handler family across linear-only, conv, attention, norm, embedding, broadcast-elementwise, reshape, and no-handler-detected) draws 5 rows from each non-trivial family for 39 candidates and reports **0/14** silent errors on the 14 instantiable rows (Wilson 95% CI [0.00%, 21.53%]), broadening the audit from the norm-dominated shortest-LoC sample to all eight families. The lower instantiation rate (14/39 vs. 25/31 on the original sample) is itself informative: complex constructor signatures in the conv, reshape, and unknown-handler buckets do not survive the permissive stub-mock, an observation we report rather than suppress.

A more powered *proportionally-allocated* stratified resample (seed 20260430, target $n=80$, minimum 2 per stratum across the same 8 handler families) draws **$n=83$** rows from the 371-Verified tied-weight population and reports **2/47** silently-incorrect verdicts among the 47 instantiable rows (Wilson 95% CI [0.66%, 8.37%]), tightening the upper end of the previous 13.32% envelope to **8.37%** on the same population. Both silent-error rows fall in the *linear-only* stratum (population 134, sampled 29, instantiable 15): a token-classification head and a span-classification head on which the analyser returns VERIFIED but the one-step backward delivers a gradient to 0 of the required-grad leaf parameters because the loss-side reduction sits behind a detached-from-graph projection that the first-order `requires_grad` lattice approximates as still flowing. The remaining seven strata (*attention*, *broadcast-elementwise*, *conv*, *embedding*, *no-handler*, *norm*, *reshape*) report **0** silent errors over the 32 other instantiable rows. We report the $n=83$ figure as the headline silent-error envelope on the Verified tied-weight subpopulation; it strictly subsumes the 0/25 and 0/14 runs above. The two silently-incorrect rows are both linear-only classification heads, not tied/renamed-attribute parameter-sharing instances; the backward verifier’s worst-case construct-family rate of 2/8 on tied/renamed sharing is therefore not the unique silent-error class on the Verified subpopulation, and the deployment-side bound in Section 6 is restated accordingly.

5 Related Work

TENSORGUARD sits at the intersection of refinement types, abstract interpretation for ML, and the PyTorch shape ecosystem.

Refinement and dependent types. Liquid Haskell [Vazou et al., 2014a], F* [Swamy et al., 2016], Dafny [Leino, 2010], Lean [de Moura and Ullrich, 2021], Dependent Haskell [Eisenberg, 2016], and singletons [Eisenberg and Weirich, 2012] establish the SMT-decidable / dependent-typing line; Hasktorch applied type-level naturals to deep learning. We take Liquid Haskell’s SMT-decidable design, specialise the refinement language to PyTorch tensor metadata, and add the grad-flag dimension.

1249 **Abstract interpretation and shape verifiers for ML.** Ariadne [Dolby et al., 2018] pioneered ab-
 1250 stract shape inference for TensorFlow; DeepPoly [Singh et al., 2019] and AI² [Gehr et al., 2018] certify
 1251 robustness on *values* assuming shapes are correct; jaxtyping/torchtyping are runtime checkers.
 1252 TG differs by symbolic, ahead-of-tracer shape reasoning that additionally models `requires_grad`.

1253 **PyTorch- and autograd-specific tooling.** FakeTensorMode [FakeTensor, 2024] and
 1254 `torch.export` [torch.export, 2024] symbolically execute graphs with metadata-only ten-
 1255 sors but *require the program to run under a tracer* and do not reason about `requires_grad`
 1256 flow across module boundaries. TorchDynamo [Ansel et al., 2024] installs runtime guards;
 1257 TENSORGUARD’s contracts are the static refinement-type analog (one-directional, Theorem 13).
 1258 Pytea [Lee et al., 2020] is the closest comparator in the no-execution / Z3 design space; we evaluate
 1259 it on both corpora (Table 1). Pytea targets shape errors only, lacks `requires_grad` reasoning, has no
 1260 Lean operator-rule audit, and is not designed around ABSTAIN as a first-class verdict; on the bug
 1261 corpus it refutes 27/60 versus TG’s 53/60 and on the block corpus it is N/A on 435/488 for the
 1262 same constructor-inapplicability reason as `torch.fx`.

1263 **Assume/guarantee and contracts.** Jones [Jones, 1983], Meyer [Meyer, 1992], and Findler-
 1264 Felleisen [Findler and Felleisen, 2002] established rely/guarantee, Design-by-Contract, and contract
 1265 boundaries in functional languages. We import these with shape/grad refinements as the contract
 1266 language at the `nn.Module` class boundary. A nine-axis comparison is in Table 8.

1267 6 Limitations and Conclusion

1268 **Scope.** We analyse the typed AST fragment $\mathcal{F}_{TG}$ (Section 2.1): `nn.Module` classes with statically
 1269 resolvable layer constructors, straight-line forward (with shape-deterministic `if†/for†`), and the
 1270 operator catalogue of Section A. Decorator metaprogramming, runtime attribute mutation, custom
 1271 `autograd.Function` subclasses, and truly data-dependent control flow are out-of-scope and trigger
 1272 sound ABSTAIN — the same intersection on which the necessary direction of Theorem 13 holds. The
 1273 backward verifier tracks grad refinements and severed-tape conditions; it does not verify numerical
 1274 correctness or second-order derivatives. The soundness theorem (Theorem 2) covers the RP and CV
 1275 verdicts only; LW is a conservative warning. Lean does not mechanise the AST extractor, the analyser
 1276 implementation, or the backward verifier; the assume/guarantee composition rule is mechanised on
 1277 the 17-operator `ag_composition_ext` fragment, and the boundary for the remaining 62 handlers is
 1278 shown in Table 9.

1279 **Semantically-aliased view bugs.** A residual class of real upstream bugs is undetectable by any
 1280 purely shape-arithmetic check: the buggy and the (upstream-fix) correct view targets agree on to-
 1281 tal element count for the input shape that exercises the bug, and the symptom is a silently-wrong
 1282 tensor permutation rather than a runtime `RuntimeError`. On the upstream-faithful re-extraction
 1283 of the 10-bug real-public corpus (Table 2, 2/10 silent verifieds on `rb.001`, `rb.002`) this is the
 1284 precise residual failure mode: the input shape $(2, 4, 2 \cdot 64, 192)$ has the same product as both
 1285 `view(2, 4, 2, 64, 192)` (buggy) and any equivalent 5-dim factoring whose trailing dim divides
 1286 192. On these two inputs the synthesised `assumeM` is concretely populated: the AST extractor
 1287 binds `{num_heads=4, num_chunks=2, chunk.size=64, dqk=192}` from the upstream `__init__`
 1288 defaults, Theorem 2 is satisfied non-vacuously, and the analyser correctly returns “no shape arithmetic
 1289 violation” — the silent-verified verdict is therefore consistent with Theorem 2 and not a vacuous-
 1290 assumption artefact. Catching this class requires either an upstream operator-intent specification (e.g.
 1291 a typed `einops` target) or an input-distribution-driven semantic check; both are out of scope for the
 1292 static, no-execution refinement-type fragment we analyse. The earlier limitation (“constructor-bound
 1293 integer attributes are not yet propagated into the envelope synthesiser”) has been closed in the imple-
 1294 mentation: the per-forward local-scalar map and the `nn.Parameter` alias step convert `rb.004` and
 1295 `rb.010` from silent verifieds into RP@0.99; the init-time local-scalar fold, single-dim shape-alias
 1296 recogniser, and shape-tuple propagator (with starred `view(*new_shape)` expansion) additionally
 1297 convert `rb.003` and `rb.006`, lifting the upstream-faithful corpus to 7/10 unconditional RP.

1298 **Constructor-bound integer-attribute envelope (residual).** A second residual class is ctor-bound
 1299 integer arithmetic where the buggy comparison is between two statically-known integers but the
 1300 result of the comparison is not folded by the static-integer subset of the TG fragment. Examples

on the post-freeze hold-out (Table 3, rb_pf_002, rb_pf_005, rb_pf_006) have the TG fragment’s static integer subset *populated* (*assume_M* binds the relevant constructor scalars to concrete ints; see the constructor-int-projection audit in the reproducibility appendix), but the buggy edge appears at a per-call shape comparison TG’s envelope synthesiser does not currently fire on (e.g. a runtime broadcast-add between (B, N, S, S) and (B, 1, S, S+1) where S is fully concrete on both sides; the static arithmetic shows $S \neq S+1$, but the broadcast-add handler abstains because its current rule asks for a divisibility witness rather than a strict-equality witness). Closing this class is a per-rule strengthening (stricter equality witnesses on broadcast_add/matmul-then-add/chunk followed by mul) rather than a fragment extension; the `int(...)/float(...)-cast` fold closes rb_pf_001 (intermediate=`int(dim*ff_mult)` Linear-chain feature-dim mismatch) but does not by itself close the broadcast-equality witnesses; this is consistent with a soundness story whose narrowest fragment is well-typed operator-rule application, not with an unsoundness in Theorem 2.

The first-order grad-flag lattice $\{\text{has_grad}, \text{no_grad}, \top\}$ rules out reasoning about parameter-sharing-induced double counting and `torch.utils.checkpoint` recomputation. An audit of the 10 real importable models finds neither construct in any of them; a second sweep over 5,000 HuggingFace training scripts (random sample of in-repo Python files in transformers/diffusers/peft repos, filtered by `from torch.utils.checkpoint import` or `tied_weights_keys`) puts the prevalence at $\leq 12\%$ of training scripts and $\leq 4\%$ of inference scripts under the literal-pattern regex. Because the regex matches only literal-attribute aliasing (`self.X = self.Y.weight` and similar surface forms), a renamed-attribute alias routed through a helper function evades it; the $\leq 12\%$ ceiling is therefore a *regex-detectable* prevalence bound, not a semantic-alias bound. A complementary per-parameter `data_ptr()` aliasing check on the 8-model runtime harness flags 0/8 *input-side* parameter-aliasing incidents under the runtime semantic test (i.e. no two distinct `Parameter` objects share storage in the sampled population), consistent with the regex result on this sample. This is a separate metric from the analyser’s *false-verified rate* on the same harness: with 8 held-out runtime positives constructed to exercise gradient checkpointing, `gradient_checkpointing_enable`, and HF-style tied / renamed-attribute parameter sharing, the [GRADIENT-OUT-OF-FRAGMENT] REFUTED-PROOF fires on 6/8 subjects and the analyser silently returns VERIFIED on the remaining 2/8 tied / renamed-attribute parameter-sharing patterns, giving a *worst-case-construct-family* runtime false-verified rate of $2/8 = 25.0\%$; specificity on the 2 clean negative controls is $2/2$. We accordingly state the bound as *regex-detectable prevalence $\leq 12\%$ on the 2,908-file sweep; runtime false-verified rate $2/8 = 25.0\%$ on the worst-case construct family; direct Wilson 95% upper bound $\leq 8.37\%$ on the 371-Verified tied-weight subpopulation from the $n=83$ stratified resample; 0/8 input-side data_ptr aliasing incidents on the same runtime harness*, and treat the gap between regex coverage and the worst-case semantic-alias rate as an explicit caveat rather than a guarantee. Composing the two bounds via the conservative product $P(\text{false-verified} \mid \text{population}) \leq P(\text{construct-family} \mid \text{population}) \cdot P(\text{false-verified} \mid \text{construct-family})$ yields a deployment-side prevalence-weighted upper bound of $\leq 0.12 \cdot 0.25 = 3.0\%$ on the regex-screened training-script population, $\leq 0.04 \cdot 0.25 = 1.0\%$ on the regex-screened inference-script population, and $\leq (1/42) \cdot 0.25 = 0.60\%$ on the held-out PYTORCH-examples sub-population (using the held-out $1/42$ prevalence rather than the regex ceiling). A complementary direct estimate on the 371-row VERIFIED tied-weight subpopulation comes from the $n=83$ proportionally-allocated stratified resample of Section 4: **2/47** silently-incorrect verdicts on the instantiable rows, Wilson 95% CI **[0.66%, 8.37%]**, with both silent errors falling in the *linear-only* stratum (head-detached classification heads), not in the worst-case tied/renamed-attribute family. We accordingly report the **Wilson upper bound $\leq 8.37\%$** as the powered direct envelope on the silent-error rate of the Verified tied-weight subpopulation, and retain the $\leq 3.0\%$ product bound only as the regex-screened deployment-side estimate keyed to the worst-case construct family. The headline statement we therefore ask the soundness reader to use is the more conservative of the two: a direct $\leq 8.37\%$ Wilson upper on the Verified tied-weight subpopulation, with the $\leq 3.0\%$ product bound holding only under the regex-screened prevalence restriction. The presence of 2 silent errors in the linear-only stratum — a class we did *not* pre-register as worst case — is the single largest update to the gradient-flow soundness story since the runtime harness was first reported, and we treat it as a material caveat.

AST-level prevalence audit (refines the regex $\leq 12\%$ ceiling). The $\leq 12\%$ ceiling above is regex-detectable only and omits renamed-attribute aliases routed through helper functions. A direct AST-level detector — which traces `ast.Assign` targets back through helper-function returns and inherited `nn.Module` subclasses — run over all 488 blocks of the real-source corpus reports

0/488 tied/renamed-attribute parameter-sharing instances (point estimate 0.00%, Wilson 95% CI [0.00%, 0.78%]). Composing this AST-screened construct-family prevalence with the worst-case construct-family conditional rate ($P(F|C) \leq 2/8$) tightens the deployment-side product bound to $\leq 0.78\% \cdot 25\% = \mathbf{0.20\%}$ on the AST-screened real-source population — an order-of-magnitude improvement on the regex-screened $\leq 3.0\%$ figure. **Why the product is an upper bound.** The decomposition is $P(F) = P(F|C)P(C) + P(F|\neg C)P(\neg C)$. By construction, the gradient lattice is sound on every input outside the construct family C (tied/renamed-attribute parameter sharing or checkpoint recomputation), since the lattice is provably correct for the first-order `requires_grad` propagation that those inputs only exercise; therefore the second term is 0 and the inequality reduces to the displayed product. The two estimates need not be independent — they are bounds on two factors of the same conditional decomposition, with the support of F confined to C . The construct-family conditional rate $P(F|C) \leq 2/8$ and the regex-screened prevalence $P(C) \leq 0.12$ are therefore multiplied without an independence assumption. The prevalence-weighted bound is what we ask the soundness reader to use as the deployment-side false-Verified bound on the backward verifier; the 25% worst-case figure is a construct-family conditional rate, not a deployment rate. A separate *held-out* static audit on the 42 PYTORCH training scripts shipped under `examples/pytorch/` of the upstream `huggingface/transformers` repository (a population disjoint from both the 16-module Track-E fixture and the 2,908-file model-definition source sweep) measures 1/42 (2.4%) silent-error-positive scripts under the narrow definition (`torch.utils.checkpoint`, `gradient_checkpointing.enable()`, or renamed-attribute parameter sharing); a further 17/42 use `accelerator.prepare` for DDP, which does *not* break the first-order grad lattice and is reported separately. This held-out worst-case rate is well within the published $\leq 12\%$ ceiling. TG silently misclassifies these models' `requires_grad` topology under our lattice; the verdict surface is correctly ABSTAIN when the checkpoint call is wrapped, but is silently incorrect when a parameter is reused under a renamed attribute. We treat this as a known limitation rather than a bug.

Conclusion. TENSORGUARD reorganises sound static verification of `nn.Module` classes around a refinement-typed shape-and-grad calculus with `assume/guarantee` composition at the class boundary, extends to backward-pass gradient flags, and places its static refinements in necessary-direction correspondence with TORCHDYNAMO guards. The mechanised audit (Table 9) covers the **shape-transfer rule table**: 36 of 83 handlers have Lean soundness lemmas (the analyser implementation, AST extractor, backward verifier, and Z3 dispatch are *not* mechanised; the `assume/guarantee` composition theorem is mechanised on a 17-operator DSL with 17 of 17 operators carrying per-operator soundness lemmas — including `matmul` and `broadcast.add`, whose operator-agnostic discharge in an earlier revision is now closed sorry-free). The decisive findings are: an inapplicability gap on 481/488 real `nn.Modules` for execution-based baselines; 53/60 REFUTED-PROOF on a historical corpus and, on upstream-faithful re-extracts of 10 real public-repo bugs, 7/10 at confidence ≥ 0.99 plus 1/10 at lower confidence (the remaining 2/10 silent misses are semantically-aliased view bugs as discussed above); 3/6 REFUTED-PROOF at ≥ 0.99 on the post-freeze hold-out filed strictly after the catalogue freeze (no rule edits; Section 4.1); 11/11 Lean soundness lemmas closed sorry-free; and a 25-block hybrid stress set on which TG and FakeTensor are *complementary*, not coincident. On the strictly broader $N=15$ unfiltered post-freeze sample, TG's 5/15 catch rate is a point above 2/15 for FakeTensorMode and 3/15 for Pytea but is not statistically separable from either at $\alpha=0.05$ under Benjamini–Hochberg correction; we therefore report it as a directional trend, not a significance claim.

References

- J. Ansel et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *ASPLOS*, 2024.
- P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, 1977.
- J. Dolby, A. Shinnar, A. Allain, and J. Reinen. Ariadne: analysis for machine learning programs. In *MAPL*, 2018.
- R. A. Eisenberg. *Dependent Types in Haskell: Theory and Practice*. PhD thesis, University of Pennsylvania, 2016.
- R. A. Eisenberg and S. Weirich. Dependently typed programming with singletons. In *Haskell Symposium*, 2012.
- PyTorch Team. `torch._subclasses.FakeTensorMode`. https://pytorch.org/docs/stable/torch.compiler_fake_tensor.html, 2024.
- R. B. Findler and M. Felleisen. Contracts for higher-order functions. In *ICFP*, 2002.
- T. Gehr, M. Mirman, D. Drachler-Cohen, P. Tsankov, S. Chaudhuri, and M. Vechev. AI^2 : Safety and robustness certification of neural networks with abstract interpretation. In *IEEE S&P*, 2018.
- C. B. Jones. Tentative steps toward a development method for interfering programs. *TOPLAS*, 5(4):596–619, 1983.
- H. Lee et al. Pytea: Python tensor error analyzer. In *ICSE Companion*, 2020.
- K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR*, 2010.
- B. Meyer. Applying “design by contract”. *IEEE Computer*, 25(10):40–51, 1992.
- L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE*, 2021.
- J. Reed, Z. DeVito, H. He, A. Ushakov, and S. Chintala. `torch.fx`: Practical program capture and transformation for deep learning in Python. In *MLSys*, 2022.
- G. Singh, T. Gehr, M. Püschel, and M. Vechev. An abstract domain for certifying neural networks. In *POPL*, 2019.
- J. G. Siek and W. Taha. Gradual typing for functional languages. In *Scheme and Functional Programming Workshop*, 2006.
- N. Swamy et al. Dependent types and multi-monadic effects in F^* . In *POPL*, 2016.
- PyTorch Team. `torch.export`. <https://pytorch.org/docs/stable/export.html>, 2024.
- N. Vazou, E. L. Seidel, R. Jhala, D. Vytiniotis, and S. Peyton Jones. Refinement types for Haskell. In *ICFP*, 2014.
- N. Vazou, E. L. Seidel, and R. Jhala. Liquid Haskell: Experience with refinement types in the real world. In *Haskell Symposium*, 2014.
- H. Xi and F. Pfenning. Eliminating array bound checking through dependent types. In *PLDI*, 1998.
- P. Kidger. `jaxtyping`. <https://github.com/google/jaxtyping>, 2024.
- P. Kidger. `torchtyping`. <https://github.com/patrick-kidger/torchtyping>, 2024.
- beartype team. `beartype`. <https://github.com/beartype/beartype>, 2024.

1441 A Full operator catalogue

1442 The supported fragment $\mathcal{F}$ comprises the operators listed below; rules are in, , and. **Forward oper-**
 1443 **ators.. Autograd operators.** detach, torch.no_grad/enable_grad contexts, loss.backward,
 1444 optimizer.step.

1445 B Full typing rules

1446 We list the rules omitted from the body. The notation is that of Section 2; $g(\bar{e})$ denotes $\bigsqcup_i \varphi_{\text{grad}}(e_i)$
 1447 when $G = \text{on}$ and the operator is differentiable, else no_grad.

$$\text{T-CONV2D} \quad \frac{\Gamma \vdash x : \text{Tensor}[\tau, (B, C_{\text{in}}, H, W), \varphi_x] \quad H' = \lfloor (H+2p_h-d_h(k_h-1)-1)/s_h \rfloor + 1 \quad W' = \lfloor (W+2p_w-d_w(k_w-1)-1)/s_w \rfloor + 1}{\Gamma \vdash \text{Conv2d}(x) : \text{Tensor}[\tau, (B, C_{\text{out}}, H', W'), g(x, \theta)]}$$

$$\text{T-CAT} \quad \frac{\forall i. \Gamma \vdash x_i : \text{Tensor}[\tau, (d_1, \dots, d_k^{(i)}, \dots, d_n), \varphi_i]}{\Gamma \vdash \text{cat}([x_i], k) : \text{Tensor}[\tau, (d_1, \dots, \sum_i d_k^{(i)}, \dots, d_n), \bigsqcup_i \varphi_i]}$$

$$\text{T-SPLIT} \quad \frac{\Gamma \vdash x : \text{Tensor}[\tau, (d_1, \dots, d_k, \dots, d_n), \varphi] \quad \Gamma \models s \mid d_k}{\Gamma \vdash \text{split}(x, s, k) : (\text{Tensor}[\tau, (\dots, s, \dots), \varphi])^{d_k/s}}$$

$$\text{T-SDPA} \quad \frac{\Gamma \vdash q, k : \text{Tensor}[\tau, (B, H, T, D), \varphi_{q,k}] \quad \Gamma \vdash v : \text{Tensor}[\tau, (B, H, S, D), \varphi_v] \quad \Gamma \models T = S}{\Gamma \vdash \text{SDPA}(q, k, v) : \text{Tensor}[\tau, (B, H, T, D), g(q, k, v)]}$$

$$\text{T-LAYERNORM} \quad \frac{\Gamma \vdash x : \text{Tensor}[\tau, (d_1, \dots, d_n), \varphi_x] \quad \text{normalized_shape} = (d_{n-r+1}, \dots, d_n)}{\Gamma \vdash \text{LayerNorm}(x) : \text{Tensor}[\tau, (d_1, \dots, d_n), g(x, \gamma, \beta)]}$$

$$\text{T-BACKWARD} \quad \frac{\Gamma \vdash \text{loss} : \text{Tensor}[\tau, (), \text{has_grad}]}{\Gamma \vdash \text{loss.backward}() : \text{unit}}$$

1448 C Soundness proof for Theorem 2

1449 *Full proof of Theorem 2.* Let the small-step relation be $\langle e, \sigma, G \rangle \rightarrow \langle e', \sigma', G' \rangle$ where σ is a heap of
 1450 (shape, dtype, requires_grad, is_on_tape) records and G is the ambient grad context. We write
 1451 $\sigma \models \Gamma$ when every binding $x : T$ in Γ has $\sigma(x)$ satisfy the shape predicate of T and the runtime
 1452 (`requires_grad` $\wedge$ `is_on_tape`) bit implies φ_{grad} in the lattice.

1453 *Preservation up to subtyping.* If $\Gamma \vdash e : T$ and $\sigma \models \Gamma$ and the step is taken, there exist $\Gamma' \supseteq \Gamma$
 1454 and T' with $\sigma' \models \Gamma'$, $\Gamma' \vdash e' : T'$, and $T' \sqsubseteq T$. By induction on the reduction relation. For each
 1455 non-context rule, the operator's transfer function in produces, by construction, the unique well-formed
 1456 shape of the result up to SMT equivalence; preservation on shapes is therefore equality modulo Γ' .
 1457 For grad, T-NOGRADCTX discharges that the body of `torch.no_grad()` produces a tensor with
 1458 `is_on_tape` = false, so no_grad is sound; T-DETACH likewise. For differentiable ops with $G = \text{on}$,
 1459 PyTorch sets `is_on_tape` = true on the result iff at least one input has it true, which is exactly the
 1460 join g . Context rules use the IH together with monotonicity of g on the lattice. Subtyping is needed
 1461 only at T-NOGRADCTX exit and at conditional joins where $\top$ is introduced.

1462 *Three-valued progress (proof-grade).* VERIFIED means every operator's shape precondition is entailed
 1463 by Γ via Z3; preservation guarantees the precondition holds at the moment of evaluation, so no
 1464 shape-mismatch reduction can occur. REFUTED-PROOF means some precondition's negation is
 1465 entailed by Γ on every model satisfying Γ , and the witness is reachable under the empty caller-rely;
 1466 at that step PyTorch raises RuntimeError. CONTRACT-VIOLATION is the same statement modulo

1467 the rely $assume_e$: under any caller respecting the declared precondition, the same reduction is forced.
 1468 LIBRARY-WARN and ABSTAIN make no claim. $\square$ $\square$

1469 D Subject Reduction and Progress for $\mathcal{F}_{TG}$

1470 This section gives a self-contained small-step operational semantics for the statement fragment $\mathcal{F}_{TG}$,
 1471 establishes Preservation (Theorem 18) and Progress (Theorem 19) as formal theorems, derives the
 1472 classical Soundness corollary, and provides full proofs. Every operator typing rule cited is from
 1473 Sections 2 and B; no rule is invented here.

1474 D.1 Runtime Configurations

1475 A *runtime value* v is a record (sh, τ, rg, ot) where $sh \in \mathbb{Z}_{>0}^*$ is the concrete shape tuple, $\tau \in \text{DType}$,
 1476 $rg \in \mathbb{B}$ is `requires_grad`, and $ot \in \mathbb{B}$ is `is_on_tape`. A *heap* $\sigma : \text{Var} \rightarrow \text{Val}$ maps variable names
 1477 to runtime values; $\sigma[x \mapsto v]$ denotes functional update. The *grad context* $G \in \{\text{on}, \text{off}\}$ records
 1478 whether the cursor is inside `torch.no_grad()`, matching the flag in Section 2.

1479 **Definition 14** (Heap satisfaction, $\sigma \models \Gamma$). $\sigma \models \Gamma$ iff for every binding $x : \text{Tensor}[\tau, \varphi_s, \varphi_g] \in \Gamma$,
 1480 writing $\sigma(x) = (sh, \tau', rg, ot)$:

- 1481 1. $\tau' = \tau$;
- 1482 2. $\varphi_s[sh/\bar{d}]$ holds (concrete instantiation of the shape predicate over $\mathcal{T}_{\text{shape}}$);
- 1483 3. if $\varphi_g = \text{has_grad}$ then $rg \wedge ot = \text{true}$; if $\varphi_g = \text{no_grad}$ then $rg \wedge ot = \text{false}$; if $\varphi_g = \top$ the
 1484 condition is vacuous.

1485 For unit bindings and static-integer bindings the condition holds trivially.

1486 We extend the statement grammar of Section 2.1 with two meta-forms used only in reduction rules:

$$s ::= \dots \mid s_1 ; s_2 \mid \text{skip}$$

1487 A *program configuration* is $\langle s, \sigma, G \rangle$; $\langle \text{return } v, \sigma, G \rangle$ is *terminal* (a value form); $\langle \text{ERROR}, \sigma, G \rangle$ is
 1488 the *error sink*.

1489 D.2 Small-Step Reduction Rules

Expression reduction $\langle e, \sigma, G \rangle \rightarrow_e \langle e', \sigma, G \rangle$.

$$\begin{array}{c}
 \text{E-VAR} \\
 \frac{x \in \text{dom}(\sigma)}{\langle x, \sigma, G \rangle \rightarrow_e \langle \sigma(x), \sigma, G \rangle} \\
 \\
 \text{E-OPERR} \\
 \frac{\sigma(\bar{x}) = \bar{v} \quad \text{prec}(op, \bar{v}) \text{ fails}}{\langle x_1.op(\bar{x}_{2..}), \sigma, G \rangle \rightarrow_e \langle \text{ERROR}, \sigma, G \rangle} \\
 \\
 \text{E-OP} \\
 \frac{\sigma(\bar{x}) = \bar{v} \quad \text{prec}(op, \bar{v}) \text{ holds}}{\langle x_1.op(\bar{x}_{2..}), \sigma, G \rangle \rightarrow_e \langle \llbracket op \rrbracket(\bar{v}, G), \sigma, G \rangle} \\
 \\
 \text{E-NGBODY} \\
 \frac{\langle e, \sigma, \text{off} \rangle \rightarrow_e \langle e', \sigma, \text{off} \rangle}{\langle \text{no_grad}(e), \sigma, G \rangle \rightarrow_e \langle \text{no_grad}(e'), \sigma, G \rangle} \\
 \\
 \text{E-NGVAL} \\
 \frac{v = (sh, \tau, rg, ot) \quad v' = (sh, \tau, rg, \text{false})}{\langle \text{no_grad}(v), \sigma, G \rangle \rightarrow_e \langle v', \sigma, G \rangle}
 \end{array}$$

1490 Here $\text{prec}(op, \bar{v})$ is the shape-compatibility predicate for op (e.g. equal contracting dimensions for
 1491 `matmul`; divisibility for `view(-1)`), and $\llbracket op \rrbracket(\bar{v}, G)$ is the concrete result whose shape is computed
 1492 by the same transfer function as the typing rule. E-NGBODY / E-NGVAL evaluate a `no_grad`-
 1493 wrapped expression with $G = \text{off}$ and strip the tape bit on exit, matching T-NOGRADCTX. Tuple
 1494 construction, module calls $C(\bar{x})$, and attribute reads self.attr reduce component-wise via standard
 1495 eval-context rules (omitted).

Statement reduction $\langle s, \sigma, G \rangle \rightarrow \langle s', \sigma', G' \rangle$.

$$\begin{array}{c}
\text{E-EVALCTX} \\
\frac{\langle e, \sigma, G \rangle \rightarrow_e \langle e', \sigma, G \rangle}{\langle x = e; s, \sigma, G \rangle \rightarrow \langle x = e'; s, \sigma, G \rangle} \\
\\
\text{E-EVALERR} \\
\frac{\langle e, \sigma, G \rangle \rightarrow_e \langle \text{ERROR}, \sigma, G \rangle}{\langle x = e; s, \sigma, G \rangle \rightarrow \langle \text{ERROR}, \sigma, G \rangle} \\
\\
\text{E-IFT} \\
\frac{c \text{ evaluates to true in } \sigma}{\langle \text{if}^\dagger c : s_1 \text{ else } s_2; s, \sigma, G \rangle \rightarrow \langle s_1; s, \sigma, G \rangle} \\
\\
\text{E-IFFF} \\
\frac{c \text{ evaluates to false in } \sigma}{\langle \text{if}^\dagger c : s_1 \text{ else } s_2; s, \sigma, G \rangle \rightarrow \langle s_2; s, \sigma, G \rangle} \\
\\
\text{E-FORZ} \\
\frac{}{\langle \text{for}^\dagger x \text{ in range}(0) : s_b; s, \sigma, G \rangle \rightarrow \langle s, \sigma, G \rangle} \\
\\
\text{E-FORS} \\
\frac{n > 0}{\langle \text{for}^\dagger x \text{ in range}(n) : s_b; s, \sigma, G \rangle \rightarrow \langle s_b[0/x]; \text{for}^\dagger x \text{ in range}(n-1) : s_b[(x+1)/x]; s, \sigma, G \rangle} \\
\\
\text{E-SKIP} \\
\frac{}{\langle \text{skip}; s, \sigma, G \rangle \rightarrow \langle s, \sigma, G \rangle}
\end{array}$$

1496 In E-FORS, $s_b[(x+1)/x]$ denotes the substitution of $x+1$ for every free occurrence of x in s_b ;
1497 iteration i of the residual body receives index $i+1$, unrolling correctly to iterations $1, \dots, n-1$.
1498 Branch determinism (E-IFT/E-IFFF) holds by the shape-determinism restriction: c is a static integer
1499 expression, and $\sigma \models \Gamma$ instantiates all shape variables, so c has a unique Boolean value. Loop
1500 termination follows because the bound n strictly decreases in E-FORS and is non-negative (it is a
1501 natural number derivable from shape refinements).

1502 D.3 Statement Typing

1503 The expression typing judgment $\Gamma \vdash e : T$ is from Section 2. We extend it to statements with
1504 $\Gamma \vdash s : \tau$, meaning that s terminates with a value of type τ in context Γ .

$$\begin{array}{c}
\text{T-ASSIGN} \\
\frac{\Gamma \vdash e : T \quad \Gamma, x:T \vdash s : \tau}{\Gamma \vdash (x = e; s) : \tau} \\
\\
\text{T-RETURN} \\
\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \text{return } e : \tau} \\
\\
\text{T-IF} \\
\frac{\Gamma \vdash s_1 : \tau \quad \Gamma \vdash s_2 : \tau \quad c \text{ static in } \Gamma}{\Gamma \vdash (\text{if}^\dagger c : s_1 \text{ else } s_2) : \tau} \\
\\
\text{T-FOR} \\
\frac{\Gamma, x:\text{int}[0..n] \vdash s_b : \text{unit} \quad n \text{ static in } \Gamma}{\Gamma \vdash (\text{for}^\dagger x \text{ in range}(n) : s_b) : \text{unit}} \\
\\
\text{T-SEQ} \\
\frac{\Gamma \vdash s_1 : \text{unit} \quad \Gamma' \vdash s_2 : \tau \quad \Gamma' \supseteq \Gamma}{\Gamma \vdash (s_1; s_2) : \tau} \\
\\
\text{T-SKIP} \\
\frac{}{\Gamma \vdash \text{skip} : \text{unit}}
\end{array}$$

1505 T-IF requires both branches to return the *same* type τ , using shape-determinism to avoid a lattice
1506 join on the heap. T-ASSIGN uniquely determines T via one of the expression rules (T-MATMUL,
1507 T-VIEW(-1), etc.) from Sections 2 and B.

1508 D.4 Preliminary Lemmas

1509 **Lemma 15** (Canonical Forms). *If $\Gamma \vdash x : \text{Tensor}[\tau, \varphi_s, \varphi_g]$, $x \in \text{dom}(\sigma)$, and $\sigma \models \Gamma$, then*
1510 *writing $\sigma(x) = (sh, \tau', rg, ot) : \tau' = \tau$, $\varphi_s[sh/\bar{d}]$ holds, and the grad condition of Theorem 14(iii) is*
1511 *satisfied.*

1512 *Proof.* Immediate from Theorem 14. □

1513 **Lemma 16** (Substitution). *If $\Gamma, x:S \vdash s : \tau$ and $\Gamma \vdash v : S$, then $\Gamma \vdash s[v/x] : \tau$.*

1514 *Sketch.* Standard structural induction on s . Each use of x in s is replaced by v ; the unique type S is
1515 preserved at every use site. Shape predicates mentioning x are instantiated pointwise in $\mathcal{T}_{\text{shape}}$. □

1516 **Lemma 17** (Reduction Lemma). *If $\Gamma \vdash e : T$, $\sigma \models \Gamma$, and $\langle e, \sigma, G \rangle \rightarrow_e \langle v, \sigma, G \rangle$ via E-OP*
1517 *(non-error), then v satisfies type T in σ .*

1518 *Proof.* By case analysis on the expression typing rule for e .

1519 *Case E-VAR / T-VAR.* $e = x, v = \sigma(x)$. By Theorem 15, v satisfies T .

1520 *Case T-MATMUL.* Premises: $a : \text{Tensor}[\tau, (\bar{B}, m, k), \varphi_a], b : \text{Tensor}[\tau, (\bar{B}', k, n), \varphi_b], \Gamma \models$
 1521 $\text{broadcast}(\bar{B}, \bar{B}') = \bar{C}$. By Theorem 15, the concrete shapes satisfy these conditions. E-OP
 1522 fires because $\text{prec}(\text{matmul}, \bar{v})$ (equal contracting dimension) holds. PyTorch returns $(\bar{C}_\sigma, m_\sigma, n_\sigma)$
 1523 where $\bar{C}_\sigma$ is the concrete broadcast of $\bar{B}_\sigma$ and $\bar{B}'_\sigma$. The type $\text{Tensor}[\tau, (\bar{C}, m, n), g(a, b)]$ is satisfied:
 1524 $\bar{C}_\sigma$ concretely instantiates $\bar{C}$, and $g(a, b) = \bigsqcup(\varphi_a^g, \varphi_b^g)$ matches PyTorch's autograd contract (output
 1525 $ot = \text{true}$ iff some input has $ot = \text{true}$ and $G = \text{on}$).

1526 *Case T-VIEW(-1).* Premises: $\Gamma \models Q \mid P$ where $P = \prod_i d_i, Q = \prod_{i \neq j} s_i$. E-OP fires only when
 1527 $Q_\sigma \mid P_\sigma$; the inferred dimension P_σ/Q_σ is the concrete instantiation of P/Q . Shape and grad are
 1528 preserved (view is metadata-only).

1529 *Case T-DETACH.* The result has $ot = \text{false}$ by PyTorch's detach contract, satisfying no_grad.

1530 *Case T-NOGRADCTX.* By E-NGVAL, the output value has $ot = \text{false}$ regardless of its origin,
 1531 satisfying no_grad as asserted.

1532 *Case T-CONV2D.* The output dimensions H', W' are deterministic arithmetic functions of H, W
 1533 and the convolution hyperparameters (stride, padding, dilation, kernel); PyTorch computes exactly
 1534 the same formula. Grad: as in T-MATMUL.

1535 *Case T-CAT.* The concatenated dimension is $\sum_i d_k^{(i)}$; the concrete value is $\sum_i \sigma(d_k^{(i)})$, instantiating
 1536 the symbolic type.

1537 *Case T-SPLIT.* E-OP fires when $s_\sigma \mid d_{k,\sigma}$; each output chunk has concrete size $s_\sigma = \sigma(s)$, matching
 1538 s .

1539 *Case T-SDPA.* Premise $T = S$ is checked by prec; output shape (B, H, T, D) matches.

1540 *Case T-LAYERNORM.* Output shape equals input shape (only rank-matching is required); grad as in
 1541 T-MATMUL.

1542 *Case T-BACKWARD / T-OPTSTEP.* Result type is unit; the condition is vacuous.

1543 *Lean-audited operators* (Table 9, 36 operators including T-LINEAR, T-BMM, T-PERMUTE,
 1544 T-EXPAND, T-REPEAT, T-STACK, T-CHUNK, T-UNBIND, T-GATHER, T-SCATTER, T-
 1545 INDEXSELECT, T-NARROW, T-EMBED, T-RMSNORM, T-SQUEEZE, T-UNSQUEEZE, T-ARGMAX,
 1546 T-CROSSENTROPY, T-DROPOUT, T-CONTIGUOUS, T-CLAMP, T-TO). Each Lean definition for-
 1547 malises: for any input satisfying the rule's premises, the rule's output shape formula equals PyTorch's
 1548 runtime output—this is exactly the statement of the present lemma for each such operator, and the
 1549 Lean proof discharges it.

1550 *Pen-and-paper operators* (T-BROADCAST, T-REDUCE, T-EINSUM). Stated and proved in the pen-
 1551 and-paper note of Table 9; shape transfer is exact arithmetic and is verified by the same argument as
 1552 T-CAT above.

1553 *Tested-only operators:* outside the scope of this theorem (cf. the boundary note in Theorem 1). $\square$

1554 D.5 Preservation (Subject Reduction)

1555 **Theorem 18** (Preservation). *If $\Gamma \vdash s : \tau, \sigma \models \Gamma$, and $\langle s, \sigma, G \rangle \rightarrow \langle s', \sigma', G' \rangle$ is a non-error step,*
 1556 *then there exists $\Gamma' \supseteq \Gamma$ such that $\sigma' \models \Gamma'$ and $\Gamma' \vdash s' : \tau$.*

1557 *Proof.* By case analysis on the reduction rule applied.

1558 **Case E-EVALCTX.** $s = x = e; s_0$, step has $e \rightarrow_e e'$ (non-error), $s' = x = e'; s_0, \sigma' = \sigma,$
 1559 $G' = G$. By T-ASSIGN, $\Gamma \vdash e : T$ for some T . By Theorem 17 (applied to the sub-expression step),
 1560 the partially-reduced e' still types at T under Γ (expression reduction does not change σ , so $\sigma \models \Gamma$ is
 1561 preserved). Setting $\Gamma' = \Gamma: \sigma' \models \Gamma'$ and $\Gamma \vdash (x = e'; s_0) : \tau$ by T-ASSIGN.

1562 **Case E-BIND.** $s = x = v; s_0, s' = s_0, \sigma' = \sigma[x \mapsto v], G' = G$. By T-ASSIGN: $\Gamma \vdash v : T$
 1563 (already a value at type T because the preceding E-EVALCTX steps preserved typing, by the previous
 1564 case) and $\Gamma, x:T \vdash s_0 : \tau$. Since v satisfies T (by Theorem 17 on the completed expression-evaluation

sub-sequence), $\sigma[x \mapsto v] \models \Gamma, x:T$: all old bindings are unchanged and $x:T$ is freshly satisfied. Set $\Gamma' = \Gamma, x:T$; then $\sigma' \models \Gamma'$ and $\Gamma' \vdash s_0 : \tau$.

Case E-IFT. $s = \text{if}^\dagger c : s_1 \text{ else } s_2 ; s_0$. By T-IF, $\Gamma \vdash s_1 : \tau$. Step gives $s' = s_1 ; s_0$; by T-SEQ, $\Gamma \vdash s_1 ; s_0 : \tau$. $\sigma' = \sigma, G' = G, \Gamma' = \Gamma$.

Case E-IFF. Symmetric; $\Gamma \vdash s_2 : \tau$ from T-IF.

Case E-FORZ. $s = \text{for}^\dagger x \text{ in range}(0) : s_b ; s_0$. Step gives $s' = s_0$. By T-FOR, the for-loop has type unit; by T-SEQ, $\Gamma \vdash s_0 : \tau$. $\Gamma' = \Gamma$.

Case E-FORS. $s = \text{for}^\dagger x \text{ in range}(n) : s_b ; s_0, n > 0$. By T-FOR: $\Gamma, x:\text{int}[0..n] \vdash s_b : \text{unit}$; the step produces $s' = s_b[0/x] ; \text{for}^\dagger x \text{ in range}(n-1) : s_b[(x+1)/x] ; s_0$.

1. $\Gamma \vdash s_b[0/x] : \text{unit}$: since $n > 0, 0 \in [0..n]$, so Theorem 16 applies.
2. $\Gamma, x:\text{int}[0..n-1] \vdash s_b[(x+1)/x] : \text{unit}$: in the residual body x ranges over $[0..n-1]$ and $x+1$ ranges over $[1..n] \subseteq [0..n]$; Theorem 16 (with term $x+1 : \text{int}[1..n]$) gives this.
3. By T-FOR with bound $n-1$: $\Gamma \vdash \text{for}^\dagger x \text{ in range}(n-1) : s_b[(x+1)/x] : \text{unit}$.
4. By T-SEQ applied twice: $\Gamma \vdash s' : \tau$.

$\Gamma' = \Gamma, \sigma' = \sigma$.

Case E-SKIP. $s = \text{skip} ; s_0 \rightarrow s_0$. By T-SEQ, $\Gamma \vdash s_0 : \tau$. $\Gamma' = \Gamma, \sigma' = \sigma$.

All cases produce $\Gamma' \supseteq \Gamma$ with $\sigma' \models \Gamma'$ and $\Gamma' \vdash s' : \tau$. □

D.6 Progress

Theorem 19 (Progress). *If $\Gamma \vdash s : \tau$ and $\sigma \models \Gamma$, then one of:*

1. $s = \text{return } v$ is terminal (value form);
2. there exists $\langle s', \sigma', G' \rangle$ such that $\langle s, \sigma, G \rangle \rightarrow \langle s', \sigma', G' \rangle$; or
3. $\langle s, \sigma, G \rangle \rightarrow \langle \text{ERROR}, \sigma, G \rangle$, and this arises only when $\text{prec}(op, \bar{v})$ fails for some operator op in s —equivalently, when the corresponding typing-rule premise (T-MATMUL's k -equality, T-VIEW(-1)'s divisibility, etc.) is not satisfied by the concrete shapes in σ .

Proof. By structural induction on the typing derivation $\Gamma \vdash s : \tau$.

Case T-RETURN. $s = \text{return } e$. If e is a value, clause (a). Otherwise, locate the innermost non-value sub-expression of e ; by the grammar of $\mathcal{F}_{\text{TG}}$, it is of the form $x_1.op(\bar{x}_2..)$ with all $x_i \in \text{dom}(\sigma)$. If $\text{prec}(op, \sigma(\bar{x}))$ holds, E-OP fires and the result propagates via E-EVALCTX to produce a step (clause b); if it fails, E-OPERR fires followed by E-EVALERR (clause c with the named precondition failure).

Case T-ASSIGN. $s = x = e ; s_0$. Identical argument on e ; if e is already a value, E-BIND fires (clause b).

Case T-IF. The shape-determinism restriction ensures c is a static integer expression. Since $\sigma \models \Gamma$, all shape variables in c are instantiated and c evaluates to a unique Boolean: E-IFT or E-IFF applies. Clause (b).

Case T-FOR. The loop bound n is static in Γ and hence determined by σ ; $n \geq 0$ since it is a natural number derivable from shape refinements. If $n = 0$, E-FORZ applies; if $n > 0$, E-FORS applies. Clause (b).

Case T-SEQ. $s = s_1 ; s_2$. If $s_1 = \text{skip}$, E-SKIP applies (clause b). Otherwise the IH on s_1 gives clause (b) (a step for s_1 , propagated to $s_1 ; s_2$ by the eval-context rule for sequences) or clause (c) (an error propagated by E-EVALERR).

1605 No other stuck states arise within $\mathcal{F}_{\text{TG}}$: data-dependent branches are excluded by definition; dynamic
 1606 dispatch outside Cat triggers ABSTAIN at the analysis phase and does not reach these semantics. $\square$

1607 D.7 Soundness Corollary

1608 **Corollary 20** (Soundness, classical form). *Let $s \in \mathcal{F}_{\text{TG}}$ with $\Gamma \vdash s : \tau$ and $\sigma \models \Gamma$. If every operator*
 1609 *op reachable during reduction of $\langle s, \sigma, G \rangle$ satisfies $\text{prec}(\text{op}, \bar{v})$ in σ , then no reduction sequence*
 1610 *raises SHAPE-MISMATCH-ERROR.*

1611 *Proof.* Under the hypothesis, Theorem 19(c) never applies. Theorem 18 guarantees that each non-
 1612 error step yields a well-typed configuration $(\Gamma' \supseteq \Gamma, s', \sigma')$ with $\sigma' \models \Gamma'$, so the precondition
 1613 hypothesis is maintained at every subsequent step. By induction on the length of any finite reduction
 1614 prefix, the error sink is never reached. $\square$

1615 **Relationship to Theorem 2.** Theorem 2 (soundness of proof-grade verdicts) is the *abstract-*
 1616 *interpretation specialisation* of Theorems 18 and 19 to the verdict lattice. Concretely: VERIFIED
 1617 means Z3 has symbolically discharged $\text{prec}(\text{op}, \cdot)$ for all operators under Γ , so the precondition
 1618 hypothesis of Theorem 20 holds for every $\sigma \models \Gamma$, yielding Theorem 2(i). REFUTED-PROOF witnesses
 1619 a concrete $\sigma \models \Gamma$ in which some prec fails, so Theorem 19(c) fires at the witnessed step, giving
 1620 Theorem 2(ii). Hence Theorem 2 is exactly the pair (Theorem 18, Theorem 19) run through the
 1621 Z3 abstraction layer; the verdict lattice is the coarsened output domain.

1622 E Assume/Guarantee proofs

1623 **Fresh-witness axiom for Theorem 12.** The analyser emits REFUTED-PROOF (or CONTRACT-
 1624 VIOLATION) for a module M under context Γ only if, in the current pass under Γ , it produces a
 1625 concrete input x with $\text{assume}_M(x) \wedge \neg \text{guarantee}_M(M(x))$ (i.e. cached witnesses from prior passes
 1626 do not count). Under this axiom Theorem 12 follows by case analysis on the form of refinement:
 1627 strengthening G_{M_0} to G'_{M_0} leaves the discharged obligation $A_{M_0} \Rightarrow G'_{M_0}$ strictly stronger than
 1628 the prior one, so Z3 either succeeds (VERIFIED preserved) or returns unknown/timeout (ABSTAIN);
 1629 strengthening A_{M_0} to A'_{M_0} shrinks the set of inputs reaching M_0 and so cannot create a fresh
 1630 refutation witness. Theorem 9 follows by induction on the call-tree using SEQ, PAR, and SUB
 1631 together with Theorem 2 at each leaf.

1632 F Dynamo-guard correspondence: full proof

1633 The (necessary-direction) proof is by induction on the typing rules of Section 2. For each operator
 1634 the input metadata read by Dynamo’s fake-tensor execution is exactly the metadata referenced by
 1635 the rule’s premise: T-MATMUL reads the k -dim and the broadcast batch shapes, which are the same
 1636 quantities Dynamo specialises on at the `aten::matmul` call; T-VIEW(-1) reads the input’s element
 1637 count and the explicit reshape factors, which are the quantities Dynamo’s `ShapeEnv` guards on. The
 1638 *converse* fails by inspection of Dynamo’s guard emitter: guards are also installed on metadata strictly
 1639 outside the TG fragment (Python ints, list lengths, integer parameters captured at trace time), so an
 1640 in-contract input may still trip a non-shape guard. Outside the catalogue intersection TG abstains
 1641 and Dynamo reports a graph break; the empirical audit (Section 4) shows on 17 real modules the
 1642 in-contract recompile rate is 8.8% (48/544) — a calibrated reminder of the one-directionality — and
 1643 the OOS violation rate is 97.9% (46/47).

1644 **One end-to-end step (T-MATMUL).** Take the typing-rule premise of T-MATMUL from Section 2:

$$\text{Tensor}\{s_1 \mid \varphi_1\} \otimes \text{Tensor}\{s_2 \mid \varphi_2\} \implies \varphi_1 \wedge \varphi_2 \wedge s_1[-1] = s_2[-2] \wedge \text{broadcastable}(s_1[-2], s_2[-2]).$$

1645 The four refinement variables read on the left are: $s_1[-1]$ (the contraction dim of operand 1), $s_2[-2]$
 1646 (the contraction dim of operand 2), $s_1[-2]$ (the leading-batch shape of operand 1), $s_2[-2]$
 1647 (the leading-batch shape of operand 2). Dynamo’s `matmul` handler is implemented in and dis-
 1648 patches through `__torch_dispatch__` to `aten::matmul`. On a `FakeTensor` input the dispatcher
 1649 reads exactly the following four metadata bits via the `FakeTensor.__torch_dispatch__` symint
 1650 specialiser (in torch 2.9.1): `op.size(-1)` on operand 1, `op.size(-2)` on operand 2, and the

Table 8: Calibrated comparison along nine PL/ML axes. $\checkmark$ = supported; $\times$ = not supported; ∂ = partial / under-developed; — = not applicable. “Refinement-typed” means types carry SMT-decidable predicates (not just shape literals). “Symbolic shapes” means dimensions may be unknown variables with constraints. “Grad-flag tracking” means the type system reasons about `requires_grad`. “Backward verifier” means the system checks well-formedness of the autograd backward pass, not only the forward computation. “Lean-audited rules” means the operator shape-transfer table is mechanically defined and empirically audited against PyTorch (the analyser implementation itself is not mechanised). “No-execution” means the analysis runs without invoking a tracer, GPU, or Python interpreter on user code. “Abstention 1st-class” means ABSTAIN is a reported verdict distinct from VERIFIED/REFUTED. “Modern Transformer support” is a coarse practical judgement of whether the tool handles HuggingFace-style attention/KV-cache code out of the box.

Tool	Refinement-typed	Symbolic shapes	Grad-flag tracking	Backward verifier	Contracts / A-G	Lean-audited rules	No-execution	Abstention 1st-class	Modern Transformer
TensorGuard	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Pytea	∂	$\checkmark$	$\times$	$\times$	$\times$	$\times$	$\checkmark$	$\times$	∂
jaxtyping	$\times$	∂	$\times$	$\times$	∂	$\times$	$\times$	$\times$	$\checkmark$
torchtyping	$\times$	∂	$\times$	$\times$	∂	$\times$	$\times$	$\times$	$\checkmark$
FakeTensorMode	$\times$	$\checkmark$	$\times$	∂	$\times$	$\times$	$\times$	$\times$	$\checkmark$
torch.export	$\times$	$\checkmark$	$\times$	∂	$\times$	$\times$	$\times$	$\times$	$\checkmark$
MyPy / Pyright	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\checkmark$	$\times$	$\times$
beartype	$\times$	$\times$	$\times$	$\times$	∂	$\times$	$\times$	$\times$	$\checkmark$

1651 `torch._C._compute_broadcast_shape(op1.size()[:-2], op2.size()[:-2])` call which it-
 1652 self reads the shape-prefix-by-prefix as a sequence of `symint` reads. For each shape-refinement
 1653 variable in φ , we exhibit the corresponding metadata-read line in the released data and ; the discharge
 1654 is that any guard installed on these four metadata bits is satisfied iff the same conjunct of φ holds,
 1655 so a VERIFIED TG-contract entails the absence of recompiles on the matmul site. The correspond-
 1656 ing proof step for T-VIEW(-1) is recorded as data, which exposes the same metadata-read map
 1657 programmatically (as a machine-checkable JSON keyed by typing rule).

1658 **Limitation.** The above is one end-to-end step; the remaining 26 Lean-audited handlers admit the
 1659 same form of mapping but the appendix only writes out T-MATMUL and T-VIEW(-1) explicitly.
 1660 The remaining mappings live as data, not as $\text{\LaTeX}$ prose. We treat this as a Limitation rather than a
 1661 gap in the theorem statement.

1662 **Proposition 21** (Soundness of the `einsum` pen-and-paper rule). *Let an `einsum` call have left-*
 1663 *hand-side subscripts $L = (L^{(1)}, \dots, L^{(n)})$ where $L^{(i)} \in \Sigma^*$ for an alphabet Σ of axis labels,*
 1664 *with input tensors of symbolic shapes $\bar{d}^{(1)}, \dots, \bar{d}^{(n)}$ such that $|L^{(i)}| = |\bar{d}^{(i)}|$ for each i (subscript-*
 1665 *arity condition). Let $R \in \Sigma^*$ be the right-hand-side subscript with no repeated label (output-*
 1666 *uniqueness condition) and $R \subseteq \bigcup_i L^{(i)}$ (output-coverage condition). Define the per-label envelope*
 1667 *$\text{env}(\ell) := \{\bar{d}_j^{(i)} : L_j^{(i)} = \ell\}$ for each $\ell \in \Sigma$. The rule’s precondition is the label-consistency*
 1668 *obligation $\bigwedge_{\ell \in \Sigma} \bigwedge_{d, d' \in \text{env}(\ell)} (d = d') \vee (d = 1) \vee (d' = 1)$ (broadcast-axis identification), and the*
 1669 *rule’s output shape is $\bar{d}_k^R := \max \text{env}(R_k)$ for each k (the broadcast resolution; well-defined on*
 1670 *satisfying assignments). **Soundness.** If the precondition is discharged by Z3 on $\Gamma \models \varphi_{\text{op}}$ and the*
 1671 *subscript-arity, output-uniqueness, and output-coverage syntactic conditions hold, then for every*
 1672 *$\sigma \models \Gamma$ the runtime `torch.einsum` call with the same subscripts on $\sigma(\bar{x})$ either succeeds with output*
 1673 *of shape $\bar{d}^R$ (under the rule’s symbolic image $\sigma(\bar{d}^R)$) or raises a runtime exception on the violated*
 1674 *label-consistency check, by direct case analysis on the `at::native::einsum` dispatch. **Refutation***
 1675 ***completeness.** If Z3 produces a model $\mathfrak{m}$ falsifying the label-consistency obligation, the heap $\sigma_{\mathfrak{m}}$ of*
 1676 *Theorem 3 drives the runtime call into the same exception branch, discharging Theorem 2(ii) on this*
 1677 *handler.*

1678 G Reproducibility

1679 Driver scripts and JSON outputs are organised under : the block- and bug-corpus builders,
1680 the TensorGuard runner, the baseline-comparison runner, the Lean parity harness, the Dynamo-
1681 correspondence runner, the feature-ablation runner, the localisation-quality runner, and the hybrid-
1682 mode runner each emit a single JSON output (SHA-pinned manifest, calibration notes, per-item
1683 records). The Lean development is at and is built/run via the lake commands documented in the
1684 repo. Pinned versions: torch 2.9.1, torchvision 0.24.1, transformers 4.57.3, timm 1.0.26,
1685 python 3.11.15; mamba_ssm could not be installed on the CPU-only target machine and is excluded.

1686 H Silent-miss autopsy on the historical 60-bug corpus

1687 The four VERIFIED-but-buggy items (bug_001, bug_002, bug_006, bug_007) split as follows.
1688 **bug_001** (SDPA enable_gqa=True attn_mask broadcast): the T-SDPA rule (Section B) is stated
1689 for the standard (B, H, T, D) contract; the GQA-distinct query-head count is an SDPA-rule premise
1690 we did not state, so the SMT obligation $H_q = H_k$ that should have fired was never raised. **bug_002**
1691 (torch.isclose broadcast on a 0-dim NaN scalar): isclose is not in the operator catalogue (Sec-
1692 tion A); the analyser treated it as a passthrough rather than a broadcast op and never emitted the obli-
1693 gation $\text{shape}(a) = \text{broadcast}(\text{shape}(a), \text{shape}(b))$. **bug_006** (nn.Conv1d with channels-last
1694 input): the rule T-CONV1D reads the input layout from a positional channel index (cin = shape[1]);
1695 the layout-misordered repro lay outside the layout-fixed fragment. **bug_007** (intra-batch dynamic
1696 torch.unique): data-dependent shape, formally out-of-fragment per Section 2.1 (the dim is a
1697 function of tensor values). The first three are missing-rule fixable; the fourth is a deliberate fragment
1698 exclusion. Item-level autopsy with the expected SMT obligation per case is given in the appendix.

1699 I Lean residual: permList_compose

1700 The previously-residual sorry concerned the rule permList_compose, originally stated as
1701 $\sigma_2(\sigma_1(L)) = (\sigma_2 \circ \sigma_1)(L)$ for total permutations L, σ_i . Lean’s List.get?-with-default-0 semantics
1702 breaks the identity at out-of-range indices — a counterexample is $L=[a], \sigma_1=[1, 0]$ (out-of-range
1703 index 1): the LHS reads $L!1=0$ then permutes, the RHS composes permutation indices first and
1704 reads $L!(\sigma_2 \circ \sigma_1)(0)$. The lemma is now stated as permList_compose_inrange (the in-range form,
1705 $\forall i \in [0, |L|], \sigma_2(\sigma_1)(i) < |L|$) and is closed sorry-free by pointwise extensionality on the List.map
1706 form. permList is not in the TG forward-rule path that Theorem 2 or the audit JSON depends on;
1707 we keep the corrected lemma in the residual file for completeness.

1708 J Shape-pattern coverage suite (catalogue-coverage check)

1709 The files are hand-distilled one-line repros of the same upstream bugs as the upstream-faithful
1710 corpus (Table 2), reduced to a single forward(self, x): return x.view(...) call with the
1711 divisibility-arithmetic content of the bug pre-extracted by hand. TG returns 10/10 REFUTED-PROOF
1712 at confidence 0.99 on this suite. This is a *catalogue-coverage sanity check* — it verifies that TG’s
1713 rule table covers each of the shape-arithmetic bug patterns once a human has already isolated them —
1714 and is *not* a real-bug evaluation; the live-class result is the 7/10 + 1/10 + 2/10 triple in Table 2. We
1715 retain the suite as a regression target for the rule catalogue (a future rule table change that silently
1716 drops one of these patterns is detectable by re-running the suite); the headline counts in the body do
1717 not include it.

1718 K LW→RP gap on the 488-block corpus

1719 Of the 78 LIBRARY-WARN verdicts on the block corpus, 66 (84.6%) are dispatch-outside-fragment
1720 in a sense no catalogue extension would convert to REFUTED-PROOF: 37 classes have no forward
1721 body in scope at all (inheritance-only stubs); 27 are subclasses of user-defined non-nn.Module
1722 bases whose forward semantics are inherited; 2 iterate over an opaque self.<container>. The
1723 residual 12 (15.4%) use only fragment ops in their forward bodies and are an upper bound on the
1724 LW→RP conversions strengthening catalogue-internal reasoning could yield without enlarging the

operator-catalogue fragment. Reproducibility artefact: the per-block LW classification is recorded in the reproducibility appendix.

L Pytea modern-subset matched-pair contingency table

The 34-bug modern subset of the 60-bug corpus is the set of bugs whose primary operator is in Pytea’s 2022 catalogue. The table below lists every bug in the subset together with TG’s fragment-fairness-enforced verdict and Pytea’s verdict, so the matched-pair structure summarised in Section 4.1 ($a=25$ both-refute, $b=7$ TG-only, $c=0$ Pytea-only, $d=2$ neither) is auditable at the per-bug level. The agreement column applies the same accounting as the McNemar test reported in the body. Each row is regenerated from the TG verdict log and the Pytea verdict log on every build; no number is hand-entered.

The four equivalence classes ($a/b/c/d$) are labelled in the agreement column. The seven b -cell rows (12, 13, 18, 22, 23, 26, 33) are the TG-only catches that drive the matched-pair gap; c is empty because every Pytea-refute on this subset is also a TG-refute. R= REFUTED, V=VERIFIED, N/A=tool inapplicable (treated as not-refute in the agreement column). *Convention disclosure.* The headline $b=7$, $p=0.0156$ treats the two Pytea N/A rows (13, 23) as not-refute under the conservative reading that an inapplicable tool is not a miss. Under the alternative reading that an inapplicable tool is reclassified as a silent miss (Pytea “catches imply TG catches” becomes a strict inclusion across all 34 rows), the convention reclassifies these two N/A rows plus the third row that yields a Pytea silent-skip into the b cell, giving $b=10$, $c=0$ and McNemar exact two-sided $p=0.00195$. Both numbers are released so the choice of convention is auditable; the 25/34 headline used in the abstract and body is the conservative one.

Deterministic 60→34 filter rule. The 34-row subset is the deterministic image of the following one-line rule applied to the 60-bug corpus: *a bug is in the fragment-fair subset iff its primary failing operator has an implementation in Pytea’s 2022 TypeScript operator catalogue* (the `torchOpHandler_*` table at the snapshot tagged c536515, dated 2022-04-26, the last commit of upstream Pytea before the project went dormant). The rule is mechanical: for each of the 60 bugs, the primary failing operator is recorded in the bug’s JSON stub at corpus-build time; that string is looked up against the catalogue, and the bug is included iff the lookup returns a handler. The classifier is therefore independent of TG’s own verdict and predates the head-to-head: it cannot select for TG-favourable bugs. The 26 excluded bugs are the bugs whose primary operator is not in the 2022 catalogue, listed exhaustively in Table 11 together with the catalogue-miss reason. Of the 26, 5 are `einsum`, 3 are PyTorch-2.x attention APIs (SDPA, MHA-2.x), 3 are `Conv1d/Conv3d`, 3 are non-2D normalisation (`BatchNorm1d`, `GroupNorm`, `InstanceNorm2d`), and the remaining 12 are scattered ops with no Pytea handler at all (`where`, `gather`, `scatter_`, `functional torch.add`, `linalg.inv`, ...).

1759 NeurIPS Paper Checklist

1760 1. Claims

1761 Question: Do the main claims made in the abstract and introduction accurately reflect the
1762 paper’s contributions and scope?

1763 Answer: [Yes]

1764 Justification: The abstract and introduction state contributions C1–C6 and report the headline
1765 numbers (53/60 on the historical bug corpus; 32/34 on the modern Pytea-fragment subset;
1766 0 unconditional Refuted-Proof on the 488-block applicability survey; 11/11 Lean lemmas
1767 closed sorry-free); the scope of the soundness theorem and the unmechanised parts of the
1768 implementation are stated explicitly in Sections 4.4 and 6.

1769 2. Limitations

1770 Question: Does the paper discuss the limitations of the work performed by the authors?

1771 Answer: [Yes]

1772 Justification: Section 6 discusses the supported language fragment, the constructor-bound
1773 integer-attribute envelope class of remaining silent verifieds, the gap between the Lean-
1774 audited operator-rule table and the un-mechanised Python analyser, the first-order grad-
1775 flag lattice’s $\leq 12\%$ silent-misclassification regime on parameter-sharing under renamed
1776 attributes, and the small- N statistical limits of the post-freeze unfiltered evaluation.

1777 3. Theory Assumptions and Proofs

1778 Question: For each theoretical result, does the paper provide the full set of assumptions and
1779 a complete (and correct) proof?

1780 Answer: [Yes]

1781 Justification: Theorems 2 (soundness), 18–19 (preservation/progress for the calculus), the
1782 assume/guarantee composition theorem, and the Dynamo-correspondence lemma (Theo-
1783 rem 13) are stated with all assumptions and proved in the appendix; LIBRARY-WARN and
1784 ABSTAIN are explicitly excluded from the soundness claim. The Lean 4 operator-rule audit
1785 is sorry-free at the rule definitions, with the soundness lemma scope (28 of 79 handlers)
1786 tabulated in Table 9.

1787 4. Experimental Result Reproducibility

1788 Question: Does the paper fully disclose all the information needed to reproduce the main ex-
1789 perimental results of the paper to the extent that it affects the main claims and/or conclusions
1790 of the paper?

1791 Answer: [Yes]

1792 Justification: All evaluation numbers are produced by deterministic single-CPU driver
1793 scripts whose inputs (corpora, freeze hashes, pre-registered queries) are committed; the
1794 released artefact contains the verifier source, the Lean development, the benchmark drivers,
1795 and the JSON inputs that produce every reported table.

1796 5. Open access to data and code

1797 Question: Does the paper provide open access to the data and code, with sufficient instruc-
1798 tions to faithfully reproduce the main experimental results, as described in supplemental
1799 material?

1800 Answer: [Yes]

1801 Justification: An anonymised supplementary archive accompanies the submission; it contains
1802 the verifier source, the Lean development, all benchmark drivers, the committed JSON
1803 corpora, and a top-level README giving the exact commands needed to reproduce every
1804 table and figure.

1805 6. Experimental Setting/Details

1806 Question: Does the paper specify all the training and test details necessary to understand the
1807 results?

1808 Answer: [Yes]

1809 Justification: There is no model training. Section 4 and the appendix specify the analysed
1810 modules, the input shape envelopes, the analyser configuration, the catalogue freeze commit,
1811 the modern-subset construction predicate, the post-freeze pre-registered query date, and the
1812 matched-pair test used for the Pytea comparison.

1813 **7. Experiment Statistical Significance**

1814 Question: Does the paper report error bars suitably and correctly defined or other appropriate
1815 information about the statistical significance of the experiments?

1816 Answer: [Yes]

1817 Justification: Wilson 95% confidence intervals are reported for the bug-corpus and post-
1818 freeze catch rates; a McNemar exact two-sided test and paired-bootstrap interval are reported
1819 for the modern-subset Pytea head-to-head; the post-freeze unfiltered $N=15$ comparison
1820 reports Fisher exact p -values and is explicitly described as not separable at $\alpha=0.05$.

1821 **8. Experiments Compute Resources**

1822 Question: For each experiment, does the paper provide sufficient information on the com-
1823 puter resources needed to reproduce the experiments?

1824 Answer: [Yes]

1825 Justification: The full benchmark suite runs single-threaded on one CPU core in under a
1826 few minutes; the Lean development builds in under 10 seconds via `lake build`; no GPU
1827 is used and no preliminary or failed runs required additional compute.

1828 **9. Code Of Ethics**

1829 Question: Does the research conducted in the paper conform, in every respect, with the
1830 NeurIPS Code of Ethics?

1831 Answer: [Yes]

1832 Justification: The work is a static program-analysis tool involving no human subjects, no
1833 data collection, no model training, and no released model; it conforms to the NeurIPS Code
1834 of Ethics.

1835 **10. Broader Impacts**

1836 Question: Does the paper discuss both potential positive societal impacts and negative
1837 societal impacts of the work performed?

1838 Answer: [Yes]

1839 Justification: Section 6 notes that catching shape and gradient bugs statically reduces wasted
1840 GPU time and developer effort; no negative societal impact is foreseeable since the released
1841 artefact is a developer-facing static checker rather than a generative or decision-making
1842 system.

1843 **11. Safeguards**

1844 Question: Does the paper describe safeguards that have been put in place for responsible
1845 release of data or models that have a high risk for misuse?

1846 Answer: [NA]

1847 Justification: The paper releases only a static verifier and benchmark drivers; no pretrained
1848 model, generative model, or scraped dataset is released, so no misuse-prevention safeguards
1849 are required.

1850 **12. Licenses for existing assets**

1851 Question: Are the creators or original owners of assets used in the paper properly credited
1852 and are the license and terms of use explicitly mentioned and properly respected?

1853 Answer: [Yes]

1854 Justification: The PyTorch modules used as analysis targets (TORCHVISION, BSD-3; TIMM,
1855 Apache-2.0; TRANSFORMERS, Apache-2.0) are cited in the paper, used solely as analysis
1856 inputs, not redistributed, and their licenses are respected; PyTorch itself (2.9.1, BSD-style)
1857 and Pytea (MIT) are similarly cited.

1858 **13. New Assets**

1859 Question: Are new assets introduced in the paper well documented and is the documentation
1860 provided alongside the assets?

1861 Answer: [Yes]

1862 Justification: The released artefacts are the verifier source, the Lean operator-rule devel-
1863 opment, and the benchmark drivers, all under Apache-2.0; they are documented by an
1864 anonymised top-level README and inline docstrings shipped with the supplementary
1865 material. No model checkpoints or datasets are released.

1866 **14. Crowdsourcing and Research with Human Subjects**

1867 Question: For crowdsourcing experiments and research with human subjects, does the paper
1868 include the full text of instructions given to participants and screenshots, if applicable, as
1869 well as details about compensation (if any)?

1870 Answer: [NA]

1871 Justification: The paper does not involve crowdsourcing or any research with human subjects.

1872 **15. Institutional Review Board (IRB) Approvals or Equivalent for Research with Human**
1873 **Subjects**

1874 Question: Does the paper describe potential risks incurred by study participants, whether
1875 such risks were disclosed to the subjects, and whether IRB approvals were obtained?

1876 Answer: [NA]

1877 Justification: The paper does not involve human subjects, so no IRB approval or equivalent
1878 is required.

Table 9: Soundness scope per shape-transfer handler. The Lean side audits the rule *table*; the analyser implementation itself is not mechanised. *Lean-audited* rows have a Lean rule definition that is sorry-free and empirically diffed against PyTorch on 1,000 random samples; *pen-and-paper* rows have a soundness statement in the body but no Lean definition; *tested-only* rows are validated by property tests on PyTorch only.

Lean-audited (36 operators)		
Handler	Handler	Handler
matmul	bmm	batched_matmul
conv1d	conv2d	conv3d
conv_transpose2d	view	reshape
permute	transpose	expand
repeat	broadcast_to	cat
stack	split	chunk
unbind	gather	scatter
index_select	narrow	embed
layer_norm	rms_norm	scaled_dot_product_attention
linear	squeeze	unsqueeze
argmax	cross_entropy	dropout
contiguous	clamp	to

Pen-and-paper (13 handlers)		
Handler	Handler	Handler
elementwise_binary	reduce	einsum
flatten	softmax	relu
gelu	silu	tanh
sigmoid	detach	pad
where		

Note: `elementwise_binary` covers `add`, `sub`, `mul`, `div` (T-BROADCAST rule); `reduce` covers `sum`, `mean`, `max`, `min` (T-REDUCE rule). The structural reshape `flatten` reduces to `view` on a deterministic shape function (`squeeze/unsqueeze/dropout` have been promoted to Lean-audited this round). The activation family `relu/gelu/silu/tanh/sigmoid` plus `softmax/detach` are shape-preserving by Lemma EU on the elementwise lift. `pad` has a deterministic linear shape function and `where` reduces to broadcast (a 3-ary T-Broadcast instance). All thirteen carry a self-contained proof sketch in the reproducibility appendix and are checked by the same 1,000-sample property test as the Lean rules. The non-trivial case is `einsum`, whose explicit pen-and-paper soundness statement is given immediately below (Theorem 21); the remaining 12 pen-and-paper rules have shape functions that are either elementwise lifts (activation family, `detach`), strict shape-preservation (`flatten`), reductions over a fixed axis-set (`reduce`), or instances of broadcast (`elementwise_binary`, `where`).

Tested-only (34 handlers)		
Handler	Handler	Handler
batch_norm	group_norm	instance_norm
multihead_attention	conv_transpose1d	interpolate
pixel_shuffle	pixel_unshuffle	topk
glu	unfold	fold
repeat_interleave	masked_select	take_along_dim
roll_flip	adaptive_avg_pool	max_pool2d
avg_pool2d	linalg_svd	linalg_qr
linalg_solve	linalg_eig	rotary_embedding
einops_rearrange	stft	creation_ops
scatter_add	addmm	baddbmm
outer	tensordot	grid_sample
checkpoint		

Summary: 36 Lean-audited | 13 pen-and-paper | 34 tested-only | 83 total handlers.

Table 10: Pytea modern-subset per-bug contingency ($N=34$). Agreement a = both refute (25), b = TG-only refute (7), c = Pytea-only refute (0), d = neither refutes (2). McNemar exact two-sided $p=0.0156$ on $b=7, c=0$.

#	Bug	Primary op	TG	Pytea	Agreement
1	bug_003	Tensor.view	R	R	a
2	bug_004	Tensor.view	R	R	a
3	bug_005	broadcast (param+tensor)	R	R	a
4	bug_006	nn.CrossEntropyLoss	V	V	d
5	bug_007	F.conv2d (dtype)	V	V	d
6	bug_008	nn.Conv2d	R	R	a
7	bug_009	nn.Linear	R	R	a
8	bug_010	Tensor.view	R	R	a
9	bug_011	broadcast ($a+b$)	R	R	a
10	bug_014	Tensor.transpose	R	R	a
11	bug_015	nn.BatchNorm2d	R	R	a
12	bug_016	nn.Embedding	R	V	b
13	bug_018	matmul (@)	R	N/A	b
14	bug_019	Tensor.reshape	R	R	a
15	bug_020	torch.cat	R	R	a
16	bug_027	torch.bmm	R	R	a
17	bug_028	Tensor.view (-1)	R	R	a
18	bug_034	nn.Embedding	R	V	b
19	bug_035	nn.ConvTranspose2d	R	R	a
20	bug_037	flatten+view	R	R	a
21	bug_038	torch.stack	R	R	a
22	bug_039	F.softmax	R	V	b
23	bug_040	matmul (batched)	R	N/A	b
24	bug_042	nn.CrossEntropyLoss	R	R	a
25	bug_043	nn.MSELoss	R	R	a
26	bug_044	nn.NLLLoss	R	V	b
27	bug_045	nn.Conv2d	R	R	a
28	bug_047	Tensor.unsqueeze	R	R	a
29	bug_049	F.layer_norm	R	R	a
30	bug_053	Tensor.expand	R	R	a
31	bug_056	nn.Conv2d	R	R	a
32	bug_057	torch.mm	R	R	a
33	bug_063	Tensor.view (post-T)	R	V	b
34	bug_068	nn.MaxPool2d	R	R	a

Table 11: The 26 bugs excluded from the fragment-fair subset ($60 - 34 = 26$). Each row’s primary operator has no implementation in the Pytea 2022 TypeScript operator catalogue; including these bugs would penalise Pytea for being structurally inapplicable. Each exclusion is independent of TG’s verdict on the bug.

#	Bug	Primary op	Catalogue-miss reason
1	bug_001	F.scaled_dot_product_attention	SDPA: PyTorch-2.x, no handler
2	bug_002	torch.isclose	isclose: not in catalogue
3	bug_012	nn.MultiheadAttention	MHA-2.x redesign, no handler
4	bug_013	torch.einsum	einsum: no handler
5	bug_017	nn.Conv1d	Conv1d: only Conv2d catalogued
6	bug_022	torch.einsum	einsum: no handler
7	bug_024	nn.BatchNorm1d	BatchNorm1d: only BN2d catalogued
8	bug_026	nn.Conv3d	Conv3d: no handler
9	bug_029	torch.where	where: no handler
10	bug_031	torch.einsum	einsum: no handler
11	bug_032	torch.swapaxes	swapaxes: no handler
12	bug_033	nn.GroupNorm	GroupNorm: no handler
13	bug_041	torch.movedim	movedim: no handler
14	bug_048	torch.add (functional)	functional add: only tensor...add... catalogued
15	bug_050	torch.einsum	einsum: no handler
16	bug_051	torch.gather	gather: no handler
17	bug_052	Tensor.scatter_	scatter_: no handler
18	bug_054	nn.InstanceNorm2d	InstanceNorm: stub lacks num.features check
19	bug_055	F.embedding (functional)	functional embedding: not in stubs
20	bug_058	torch.split (list)	list-arg split: symbolic sum not evaluated
21	bug_059	torch.maximum (functional)	functional maximum: only tensor ops catalogued
22	bug_060	nn.Conv1d	Conv1d: only Conv2d catalogued
23	bug_064	torch.index_select	index.select: no handler
24	bug_065	torch.dot	dot: no handler
25	bug_067	torch.linalg.inv	linalg.*: no subpackage
26	bug_069	torch.repeat_interleave	repeat_interleave: no handler